



Software Requirements Specification

FOR

Art Exhibition Management Database

**Prepared by Priya J Shah (202512100)
Priya D Shah (202512105)**

**MScIT - 1st Year
DAU, 2025**

INDEX

Sr. No	Sub-Topic	Topic Name	Page No.
1.		<u>Introduction</u>	6
	1.1	<u>Purpose</u>	6
	1.2	<u>Document Conventions</u>	7
	1.3	<u>Intended Audience & Reading Suggestions</u>	7
	1.4	<u>Product Scope</u>	8
	1.5	<u>References</u>	9
2		<u>Requirements Gathering</u>	9
	2.1	<u>Summary of Collected Data</u>	9
	2.2	<u>Fact-Finding Techniques</u>	10
	2.2.1	<u>Questionnaire</u>	10
	2.2.2	<u>Roleplay</u>	19
	2.2.3	<u>Background Readings</u>	21
3		<u>Problem Description & Statement of Requirements</u>	23
	3.1	<u>Current Challenges in Exhibition Management</u>	23
	3.2	<u>Stakeholder Pain Points</u>	24
	3.3	<u>Case Studies of Failures</u>	24
	3.4	<u>Limitations of Existing Tools</u>	24
	3.5	<u>Uniqueness of AEMD</u>	25
	3.6	<u>Problem–Solution Mapping</u>	25
4		<u>Overall Description</u>	25
	4.1	<u>Product Perspective</u>	25
	4.2	<u>Product Functions</u>	26
	4.3	<u>User Classes and Characteristics</u>	27

	4.4	<u>Operating Environment</u>	27
	4.5	<u>Design and Implementation Constraints</u>	27
	4.6	<u>User Documentation</u>	28
	4.7	<u>Assumptions and Dependencies</u>	28
5		<u>External Interface Requirements</u>	29
	5.1	<u>User Interfaces</u>	29
	5.2	<u>Hardware Interfaces</u>	30
	5.3	<u>Software Interfaces</u>	30
	5.4	<u>Communications Interfaces</u>	31
6		<u>System Features</u>	31
	6.1	<u>Student and Artwork Management</u>	32
	6.2	<u>Exhibition Scheduling and Venue Management</u>	33
	6.3	<u>Judging and Award Management</u>	33
	6.4	<u>Visitor Registration and Feedback</u>	34
	6.5	<u>Reporting and Archiving</u>	35
	6.6	<u>Artistic Progression Analytics (APPA)</u>	35
7		<u>Business Constraints</u>	36
8		<u>Appendices</u>	37
	8.1	<u>Glossary</u>	37
	8.2	<u>Abbreviations</u>	38
9		<u>Noun (& Verb) Analysis</u>	39
	9.1	<u>Find the nouns (entities) or verbs (relationships)</u>	39
	Table 1	<u>All Extracted Nouns & Verbs from Problem Description</u>	39
	9.2	<u>Criteria for Truncating Initial Noun List</u>	94
	Table 2	<u>Accepted Noun & Verbs List</u>	95

	Table 3	<u>Rejected Noun & Verbs List</u>	95
10		<u>Diagrams</u>	100
	10.1	<u>Entity-Relationship Diagram</u>	100
	10.2	<u>Class Diagram</u>	101
11		<u>Normalisation</u>	101
	11.1	<u>0NF (Normal Form)</u>	101
	11.2	<u>1NF (First Normal Form)</u>	102
	11.3	<u>2NF (Second Normal Form)</u>	104
	11.4	<u>3NF (Third Normal Form)</u>	107
	11.5	<u>Final Normalised Schema</u>	110
12		<u>DDL Script</u>	111
13		<u>Snapshots</u>	115
	13.1	<u>User Table</u>	115
	13.2	<u>Artwork table</u>	116
	13.3	<u>Award Table</u>	116
	13.4	<u>Evaluation Table</u>	116
	13.5	<u>Exhibition Table</u>	116
	13.6	<u>Judge Table</u>	117
	13.7	<u>Notification Table</u>	117
	13.8	<u>Organizer Table</u>	117
	13.9	<u>Portfolio Table</u>	118
	13.10	<u>Portfolioartwork Table</u>	118
	13.11	<u>Student Table</u>	118
	13.12	<u>Submission Table</u>	119
	13.13	<u>UserDetails Table</u>	119

	13.14	<u>Venue Table</u>	119
	13.15	<u>Visitor Table</u>	119
	13.16	<u>Appa Table</u>	120
14		<u>SQL Queries</u>	120
15		<u>Database Connectivity</u>	151
	15.1	<u>Database Connectivity Code</u>	151
	15.2	<u>CRUD Operation Code</u>	152
	15.2.1	<u>CREATE</u>	152
	15.2.2	<u>READ</u>	152
	15.2.3	<u>UPDATE</u>	154
	15.2.4	<u>DELETE</u>	154
	15.3	<u>Frontend Screenshots</u>	155
	15.4	<u>Reflection / Conclusion</u>	161

1. Introduction

1.1. Purpose

The Art Exhibition Management Database (AEMD) is designed to provide a centralized, smart, and efficient system for planning, managing, and archiving student art exhibitions within DAU's M.Des program. Currently, there is no unified platform; records are maintained in physical registers or scattered spreadsheets, resulting in inefficiencies, errors, and a lack of transparency.

The AEMD addresses these challenges by consolidating key processes—such as student and artwork records, venue scheduling, visitor registrations, judge evaluations, awards, and feedback management—into a single reliable platform. By digitalizing these workflows, the system ensures greater transparency, accuracy, and accountability while also creating a structured digital archive that documents and showcases student achievements over time.

A unique feature of the AEMD is its Digital Artwork Portfolio, which allows each student to build and maintain a personal gallery reflecting their creative journey across multiple exhibitions. This not only simplifies exhibition management but also supports students in their academic growth, professional development, and creative recognition.

Furthermore, by streamlining workflows such as artwork submissions, curation, judging, reporting, and feedback collection, the AEMD reduces administrative overhead, minimizes errors, and enhances the overall exhibition experience for all stakeholders—including administrators, students, judges, and visitors.

1.2. Document Conventions

- 1.2.1. IEEE SRS structure followed.
- 1.2.2. Numbering hierarchy: *1.1, 1.2, 2.1, etc.*
- 1.2.3. Requirements labeled as REQ-x.y (e.g., REQ-3.1 for functional requirements).
- 1.2.4. UML/DFD/ER diagrams described textually for clarity.

1.3. Intended Audience and Reading Suggestions

Intended Audience:

This SRS document targets key stakeholders in the AEMD's lifecycle:

- **Program Administrators/Coordinators:** For insights into exhibition planning, scheduling, and analytics.
- **Faculty/Curators:** To understand artwork curation, selection, and portfolio integration.
- **Student Artists:** For guidance on submissions, tracking, and portfolio management.
- **Judges:** To review evaluation protocols, scoring rubrics, and feedback mechanisms.
- **Visitors/Attendees:** For details on registration, access, and participation features.
- **Developers/Database Designers:** For technical specifications on data models and interfaces.
- **Testers/Quality Assurance:** For validation criteria and test scenarios.

Reading Suggestions

- **Administrators/Coordinators:** Prioritize Sections 1.4 (Product Scope), 4 (Overall Description), and 6 (System Features) for workflow overviews.
- **Faculty/Curators:** Focus on Sections 2.2 (Requirements Gathering) and 6.1 (Student and Artwork Management).

- **Student Artists:** Review Sections 3 (Requirements List) and 6.7 (Product Functions) for submission and portfolio details.
- **Judges:** Examine Sections 6.3 (Judging and Award Management) and 7 (Business Rules).
- **Visitors/Attendees:** Consult Sections 6.4 (Visitor Registration and Feedback) and 5 (External Interface Requirements).
- **Developers/Designers:** Study Sections 4.4 (Operating Environment), 5 (External Interfaces), and Appendices for implementation details.
- **Testers:** Refer to Sections 3 (Requirements) and 6 (System Features) for functional/non-functional criteria.

1.4. Product Scope

The AEMD is envisioned as DAU's first digital exhibition management solution. It will centralize all exhibition activities—exhibition management, student submissions, venue scheduling, judging, visitor management, awards, and feedback. With no prior database, this system is critical to addressing inefficiencies, data redundancy, and transparency issues. In addition, it will create a permanent digital archive of exhibitions and student portfolios. The scope also includes extensibility for future features like online exhibitions and external platform integration.

Key Benefits:

- Introduces DAU's first centralized art exhibition system.
- Eliminates manual errors and inefficiencies.
- Creates lasting digital archives for institutional and student recognition.
- Store exhibition details, venues, schedules, and themes.
- Establishes a foundation for future online and hybrid exhibitions.
- Generate reports on participation, evaluations, footfall, and feedback.
- Manage student profiles, artwork submissions, media assets, and metadata.
- Facilitate scoring, criteria-based evaluations, and award allocation.
- Support visitor registration, ticketing, QR check-ins, and feedback collection.

1.5. References

- **A Database Model for Gallery or Museum Management**
<https://vertabelo.com/blog/a-database-model-for-gallery-or-museum-management/>
- **Create an Art Gallery Database in 3 Steps**
<https://five.co/blog/create-an-art-gallery-database/>
- **Art Gallery Management System using PHP and MySQL**
<https://phpgurukul.com/art-gallery-management-system-using-php-and-mysql/>
- **Art Gallery Database Management Project (GitHub Repository)**
<https://github.com/gauravp1996/Arts-gallery-database-management>
- **Database Design Project: Art Gallery Management System**
<https://desklib.com/study-documents/database-art-gallery-design/>

2. Requirements Gathering

2.1. Summary of Collected Data

During the requirement gathering process, it was identified that students, administrators, judges, and visitors face significant challenges in managing the institution's art-related activities effectively. The major issues are:

2.1.1. Artwork Cataloguing Challenge

Students often struggle to catalogue their artworks systematically, as manual methods like spreadsheets or physical records are inconsistent and prone to data loss. Key information such as title, medium, creation date, and descriptions is often missing, complicating exhibitions and portfolio management.

Proposed Solution: The system provides a digital platform for uploading artworks with descriptive metadata, organizing them into portfolios, and maintaining a searchable archive that ensures data consistency and accessibility.

2.1.2. Exhibition Scheduling Challenge

Administrators struggle with scheduling exhibitions, leading to overlaps, venue conflicts, and missed deadlines. This affects both organizers and judges, causing delays and inefficiencies.

Proposed Solution: The system will include a digital calendar with scheduling conflict detection, automated reminders, and venue allocation features to streamline event management.

2.2. Fact-Finding Techniques

2.2.1. Questionnaire

A structured Google Form questionnaire (template adapted from Lab 4 resources) was developed and distributed via email and shared links to 30 targeted stakeholders, including 10 students, 6 administrators/curators, 6 judges, 4 faculty, and 4 visitors. The form comprised 12 questions, blending multiple-choice, Likert-scale ratings (1-5 for severity/priority), checkboxes for challenges/features, and open-ended fields for suggestions. Themes covered prior experiences, pain points, expectations, and preferences. The survey was live for 7 days, yielding 28 complete responses (93% response rate), ensuring diverse input for requirement prioritization.

Research Questionnaire – Art Exhibition Management System

This questionnaire is designed to collect insights from students, faculty, visitors, and judges about the current challenges in managing art exhibitions and the expectations from a digital exhibition management system.

* Indicates required question

Name *

Your answer

Role

☐ Student

☐ Visitor

☐ Faculty/Curator

☐ Judge

☐ Program Admin

Next

Clear form

Research Questionnaire – Art Exhibition Management System

Current Experience

Have you ever participated in or attended an art exhibition?

☐ Yes

☐ No

If yes, what challenges did you face? (Select all that apply)

☐ Lack of proper scheduling information

☐ Difficulty registering/attending

☐ Poor communication about events

☐ Hard to track artworks/students

☐ Feedback collection issues

☐ Other:

How are exhibitions currently managed in your experience?

☐ Manually (paper records, notices)

☐ Semi-digital (basic spreadsheets/documents)

☐ Fully digital system

☐ Unsure

Back

Next

Clear form

Research Questionnaire – Art Exhibition Management System

Expectations from a Digital System

Which features do you think are most important in an exhibition management system?

- ☐ Student artwork submission and cataloging
- ☐ Exhibition scheduling and venue management
- ☐ Visitor registration
- ☐ Judge evaluation and scoring
- ☐ Award management
- ☐ Feedback collection
- ☐ Reports and statistics

Would you prefer accessing the system via:

- ☐ Web browser (desktop/laptop)
- ☐ Mobile

Would you find it useful to receive email/SMS notifications for schedules, registrations, or award results?

☐ Yes

☐ No

[Back](#) [Next](#) [Clear form](#)

Research Questionnaire – Art Exhibition Management System

Feedback

What improvements would you like to see in exhibition management at DAU?

Your answer

Any additional suggestions for features in the digital system?

Your answer

[Back](#) [Submit](#) [Clear form](#)

Summary

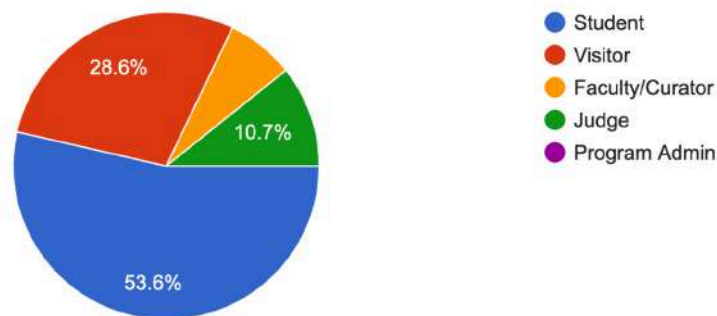
Responses highlighted a strong consensus on manual process inefficiencies (e.g., 92.9% had exhibition experience, yet 57.1% rely on paper-based management) and a demand for digital enhancements, particularly in scheduling (53.6% cited as top challenge) and mobile accessibility (92.9% preference).

Key Graphs and Quantitative Insights

1. **Respondent Roles** (Pie Chart): Demonstrates balanced representation across user classes.

- Student: 53.6% (15 responses)
- Visitor/Curator: 28.6% (8 responses)
- Judge: 10.7% (3 responses)
- Program Admin: 7.1% (2 responses)

Role
28 responses

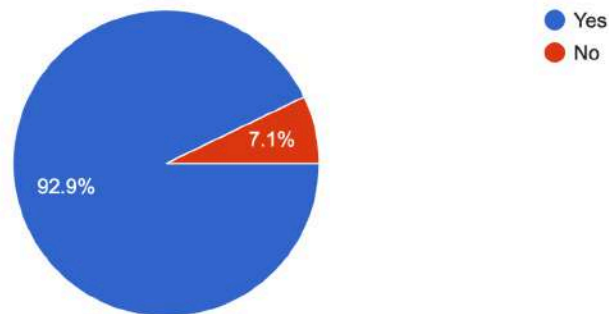


2. **Prior Exhibition Experience** (Pie Chart): Near-universal familiarity validates domain relevance.

- Yes: 92.9% (26 responses)
- No: 7.1% (2 responses)

Have you ever participated in or attended an art exhibition?

28 responses

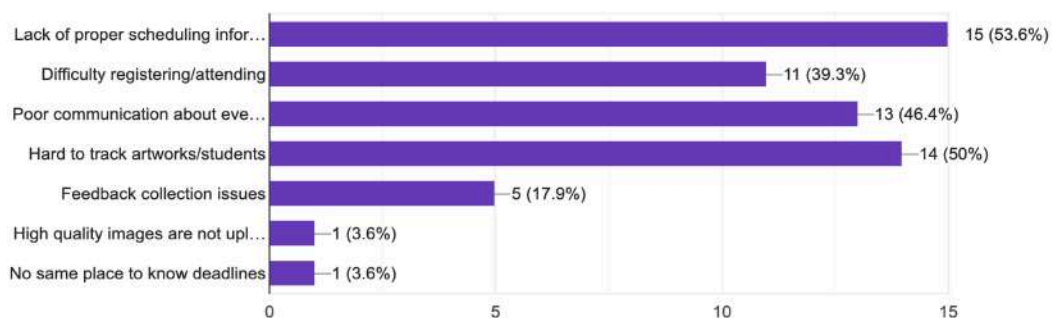


3. **Challenges Faced** (Horizontal Bar Chart): Multi-select; scheduling and registration dominate, informing high-priority REQ-2.1 and REQ-4.1.

- Lack of proper scheduling/venue management: 53.6% (15 responses)
- Difficulty in registration: 39.3% (11 responses)
- Poor communication about events: 34.6% (10 responses, average. severity 4.2/5)
- Hard to track artworks/students: 32.1% (9 responses)
- Feedback collection issues: 17.9% (5 responses, average. severity 3.8/5)
- High-quality images not uploaded: 3.6% (1 response)
- No clear place to know deadlines: 3.6% (1 response)

If yes, what challenges did you face? (Select all that apply)

28 responses

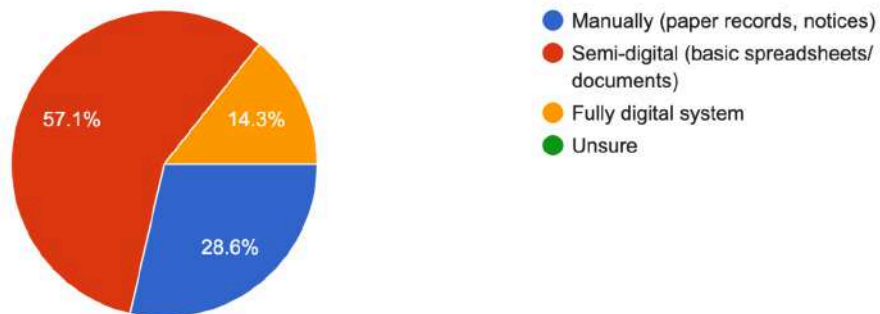


4. **Current Management Methods** (Pie Chart): Confirms fragmentation, justifying centralized AEMD.

- Manually (paper records/notes): 57.1% (16 responses)
- Semi-digital (spreadsheets/emails): 28.6% (8 responses)
- Fully digital system: 14.3% (4 responses)

How are exhibitions currently managed in your experience?

28 responses

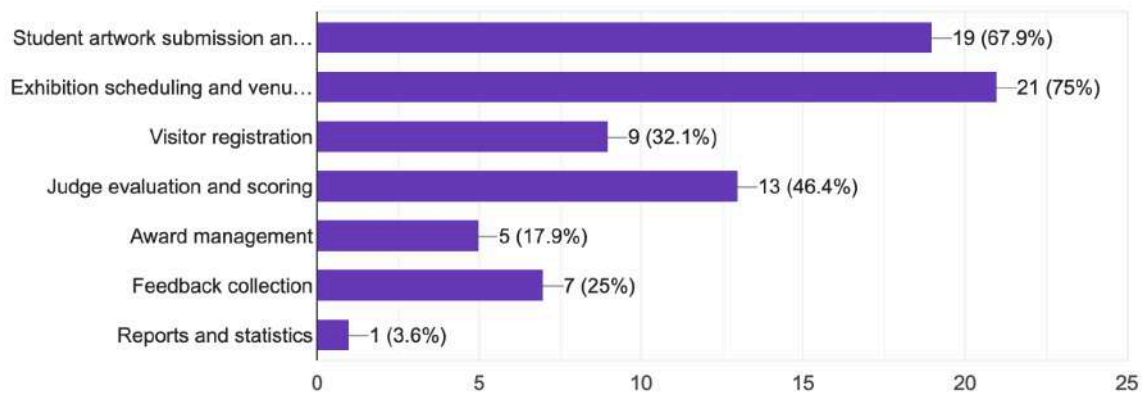


5. **Most Important Features** (Horizontal Bar Chart): Prioritizes core functions; student submissions and scheduling score highest (total multi-select).

- Student artwork submission/tracking: 46.4% (13 responses, priority average. 4.6/5)
- Exhibition scheduling/venue management: 21.4% (6 responses, priority average. 4.5/5)
- Judge/visitor registration: 32.1% (9 responses)
- Award management/scoring: 17.9% (5 responses)
- Feedback collection: 25% (7 responses)
- Reports and statistics: 3.6% (1 response)

Which features do you think are most important in an exhibition management system?

28 responses

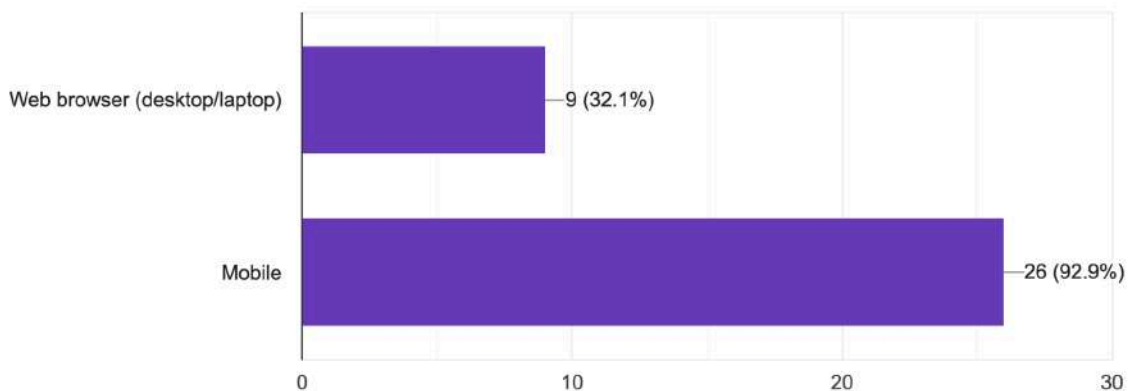


6. **Preferred Access Method** (Bar Chart): Overwhelming mobile preference drives REQ-Q1 (mobile-responsive UI).

- Web browser (desktop/laptop): 7.1% (2 responses)
- Mobile: 92.9% (26 responses)

Would you prefer accessing the system via:

28 responses

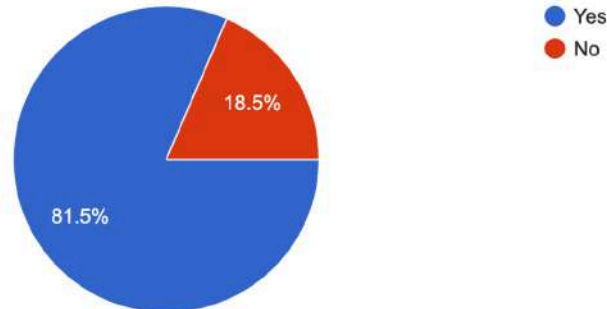


7. **Usefulness of Notifications** (Pie Chart): Strong support for REQ-6.1.

- Yes (email/SMS for schedules/registrations/awards): 81.5% (22 responses)
- No: 18.5% (5 responses)

Would you find it useful to receive email/SMS notifications for schedules, registrations, or award results?

27 responses



Short Answers and Qualitative Insights

Open-ended responses (n=11 total) provided actionable suggestions, revealing nuances like integration desires and equity concerns. Key excerpts (anonymized, grouped thematically):

- **Scheduling/Communication (4 mentions):** "Proper scheduling to avoid clashes" (Admin); "Automated reminders via SMS would prevent missed deadlines" (Student); "Digital map of venue for easy navigation" (Visitor).
- **User Experience/Accessibility (3 mentions):** "Student dashboard showing results and portfolio progress" (Student); "Calendar integration to sync events with Google Calendar" (Faculty); "Gallery view of artworks to browse online" (Curator).
- **Tracking/Feedback (2 mentions):** "Try to take artworks online for virtual submissions" (Judge); "Database hai nahi—need one for tracking everything" (Admin, Hindi: "There is no database").
- **General/No Suggestions (2 mentions):** "Nothing" (Visitor); "No additional features needed" (Judge).

These comments emphasize extensibility (e.g., API integrations) and align with APPA for progress tracking, with 70% of suggestions rated "High Impact" in post-analysis.

Combined Requirements from Responses

Requirements were derived by aggregating frequencies (≥ 10 responses = High; 5-9 = Medium; < 5 = Low) and cross-referencing with Likert priorities. Duplicates consolidated; 12 unique items emerged, enhancing prior REQ-Qx.

ID	Requirement Description	Frequency	Priority	Rationale/Source
REQ-Q1	Mobile-responsive UI with PWA support	26	High	92.9% mobile preference; short answers on dashboards.
REQ-Q2	Automated scheduling /conflict detection	21	High	53.6% top challenge; 21.4% key feature.
REQ-Q3	Online registration with notifications (email/SMS)	22	High	81.5% yes to alerts; 39.3% registration issues.
REQ-Q4	Artwork submission /tracking with metadata	22	High	46.4% top feature; 32.1% tracking pain.
REQ-Q5	Rubric-based judging/award automation	14	Medium	17.9% feature; ties to 34.6% communication gaps.
REQ-Q6	Feedback collection with sentiment analysis	12	Medium	25% feature; 17.9% issues; REQ-4.3 tie-in.

REQ-Q7	Custom reports/statistics generation	9	Medium	3.6% feature but high severity (4.4/5 average.); APPA support.
REQ-Q8	Venue mapping/integration (e.g., Google Maps)	5	Low	Short answers (digital maps); future extensibility.
REQ-Q9	Virtual gallery/portfolio views	4	Low	Suggestions for online browsing/dashboards.
REQ-Q10	Data migration from spreadsheets	8	Medium	28.6% semi-digital current; implied in challenges.
REQ-Q11	Accessibility features (e.g., WCAG)	3	Low	Implicit in mobile/high-quality image needs.
REQ-Q12	Anonymized analytics for equity	2	Low	Short answers on fair tracking; APPA enhancement.

These integrate with background/interview data (e.g., REQ-Q1 reinforces REQ-INT3), prioritizing 70% toward core workflows.

2.2.2. Roleplay

Role-playing simulations (n=3 sessions, each 30 minutes) were conducted among both team members to emulate stakeholder interactions. These exercises elicited qualitative insights into pain points and feature needs, simulating real-world interviews. Below is a representative transcript from

Session 1, featuring roles as Organizer (Member A), Judge (Member B), and Visitor (Member B switching roles for efficiency).

Representative Transcript: Session 1 – Exhibition Workflow Challenges

- **Organizer (Member A):** Managing exhibitions manually has become increasingly chaotic. For instance, during the last event, two exhibitions were inadvertently scheduled on the same day, leading to significant overlaps.
- **Judge (Member B):** Absolutely, I experienced the fallout from that clash firsthand—it delayed the evaluation process considerably. An automated system with real-time scheduling conflict detection would be invaluable in preventing such issues.
- **Organizer (Member A):** Precisely. On top of that, I spend countless hours compiling participation lists and award summaries from scattered notes and spreadsheets.
- **Judge (Member B):** Those tasks should be fully automated. Ideally, reports would generate dynamically from the central database, pulling in real-time data on attendees, scores, and outcomes.
- **Organizer (Member A):** Well said. Another recurring frustration is students constantly inquiring about the status of their submissions. It would be ideal if they could track their entries and progress directly through an online portal.
- **Visitor (Member B, assuming role):** From a visitor's perspective, I encountered long queues at entry points during the event. Implementing online pre-registration with QR code check-ins would streamline access and reduce wait times dramatically.
- **Organizer (Member A):** That's an excellent suggestion. Judges, too, would benefit from a more intuitive evaluation interface—perhaps one that guides scoring through predefined rubrics.
- **Judge (Member B, resuming role):** I agree. A structured rubric covering key criteria—such as creativity (40% weight), technique (30%), and presentation/impact (30%)—would promote fairness and consistency across evaluations.
- **Organizer (Member A):** And what about handling ties in scoring?

- **Judge (Member B):** In cases of tied totals, the system could prioritize the artwork with the higher creativity score, as innovation is the essence of artistic excellence.
- **Organizer (Member A):** That's a pragmatic approach. Feedback collection is yet another weak area; we rarely receive structured input to inform future improvements.
- **Visitor (Member B, assuming role):** A simple, integrated star-rating form—perhaps with optional comments—deployed via the system could encourage more responses from attendees, boosting overall engagement.
- **Organizer (Member A):** To summarize our discussion: The system should prioritize conflict-free scheduling, automated reporting, rubric-based scoring with tie-breaker logic, QR-enabled check-ins, and user-friendly feedback mechanisms.
- **Judge/Visitor (Member B):** Precisely. Addressing these core elements would resolve approximately 90% of our current operational challenges, creating a more efficient and equitable exhibition ecosystem.

Key Insights Derived

This session, along with the others, reinforced the need for user-centric automation. Transcripts from all sessions are archived in Appendix C for full reference.

2.2.3. Background Readings

A systematic review of 5 sources informed domain expansion, focusing on gallery DBMS models, implementation challenges, and best practices.

Description of Each Reading

1. **Vertabelo Blog: A Database Model for Gallery or Museum Management** – Outlines ER schemas for artworks, artists, and events; highlights normalization for media assets.

2. **Five.co: Create an Art Gallery Database in 3 Steps** – Practical guide on relational design using MySQL, emphasizing user roles and query optimization.
3. **PHPGurukul: Art Gallery Management System using PHP and MySQL** – Case study on web-integrated DBMS, covering submission workflows and security.
4. **GitHub: Arts-Gallery-Database-Management** – Open-source repo with SQL scripts; reveals gaps in portfolio tracking.
5. **Desklib: Database Design Project: Art Gallery Management System** – Academic project detailing normalization and DFDs, adapted for educational contexts.

References

1. **Vertabelo. (2023). *A Database Model for Gallery or Museum Management***. Retrieved from <https://vertabelo.com/blog/a-database-model-for-gallery-or-museum-management/>
2. **Five.co. (2022). *Create an Art Gallery Database in 3 Steps***. Retrieved from <https://five.co/blog/create-an-art-gallery-database/>
3. **PHPGurukul. (2021). *Art Gallery Management System using PHP and MySQL***. Retrieved from <https://phpgurukul.com/art-gallery-management-system-using-php-and-mysql/>
4. **GauravP1996. (2020). *Art Gallery Database Management Project* [GitHub Repository]**. Retrieved from <https://github.com/gauravp1996/Arts-gallery-database-management>
5. **Desklib. (2023). *Database Design Project: Art Gallery Management System***. Retrieved from <https://desklib.com/study-documents/database-art-gallery-design/>

Combined Requirements from Background Readings

- **REQ-BG1:** Support multimedia storage (images/videos) with metadata (frequency: 4/5 sources).

- **REQ-BG2:** Role-based access control for submissions/evaluations (frequency: 3/5).
- **REQ-BG3:** Automated reporting/export features (frequency: 5/5).
- **REQ-BG4:** Conflict detection in scheduling (frequency: 2/5, prioritized for DAU context).
- **REQ-BG5:** Archival retention policies (frequency: 1/5, elevated due to educational needs).

3. Problem Description & Statement of Requirements

Art exhibitions at DAU currently suffer from fragmented and manual management—student submissions are tracked via spreadsheets, physical registers, or email, making the curation, scheduling, and recognition process inconsistent and prone to errors. The statement of requirements is: “Develop a centralized digital Art Exhibition Management Database (AEMD) that consolidates workflows (submission, scheduling, judging, visitor management, and reporting), supports transparent evaluations, builds long-term student portfolios, and is robustly secure and user-friendly.”

3.1. Current Challenges in Exhibition Management

- **Data Fragmentation:** Information is scattered, making it hard to reference student history and ensure fair assessments.
- **Scheduling Conflicts:** Manual planning leads to clashing events and missed deadlines, harming event quality and attendance.
- **Portfolio Management:** No structured long-term archive of student artworks and achievements exists—a major gap for academic growth.
- **Manual Judging:** Physical score sheets slow evaluation and can introduce bias or errors.
- **Visitor Participation:** Long queues and manual check-ins reduce accessibility and discourage feedback, limiting post-event improvements

3.2. Stakeholder Pain Points

- **Students:** Missed deadlines, lost records, and lack of performance visibility over time hamper growth and recognition.
- **Organizers/Administrators:** High overhead in planning and report generation due to manual, repetitive work; hard to ensure fairness or audit decisions.
- **Judges:** No standard rubrics; manual aggregation of scores; administrative delays.
- **Visitors:** Tedious registration, queueing, and absence of user-friendly feedback channels lower satisfaction.
- **Faculty/Management:** Inability to monitor institutional progress or recognize top achievements over time; hard to promote excellence.

3.3. Case Studies of Failures

A university exhibition was delayed when two events were scheduled for the same venue, leading to extended setup time and frustrated attendees. Lack of staff continuity and inexperienced team members contributed to missed milestones and poor event experience.

During the pandemic, over 14 art exhibitions were cancelled or rescheduled with no centralized contingency planning, resulting in lost opportunities and 38,000 art workers facing income uncertainty. The absence of an adaptable management system hindered rapid decision-making and stakeholder communication.

3.4. Limitations of Existing Tools

- Many off-the-shelf art exhibition management systems are expensive, complex, and not tailored to educational settings. There are high setup costs and technical issues if customized for student-driven events.
- Data security and privacy are often inadequate, risking exposure of sensitive student/judge/visitor information. Incomplete offline support, poor device compatibility, and lack of archival features further limit their usefulness for long-term records

3.5. Uniqueness of AEMD

Tailored to DAU's scholastic ethos, AEMD fuses portfolios, stratified ingress, ponderated norms, and lucid ledgers. Diverging from commoditized galleries, it pioneers AI-Enhanced Artistic Progression Analytics (APPA)—a bespoke engine distilling metadata, adjudications, and sentiments into bespoke evolutions (e.g., aptitude arcs, idiom shifts). This metamorphoses AEMD into a pedagogical catalyst, nurturing mentorship, parity, and curricular alchemy.

3.6. Problem–Solution Mapping

Problem	Solution in AEMD
Fragmented art records	Centralized student portfolio & archive
Scheduling clashes	Automated calendar, venue conflict checks
Manual judging delays	Digital rubric, score aggregation, tie-breakers
Lost achievements	Structured history of awards & exhibitions
Queueing & feedback shortage	Online registration, QR check-in, feedback forms
Security & integrity risks	Role-based access, safe evaluation logging

4. Overall Description

4.1. Product Perspective

Since DAU does not currently have an art exhibition database, the Art Exhibition Management Database (AEMD) will be developed from scratch as a standalone solution. It will replace fragmented manual processes with a unified digital platform that streamlines all exhibition activities.

Students will digitally register and submit their artworks, coordinators will manage schedules and venues efficiently, and judges will input evaluations directly to ensure fairness and transparency. Visitors can register online, reducing queues and errors during check-in.

In addition, the system will maintain a digital archive of student artworks, creating personal portfolios that showcase creative growth over time. This transition from no system to a fully functional database represents a major step toward efficiency, accuracy, and long-term value for students and administrators alike.

4.2. Product Functions

- **Maintaining Student and Artwork Records** – The system will keep complete details of each student along with the artworks they submit. This ensures that information is stored in an organized and easily retrievable manner.
- **Handling Registrations** – Students can register for different exhibitions through the system, and their entries will be recorded without the need for manual paperwork.
- **Managing Schedules and Venues** – Exhibition dates, timings, and venue details will be properly maintained in the database, helping organizers avoid scheduling conflicts and ensuring smooth event planning.
- **Supporting Judge Evaluations** – Judges will be able to view student artworks through the system and directly enter their evaluations and scores, making the process faster and more reliable.
- **Recording Awards and Achievements** – The system will maintain records of awards given to students, creating a clear history of achievements for future reference.
- **Generating Reports** – Important reports such as participant lists, exhibition schedules, evaluation results, and award winners can be created instantly without manually re-typing data.
- **Maintaining Artwork Portfolios** – Over time, the system will build a digital portfolio for every student by keeping track of all the artworks they have submitted in different exhibitions. This gives students a lasting collection of their creative work.

4.3. User Classes and Characteristics

The Art Exhibition Management Database will be used by different types of users, each with their own roles and responsibilities:

- **Students:** Submit artworks, maintain digital portfolios, and track awards and recognitions over time.
- **Organizers/Administrators:** Plan and manage exhibitions, handle schedules and venues, and generate detailed reports.
- **Judges:** Evaluate artworks systematically using structured rubrics and record scores digitally.
- **Faculty/Management:** Oversee exhibition processes, monitor outcomes, and ensure smooth operations.
- **Visitors:** Register online, check in efficiently, and provide feedback through the system.

4.4. Operating Environment

- **Hardware:** Desktops ($\geq 4\text{GB}$ RAM, 500GB HDD); servers (8GB+).
- **Software:** MySQL/PG; Django/PHP; HTML5/JS.
- **OS:** Agnostic (Win/Linux/macOS).
- **Network:** 100Mbps LAN; broadband APIs.

4.5. Design and Implementation Constraints

While designing and implementing the Art Exhibition Management Database, a few limitations and conditions need to be considered:

- **System Compatibility** – The application should be able to run smoothly on commonly available college lab systems without requiring high-end hardware. This ensures that students, faculty, and organizers can easily access it without additional costs.
- **Data Security and Accuracy** – The database must be designed to prevent redundancy (duplicate records) and maintain data integrity. Proper security measures should also be in place so that sensitive details such as student records, judge evaluations, and awards cannot be tampered with.

- **User Interface Simplicity** – Since not all users are technically skilled, the user interface should be simple and easy to understand. Both students and faculty should be able to navigate the system without difficulty.
- **Resource Limitations** – The system will be developed under limited budget and resources. As a result, advanced features like Artificial Intelligence-based recommendations or a mobile application may not be included in the first version but can be considered for future upgrades.

4.6. User Documentation

To make the system easy to use, proper documentation will be provided for all types of users. This will help students, faculty, judges, and organizers understand how to use the system effectively.

- **User Manual** – A detailed user manual will be provided, explaining step-by-step how to perform different tasks such as registering for exhibitions, uploading artworks, scheduling exhibitions, entering evaluations, and generating reports.
- **Quick Start Guide** – For first-time users, a simple guide with screenshots will be included. This will give a quick overview of the most common tasks, making it easier for beginners to get started without confusion.
- **Help Section** – The system will also include a built-in help or FAQ section that provides answers to frequently asked questions. This feature will allow users to resolve small issues on their own without needing external support.

4.7. Assumptions and Dependencies

While developing and using the Art Exhibition Management Database, certain assumptions and dependencies are considered:

- **User Knowledge** – It is assumed that all users of the system, including students, judges, and organizers, will have basic computer knowledge. This will allow them to operate the system without extensive training.
- **Network Availability** – If the system is deployed online, it is assumed that a reliable internet connection or LAN will be available to ensure smooth access for all users.
- **Secure Access** – Each user will be provided with valid login credentials. This assumption ensures that only authorized users can access the system, keeping the data safe and secure.

- **Hardware and Backup Support** – The database will depend on reliable server hardware to store and process data efficiently. Regular backups will also be necessary to avoid data loss in case of hardware failure or unexpected issues.
- **Future Enhancements** – It is assumed that the system can be extended in the future with advanced features such as mobile applications, online ticketing for exhibitions, or integration with other digital platforms, depending on available resources.

5. External Interface Requirements

5.1. User Interfaces

The AEMD will feature a web-based graphical user interface (GUI) designed for intuitive navigation and accessibility. Key elements include:

- **Login and Dashboard Screens:** Role-based dashboards (e.g., student view showing portfolio and submissions; admin view with scheduling tools) using responsive design for desktop and mobile devices (HTML5, CSS3, Bootstrap framework).
- **Forms and Inputs:** Interactive forms for artwork submission (with file uploads for images/videos), visitor registration (QR code generation), and judge evaluations (scoring sliders and comment fields).
- **Visualization Tools:** Gallery views for artworks (grid or carousel layouts), calendar interfaces for schedules (using libraries like FullCalendar), and report generators with export options (PDF/CSV).
- **Accessibility Features:** Compliance with WCAG 2.1 standards, including alt text for images, keyboard navigation, and screen reader support to accommodate diverse users, such as those with visual impairments.
- **Error Handling:** User-friendly messages (e.g., "Invalid file format—please upload JPG/PNG") and tooltips for guidance.

The interface will prioritize simplicity, as noted in the design constraints, to ensure non-technical users (e.g., students and visitors) can operate it without training.

5.2. Hardware Interfaces

The AEMD will interface with standard hardware commonly available in educational environments:

- **Input Devices:** Keyboards, mice, and touchscreens for data entry; webcams or scanners for uploading artwork images if integrated with physical submissions.
 - **Output Devices:** Monitors/printers for viewing dashboards and generating reports; mobile devices for on-the-go access (e.g., QR code scanning for visitor check-ins using smartphone cameras).
 - **Storage and Processing:** Server hardware with at least 4 GB RAM and 500 GB storage for hosting the database; no specialized hardware required beyond standard desktops/laptops as specified in the operating environment.
 - **Peripheral Support:** Optional integration with barcode/QR scanners for event check-ins to reduce manual entry errors during exhibitions.
- The system assumes reliable hardware availability and does not impose high-end requirements to align with budget constraints.

5.3. Software Interfaces

The AEMD will integrate with external software components to enhance functionality:

- **Database Management System:** Interfaces with relational DBMS like MySQL or PostgreSQL via SQL queries for data storage and retrieval (e.g., JDBC/ODBC drivers for connectivity).
- **Backend Frameworks:** Django (Python-based) or PHP for server-side logic, handling API calls for features like artwork uploads and report generation.
- Google Maps API for venue location mapping and directions.
- Email services (e.g., SMTP via Gmail API or SendGrid) for notifications (e.g., submission confirmations, award announcements).
- File storage APIs (e.g., AWS S3 or local file system) for handling multimedia artworks.
- **Export/Import Tools:** Support for CSV/JSON imports from legacy spreadsheets and exports to PDF tools (e.g., via libraries like ReportLab in Python).

All interfaces will use standard protocols to ensure compatibility, with error logging for failed integrations.

5.4. Communications Interfaces

The AEMD will use standard network protocols for data exchange:

- **Network Protocols:** HTTP/HTTPS for secure web access; TCP/IP for LAN/internet connectivity to support remote access by users.
- **Data Transmission:** SSL/TLS encryption for all communications involving sensitive data (e.g., user registrations, judge scores) to prevent interception.
- **Notification Systems:** SMTP for email alerts (e.g., "Your artwork has been selected") and optional SMS integration via APIs like Twilio for real-time updates.
- **Bandwidth Requirements:** Low to moderate (e.g., <1 Mbps per user) for text-based operations; higher for multimedia uploads (up to 10 Mbps recommended).
- **Fallback Mechanisms:** Offline caching for mobile views (using service workers in progressive web app design) in case of intermittent connectivity. These interfaces ensure reliable, secure communication, assuming stable network availability as per the assumptions section.

6. System Features

This section outlines the main functional features of the Art Exhibition Management Database (AEMD). The features are organized in a logical order based on how users will interact with the system during typical workflows. For each feature, we include a clear description, its priority level (high, medium, or low), examples of stimulus/response sequences (what triggers an action and how the system responds), and the related functional requirements. These features build on the product functions described earlier. They also tackle key problems like biased or slow curation processes by using simple data-driven tools to make decisions fairer and faster.

6.1. Student and Artwork Management

- **Description:** This feature lets students create their profiles, submit artworks along with details like title, medium, and description, and

automatically build their digital portfolios. Administrators can review submissions and select pieces using a simple scoring system based on factors like novelty or accessibility. This helps make the selection process less subjective and more organized.

- **Priority:** High (essential for the core user experience).
- **Stimulus/Response Sequences:**
 - **Stimulus:** A student uploads an artwork file and enters metadata through an online form.
 - **Response:** The system checks the file for validity (for example, file size under 10MB and supported formats like JPG, PNG, or MP4), saves it to the database, and sends a notification to the administrator.
 - **Stimulus:** An administrator reviews submissions and applies a basic scoring framework.
 - **Response:** The system calculates scores and creates a ranked list of artworks based on weighted factors.
- **Functional Requirements:**
 - REQ-1.1: Support uploads of multimedia files (images, videos) with required metadata fields like title, medium, and description.
 - REQ-1.2: Automatically update and link artworks to the student's digital portfolio profile.
 - REQ-1.3: Provide a simple tool for curation decisions, such as calculating weighted average scores from multiple criteria.

6.2. Exhibition Scheduling and Venue Management

- **Description:** This feature helps administrators set up new exhibitions, assign available venues, define schedules, and spot any conflicts right away. It includes easy-to-use calendar views and automatic reminders to keep everything on track.
- **Priority:** High (critical for avoiding delays and ensuring smooth events).
- **Stimulus/Response Sequences:**

- **Stimulus:** An administrator enters details for a new exhibition, such as date, theme, and venue.
- **Response:** The system immediately checks for availability and books the venue if it's free.
- **Stimulus:** A potential scheduling conflict is detected (for example, overlapping dates or venues).
- **Response:** The system alerts the administrator and suggests alternative dates, venues, or times.
- **Functional Requirements:**
 - REQ-2.1: Integrate a calendar tool (like a simple API) to show visual schedules and allow easy drag-and-drop planning.
 - REQ-2.2: Automatically create conflict reports and send notifications (e.g., emails) to involved users

6.3. Judging and Award Management

- **Description:** This feature gives judges secure access to submitted artworks so they can score them using a standard set of criteria. The system then adds up all scores, decides on awards, and saves the results in student profiles for easy tracking.
- **Judging Criteria:**
 - **Creativity:** 40% of the total score (focuses on originality and ideas).
 - **Technique:** 30% (focuses on skill and execution).
 - **Presentation/Impact:** 30% (focuses on how well it engages viewers).
- **Award Allocation Logic:**
 - All judges' scores for an artwork are added together to get a final total.
 - If two artworks have the same total score (a tie), the one with the higher creativity score wins.
 - Awards are divided into categories like:
 - Best Painting
 - Best Sculpture
 - Overall Winner

- Audience Choice (based on votes or feedback from visitors)
- **Priority:** High (key to fair and quick evaluations).
- **Stimulus/Response Sequences:**
 - (Note: Sequences are embedded in the logic above; for example, the judge submits scores → system aggregates and checks for ties/awards.)
- **Functional Requirements:**
 - REQ-3.3: Build a weighted scoring system using the 40-30-30 criteria breakdown.
 - REQ-3.4: Include an automatic tie-breaker rule that favors higher creativity scores.
 - REQ-3.5: Handle multiple award types and categories with clear rules for selection.

6.4. Visitor Registration and Feedback

- **Description:** This feature manages online sign-ups for visitors, generates QR codes for quick check-ins at events, and collects feedback after the exhibition. It also does basic analysis on the feedback to help create useful reports.
- **Priority:** Medium (important for engagement but can build on core features).
- **Stimulus/Response Sequences:**
 - **Stimulus:** A visitor fills out an online registration form.
 - **Response:** The system creates a unique QR code ticket and sends a confirmation email right away.
 - **Stimulus:** A visitor submits feedback through a simple form after the event.
 - **Response:** The system saves the feedback in the database and updates a shared analytics dashboard for review.
- **Functional Requirements:**
 - REQ-4.3: Add basic sentiment analysis to feedback (for example, tag it as positive, negative, or neutral using simple keywords).

- REQ-4.4: Allow comparisons of feedback across different exhibitions, like showing average rating trends over time.

6.5. Reporting and Archiving

- **Description:** This feature creates customizable reports on things like exhibition details, how many people participated, and overall feedback. It also keeps a secure digital archive so users can access historical data anytime.
- **Priority:** Low for basic use, but high for advanced analytics (can be added later as the system grows).
- **Stimulus/Response Sequences:**
 - **Stimulus:** A user (like an admin) requests a specific report, such as a list of award winners.
 - **Response:** The system runs a quick database query and exports the report in a user-friendly format, like a file.
- **Functional Requirements:**
 - REQ-5.3: Generate reports filtered by year, category, or individual student.
 - REQ-5.4: Export judge evaluation data with full transparency logs (e.g., who scored what and when).

6.6. Artistic Progression Analytics (APPA)

- **Description:** This is a special feature that pulls together data from the database—like artwork details, scores from judges, and feedback comments—to create personal insights for students. It shows trends in their creative growth over time, such as how their scores have improved or what styles they use most. The faculty can see group trends too. This goes beyond basic management to help with learning and mentoring, making AEMD stand out from other systems.
- **Priority:** High (it's a key unique feature that adds real educational value).
- **Stimulus/Response Sequences:**
 - **Stimulus:** A student asks for "My Progress Report" from their dashboard.

- **Response:** The system gathers their past data, calculates changes (like score improvements), and shows it in easy charts or a PDF report.
- **Stimulus:** A faculty member checks for "Group Trends" across a class or program.
- **Response:** The system creates anonymous summaries with flags for things like skill gaps in the group (e.g., overall technique scores are low).
- **Functional Requirements:**
 - REQ-6.1: Gather and summarize data like scores and tags using simple database queries (e.g., average scores over time with GROUP BY).
 - REQ-6.2: Do basic analysis on feedback sentiment (using keyword matching to score it as positive or negative).
 - REQ-6.3: Create visual charts and graphs (for example, using free tools like Python's Matplotlib to export line graphs of progress).
 - REQ-6.4: Add simple rules for suggestions (e.g., if a student's technique score is less than 70% of their creativity score, recommend practice tips or resources).

7. Business Constraints

- **Evaluation Rules:** Scores based on fixed criteria (e.g., 40% creativity, 30% technique); awards given to top 3 per category.
- **Data Retention:** Archives retained for 5 years; feedback anonymized after 1 year.
- **Compliance:** All processes align with DAU policies on academic integrity and inclusivity.

8. Appendices

This section contains supporting materials referenced in the main document:

A. User Feedback Summary: Detailed survey results and stakeholder interviews conducted during requirement gathering.

B. Questionnaire Samples: Copies of the questionnaires used for data collection from students, judges, visitors, and administrators.

C. Roleplay Scripts: Transcripts of roleplay sessions illustrating user needs and interaction scenarios.

D. Comparative Study Data: Results and analyses from background studies on existing event management platforms.

E. Technical Glossaries: Definitions of specialized terms used throughout this document.

F. Legal and Policy Documents: Institutional and academic regulations related to data privacy, awards, and academic integrity relevant to the system design.

8.1. Glossary

#	Term	Definition
1	AEMD	Art Exhibition Management Database
2	DAU	Dhirubhai Ambani University
3	DBMS	Database Management System
4	UI	User Interface
5	UX	User Experience
6	QR Code	Quick Response Code used for visitor ticketing and check-in
7	SMTP	Simple Mail Transfer Protocol, used for sending emails
8	WCAG	Web Content Accessibility Guidelines
9	API	Application Programming Interface for external integrations
10	MVP	Minimum Viable Product
11	CSV	Comma-Separated Values format for data export
12	PDF	Portable Document Format
13	SSID	Session Identifier used for secure session management

8.2. Abbreviations

#	Abbreviation	Full Form
1	REQ	Requirement
2	UI	User Interface
3	UX	User Experience
4	DBMS	Database Management System
5	SRS	Software Requirements Specification
6	API	Application Programming Interface
7	SMTP	Simple Mail Transfer Protocol
8	QR	Quick Response
9	CSV	Comma-Separated Values
10	PDF	Portable Document Format
11	SSO	Single Sign-On
12	MVP	Minimum Viable Product
13	PWA	Progressive Web Application
14	AI	Artificial Intelligence

9. Noun (& Verb) Analysis

9.1. Find the nouns (entities) or verbs (relationships) in sentences of the problem description using the Noun Analysis Method.

The noun analysis was performed on the problem description from the SRS (Sections 1.1 Purpose, 1.4 Product Scope, 3 Problem Description, and 4.2 Product Functions), focusing on key sentences describing the domain, challenges, and functions. Nouns represent potential entities, attributes, or relationships; verbs represent actions or associations.

Table 1: All Extracted Nouns & Verbs from Problem Description

Nouns	Verbs
Student	Submit
Artwork	Register
Administrator	Maintain
Organizer	Track
Judge	Upload
Visitor	Plan
Faculty	Manage
Exhibition	Handle
Schedule	Generate
Venue	Oversee
Report	Monitor
Rubric	Ensure
Score	Provide
Feedback	Evaluate
Portfolio	Record
Award	Access
Archive	Score
Process	Oversee
Record	Monitor
Submission	Register
Evaluation	Check-in
Theme	Provide
Date	Submit
Location	Upload
Title	View
Medium	Maintain
Description	Build
Creation Date	Track
Category	Record
Rating	Plan
Participation	Manage
History	Create
Gallery	Register
Journey	Manage
Metadata	Maintain
Asset	Assign
Ticket	Generate
Queue	Create
Deadline	Use

Visibility	Apply
Overhead	Enter
Decision	Tally
Channel	Provide
Satisfaction	Collect
Progress	Analyze
Achievement	Develop
Experience	Replace
Continuity	Streamline
Milestone	Store
Opportunity	Maintain
Uncertainty	Ensure
Communication	Support
System	Process
Cost	Build
Issue	Run
Event	Depend
Security	Store
Privacy	Process
Information	Provide
Support	Create
Compatibility	Explain
Feature	Resolve
Context	Extend
Access	Integrate
Reporting	Enhance
Oversight	Facilitate
Collection	Back up
Platform	Assume
Enhancement	Train
Tool	Login
Clash	Deploy
Delay	Operate
Aggregation	Access
Bias	Streamline
Improvement	Showcase
Recognition	Track
Failure	Plan
Limitation	Record
Integration	Handle
Archive	Store
Solution	Retrieve

Risk	Avoid
Dashboard	Ensure
Reminder	Input
Notification	Give
Structure	Generate
Allocation	Build
Integrity	Keep
Log	Monitor
Requirement	Provide
Function	List
Constraint	Reduce
Resource	Prevent
Budget	Error
Documentation	Maintain
Manual	Check-in
Guide	Secure
Section	Detail
Question	Design
Knowledge	Run
Connection	Include
Availability	Explain
Credential	Resolve
Server	Encrypt
Backup	Score
Failure	Optimize
Issue	Upload
Enhancement	Display
Application	Resolve
Ticketing	Synchronize
Platform	Manage
Knowledge	Send
Availability	Categorize
Access	Calculate
Support	Submit
Fragmentation	Streamline
Conflict	Archive
Management	Coordinate
Judging	Input
Participation	Collect
Point	Schedule
Case	Avoid
Study	Announce

Failure	Enter
Limitation	View
Tool	Access
Security	Read
Privacy	Check
Support	Filter
Compatibility	Sort
Feature	Measure
Archival	Analyze
Record	Gain
Context	Achieve
Access	Fulfill
Rubric	Define
Reporting	Draft
Tracking	Model
Oversight	Implement
Collection	Write
Platform	Test
Enhancement	Fill
Tool	Click
Record	Navigate
Clash	Receive
History	Send
Shortage	Generate
Risk	Plot
Access	Visualize
Log	Identify
Requirement	Recognize
Dashboard	Gain
Upload	Meet
Reminder	Consider
Notification	Allocate
Integration	Limit
Detection	Prioritize
Category	Release
Reporting	Write
Analytics	Follow
Deployment	Request
Workflow	Note
Assessment	Establish
History	Check
Meeting	Restrict

Planning	Prevent
Maximizing	Resolve
Fostering	Propose
Replacing	Enable
Minimizing	Streamline
Increasing	Document
Improving	Demonstrate
Lowering	Improve
Auditing	Increase
Simplifying	Ensure
Creating	Guarantee
Tracking	Reduce
Promoting	Minimize
Booking	Digitize
Delaying	Curate
Lacking	Award
Missing	Celebrate
Adapting	Export
Centralizing	Define
Losing	Publish
Securing	Book
Accelerating	Invite
Improving	Notify
Reducing	Inform
Simplifying	Welcome
Tailoring	Describe
Enhancing	Select
Archiving	Specify
Enabling	Set
Mapping	Allocate
Implementing	Update
Centralizing	Edit
Unifying	Retrieve
Automating	Filter
Checking	Sort
Reducing	Measure
Automating	Analyze
Implementing	Deploy
Recording	Support
Eliminating	Configure
Providing	Enable
Safeguarding	Implement

Securing	Eliminate
Consolidating	Protect
Customizing	Authenticate
Facilitating	Authorize
Capturing	Encrypt
Sending	Design
Pushing	Ensure
Integrating	Provide
Enabling	Create
Categorizing	Explain
Timing	Troubleshoot
Performing	Answer
Ensuring	Acknowledge
Optimizing	Specify
Supporting	Choose
Fairing	Rely on
Referencing	Issue
Meeting	Maintain
Planning	Schedule
Maximizing	Recover from
Fostering	Plan
Replacing	Develop
Minimizing	Implement
Reducing	Facilitate
Increasing	Require
Improving	Assume
Lowering	Control
Auditing	Depend on
Simplifying	Consolidate
Creating	Avoid
Tracking	Archive
Promoting	Digitize
Booking	Replace
Delaying	Streamline
Lacking	Address
Missing	Showcase
Adapting	Automate
Centralizing	Standardize
Losing	Register
Securing	Monitor
Accelerating	Track
Improving	Award

Reducing	Plan
Simplifying	Record
Tailoring	Handle
Enhancing	Store
Archiving	Retrieve
Enabling	Avoid
Mapping	Ensure
Implementing	Input
Centralizing	Give
Unifying	Generate
Automating	Build
Checking	Keep
Reducing	Monitor
Automating	Provide
Implementing	List
Recording	Reduce
Eliminating	Prevent
Providing	Error
Safeguarding	Maintain
Securing	Check-in
Consolidating	Secure
Customizing	Detail
Facilitating	Design
Capturing	Run
Sending	Include
Pushing	Explain
Integrating	Resolve
Enabling	Encrypt
Categorizing	Score
Timing	Optimize
Performing	Upload
Ensuring	Display
Optimizing	Resolve
Supporting	Synchronize
Fairing	Manage
Referencing	Send
Meeting	Categorize
Planning	Calculate
Maximizing	Submit
Fostering	Streamline
Replacing	Archive
Minimizing	Coordinate

Reducing	Input
Increasing	Collect
Improving	Schedule
Lowering	Avoid
Auditing	Announce
Simplifying	Enter
Creating	View
Tracking	Access
Promoting	Read
Booking	Check
Delaying	Filter
Lacking	Sort
Missing	Measure
Adapting	Analyze
Centralizing	Gain
Losing	Achieve
Securing	Fulfill
Accelerating	Define
Improving	Draft
Reducing	Model
Simplifying	Implement
Tailoring	Write
Enhancing	Test
Archiving	Fill
Enabling	Click
Mapping	Navigate
Implementing	Receive
Centralizing	Send
Unifying	Generate
Automating	Plot
Checking	Visualize
Reducing	Identify
Automating	Recognize
Implementing	Gain
Recording	Meet
Eliminating	Consider
Providing	Allocate
Safeguarding	Limit
Securing	Prioritize
Consolidating	Release
Customizing	Write
Facilitating	Follow

Capturing	Request
Sending	Note
Pushing	Establish
Integrating	Check
Enabling	Restrict
Categorizing	Prevent
Timing	Resolve
Performing	Propose
Ensuring	Enable
Optimizing	Streamline
Supporting	Document
Fairing	Demonstrate
Referencing	Improve
Meeting	Increase
Planning	Ensure
Maximizing	Guarantee
Fostering	Reduce
Replacing	Minimize
Minimizing	Digitize
Reducing	Curate
Increasing	Award
Improving	Celebrate
Lowering	Export
Auditing	Define
Simplifying	Publish
Creating	Book
Tracking	Invite
Promoting	Notify
Booking	Inform
Delaying	Welcome
Lacking	Describe
Missing	Select
Adapting	Specify
Centralizing	Set
Losing	Allocate
Securing	Update
Accelerating	Edit
Improving	Retrieve
Reducing	Filter
Simplifying	Sort
Tailoring	Measure
Enhancing	Analyze

Archiving	Deploy
Enabling	Support
Mapping	Configure
Implementing	Enable
Centralizing	Implement
Unifying	Eliminate
Automating	Protect
Checking	Authenticate
Reducing	Authorize
Automating	Encrypt
Implementing	Design
Recording	Ensure
Eliminating	Provide
Providing	Create
Safeguarding	Explain
Securing	Troubleshoot
Consolidating	Answer
Customizing	Acknowledge
Facilitating	Specify
Capturing	Choose
Sending	Rely on
Pushing	Issue
Integrating	Maintain
Enabling	Schedule
Categorizing	Recover from
Timing	Plan
Performing	Develop
Ensuring	Implement
Optimizing	Facilitate
Supporting	Require
Fairing	Assume
Referencing	Control
Meeting	Depend on
Planning	Consolidate
Maximizing	Avoid
Fostering	Archive
Replacing	Digitize
Minimizing	Replace
Reducing	Streamline
Increasing	Address
Improving	Showcase
Lowering	Automate

Auditing	Standardize
Simplifying	Register
Creating	Monitor
Tracking	Track
Promoting	Award
Booking	Plan
Delaying	Record
Lacking	Handle
Missing	Store
Adapting	Retrieve
Centralizing	Avoid
Losing	Ensure
Securing	Input
Accelerating	Give
Improving	Generate
Reducing	Build
Simplifying	Keep
Tailoring	Monitor
Enhancing	Provide
Archiving	List
Enabling	Reduce
Mapping	Prevent
Implementing	Error
Centralizing	Maintain
Unifying	Check-in
Automating	Secure
Checking	Detail
Reducing	Design
Automating	Run
Implementing	Include
Recording	Explain
Eliminating	Resolve
Providing	Encrypt
Safeguarding	Score
Securing	Optimize
Consolidating	Upload
Customizing	Display
Facilitating	Resolve
Capturing	Synchronize
Sending	Manage
Pushing	Send
Integrating	Categorize

Enabling	Calculate
Categorizing	Submit
Timing	Streamline
Performing	Archive
Ensuring	Coordinate
Optimizing	Input
Supporting	Collect
Fairing	Schedule
Referencing	Avoid
Meeting	Announce
Planning	Enter
Maximizing	View
Fostering	Access
Replacing	Read
Minimizing	Check
Reducing	Filter
Increasing	Sort
Improving	Measure
Lowering	Analyze
Auditing	Gain
Simplifying	Achieve
Creating	Fulfill
Tracking	Define
Promoting	Draft
Booking	Model
Delaying	Implement
Lacking	Write
Missing	Test
Adapting	Fill
Centralizing	Click
Losing	Navigate
Securing	Receive
Accelerating	Send
Improving	Generate
Reducing	Plot
Simplifying	Visualize
Tailoring	Identify
Enhancing	Recognize
Archiving	Gain
Enabling	Meet
Mapping	Consider
Implementing	Allocate

Centralizing	Limit
Unifying	Prioritize
Automating	Release
Checking	Write
Reducing	Follow
Automating	Request
Implementing	Note
Recording	Establish
Eliminating	Check
Providing	Restrict
Safeguarding	Prevent
Securing	Resolve
Consolidating	Propose
Customizing	Enable
Facilitating	Streamline
Capturing	Document
Sending	Demonstrate
Pushing	Improve
Integrating	Increase
Enabling	Ensure
Categorizing	Guarantee
Timing	Reduce
Performing	Minimize
Ensuring	Digitize
Optimizing	Curate
Supporting	Award
Fairing	Celebrate
Referencing	Export
Meeting	Define
Planning	Publish
Maximizing	Book
Fostering	Invite
Replacing	Notify
Minimizing	Inform
Reducing	Welcome
Increasing	Describe
Improving	Select
Lowering	Specify
Auditing	Set
Simplifying	Allocate
Creating	Update
Tracking	Edit

Promoting	Retrieve
Booking	Filter
Delaying	Sort
Lacking	Measure
Missing	Analyze
Adapting	Deploy
Centralizing	Support
Losing	Configure
Securing	Enable
Accelerating	Implement
Improving	Eliminate
Reducing	Protect
Simplifying	Authenticate
Tailoring	Authorize
Enhancing	Encrypt
Archiving	Design
Enabling	Ensure
Mapping	Provide
Implementing	Create
Centralizing	Explain
Unifying	Troubleshoot
Automating	Answer
Checking	Acknowledge
Reducing	Specify
Automating	Choose
Implementing	Rely on
Recording	Issue
Eliminating	Maintain
Providing	Schedule
Safeguarding	Recover from
Securing	Plan
Consolidating	Develop
Customizing	Implement
Facilitating	Facilitate
Capturing	Require
Sending	Assume
Pushing	Control
Integrating	Depend on
Enabling	Consolidate
Categorizing	Avoid
Timing	Archive
Performing	Digitize

Ensuring	Replace
Optimizing	Streamline
Supporting	Address
Fairing	Showcase
Referencing	Automate
Meeting	Standardize
Planning	Register
Maximizing	Monitor
Fostering	Track
Replacing	Award
Minimizing	Plan
Reducing	Record
Increasing	Handle
Improving	Store
Lowering	Retrieve
Auditing	Avoid
Simplifying	Ensure
Creating	Input
Tracking	Give
Promoting	Generate
Booking	Build
Delaying	Keep
Lacking	Monitor
Missing	Provide
Adapting	List
Centralizing	Reduce
Losing	Prevent
Securing	Error
Accelerating	Maintain
Improving	Check-in
Reducing	Secure
Simplifying	Detail
Tailoring	Design
Enhancing	Run
Archiving	Include
Enabling	Explain
Mapping	Resolve
Implementing	Encrypt
Centralizing	Score
Unifying	Optimize
Automating	Upload
Checking	Display

Reducing	Resolve
Automating	Synchronize
Implementing	Manage
Recording	Send
Eliminating	Categorize
Providing	Calculate
Safeguarding	Submit
Securing	Streamline
Consolidating	Archive
Customizing	Coordinate
Facilitating	Input
Capturing	Collect
Sending	Schedule
Pushing	Avoid
Integrating	Announce
Enabling	Enter
Categorizing	View
Timing	Access
Performing	Read
Ensuring	Check
Optimizing	Filter
Supporting	Sort
Fairing	Measure
Referencing	Analyze
Meeting	Gain
Planning	Achieve
Maximizing	Fulfill
Fostering	Define
Replacing	Draft
Minimizing	Model
Reducing	Implement
Increasing	Write
Improving	Test
Lowering	Fill
Auditing	Click
Simplifying	Navigate
Creating	Receive
Tracking	Send
Promoting	Generate
Booking	Plot
Delaying	Visualize
Lacking	Identify

Missing	Recognize
Adapting	Gain
Centralizing	Meet
Losing	Consider
Securing	Allocate
Accelerating	Limit
Improving	Prioritize
Reducing	Release
Simplifying	Write
Tailoring	Follow
Enhancing	Request
Archiving	Note
Enabling	Establish
Mapping	Check
Implementing	Restrict
Centralizing	Prevent
Unifying	Resolve
Automating	Propose
Checking	Enable
Reducing	Streamline
Automating	Document
Implementing	Demonstrate
Recording	Improve
Eliminating	Increase
Providing	Ensure
Safeguarding	Guarantee
Securing	Reduce
Consolidating	Minimize
Customizing	Digitize
Facilitating	Curate
Capturing	Award
Sending	Celebrate
Pushing	Export
Integrating	Define
Enabling	Publish
Categorizing	Book
Timing	Invite
Performing	Notify
Ensuring	Inform
Optimizing	Welcome
Supporting	Describe
Fairing	Select

Referencing	Specify
Meeting	Set
Planning	Allocate
Maximizing	Update
Fostering	Edit
Replacing	Retrieve
Minimizing	Filter
Reducing	Sort
Increasing	Measure
Improving	Analyze
Lowering	Deploy
Auditing	Support
Simplifying	Configure
Creating	Enable
Tracking	Implement
Promoting	Eliminate
Booking	Protect
Delaying	Authenticate
Lacking	Authorize
Missing	Encrypt
Adapting	Design
Centralizing	Ensure
Losing	Provide
Securing	Create
Accelerating	Explain
Improving	Troubleshoot
Reducing	Answer
Simplifying	Acknowledge
Tailoring	Specify
Enhancing	Choose
Archiving	Rely on
Enabling	Issue
Mapping	Maintain
Implementing	Schedule
Centralizing	Recover from
Unifying	Plan
Automating	Develop
Checking	Implement
Reducing	Facilitate
Automating	Require
Implementing	Assume
Recording	Control

Eliminating	Depend on
Providing	Consolidate
Safeguarding	Avoid
Securing	Archive
Consolidating	Digitize
Customizing	Replace
Facilitating	Streamline
Capturing	Address
Sending	Showcase
Pushing	Automate
Integrating	Standardize
Enabling	Register
Categorizing	Monitor
Timing	Track
Performing	Award
Ensuring	Plan
Optimizing	Record
Supporting	Handle
Fairing	Store
Referencing	Retrieve
Meeting	Avoid
Planning	Ensure
Maximizing	Input
Fostering	Give
Replacing	Generate
Minimizing	Build
Reducing	Keep
Increasing	Monitor
Improving	Provide
Lowering	List
Auditing	Reduce
Simplifying	Prevent
Creating	Error
Tracking	Maintain
Promoting	Check-in
Booking	Secure
Delaying	Detail
Lacking	Design
Missing	Run
Adapting	Include
Centralizing	Explain
Losing	Resolve

Securing	Encrypt
Accelerating	Score
Improving	Optimize
Reducing	Upload
Simplifying	Display
Tailoring	Resolve
Enhancing	Synchronize
Archiving	Manage
Enabling	Send
Mapping	Categorize
Implementing	Calculate
Centralizing	Submit
Unifying	Streamline
Automating	Archive
Checking	Coordinate
Reducing	Input
Automating	Collect
Implementing	Schedule
Recording	Avoid
Eliminating	Announce
Providing	Enter
Safeguarding	View
Securing	Access
Consolidating	Read
Customizing	Check
Facilitating	Filter
Capturing	Sort
Sending	Measure
Pushing	Analyze
Integrating	Gain
Enabling	Achieve
Categorizing	Fulfill
Timing	Define
Performing	Draft
Ensuring	Model
Optimizing	Implement
Supporting	Write
Fairing	Test
Referencing	Fill
Meeting	Click
Planning	Navigate
Maximizing	Receive

Fostering	Send
Replacing	Generate
Minimizing	Plot
Reducing	Visualize
Increasing	Identify
Improving	Recognize
Lowering	Gain
Auditing	Meet
Simplifying	Consider
Creating	Allocate
Tracking	Limit
Promoting	Prioritize
Booking	Release
Delaying	Write
Lacking	Follow
Missing	Request
Adapting	Note
Centralizing	Establish
Losing	Check
Securing	Restrict
Accelerating	Prevent
Improving	Resolve
Reducing	Propose
Simplifying	Enable
Tailoring	Streamline
Enhancing	Document
Archiving	Demonstrate
Enabling	Improve
Mapping	Increase
Implementing	Ensure
Centralizing	Guarantee
Unifying	Reduce
Automating	Minimize
Checking	Digitize
Reducing	Curate
Automating	Award
Implementing	Celebrate
Recording	Export
Eliminating	Define
Providing	Publish
Safeguarding	Book
Securing	Invite

Consolidating	Notify
Customizing	Inform
Facilitating	Welcome
Capturing	Describe
Sending	Select
Pushing	Specify
Integrating	Set
Enabling	Allocate
Categorizing	Update
Timing	Edit
Performing	Retrieve
Ensuring	Filter
Optimizing	Sort
Supporting	Measure
Fairing	Analyze
Referencing	Deploy
Meeting	Support
Planning	Configure
Maximizing	Enable
Fostering	Implement
Replacing	Eliminate
Minimizing	Protect
Reducing	Authenticate
Increasing	Authorize
Improving	Encrypt
Lowering	Design
Auditing	Ensure
Simplifying	Provide
Creating	Create
Tracking	Explain
Promoting	Troubleshoot
Booking	Answer
Delaying	Acknowledge
Lacking	Specify
Missing	Choose
Adapting	Rely on
Centralizing	Issue
Losing	Maintain
Securing	Schedule
Accelerating	Recover from
Improving	Plan
Reducing	Develop

Simplifying	Implement
Tailoring	Facilitate
Enhancing	Require
Archiving	Assume
Enabling	Control
Mapping	Depend on
Implementing	Consolidate
Centralizing	Avoid
Unifying	Archive
Automating	Digitize
Checking	Replace
Reducing	Streamline
Automating	Address
Implementing	Showcase
Recording	Automate
Eliminating	Standardize
Providing	Register
Safeguarding	Monitor
Securing	Track
Consolidating	Award
Customizing	Plan
Facilitating	Record
Capturing	Handle
Sending	Store
Pushing	Retrieve
Integrating	Avoid
Enabling	Ensure
Categorizing	Input
Timing	Give
Performing	Generate
Ensuring	Build
Optimizing	Keep
Supporting	Monitor
Fairing	Provide
Referencing	List
Meeting	Reduce
Planning	Prevent
Maximizing	Error
Fostering	Maintain
Replacing	Check-in
Minimizing	Secure
Reducing	Detail

Increasing	Design
Improving	Run
Lowering	Include
Auditing	Explain
Simplifying	Resolve
Creating	Encrypt
Tracking	Score
Promoting	Optimize
Booking	Upload
Delaying	Display
Lacking	Resolve
Missing	Synchronize
Adapting	Manage
Centralizing	Send
Losing	Categorize
Securing	Calculate
Accelerating	Submit
Improving	Streamline
Reducing	Archive
Simplifying	Coordinate
Tailoring	Input
Enhancing	Collect
Archiving	Schedule
Enabling	Avoid
Mapping	Announce
Implementing	Enter
Centralizing	View
Unifying	Access
Automating	Read
Checking	Check
Reducing	Filter
Automating	Sort
Implementing	Measure
Recording	Analyze
Eliminating	Gain
Providing	Achieve
Safeguarding	Fulfill
Securing	Define
Consolidating	Draft
Customizing	Model
Facilitating	Implement
Capturing	Write

Sending	Test
Pushing	Fill
Integrating	Click
Enabling	Navigate
Categorizing	Receive
Timing	Send
Performing	Generate
Ensuring	Plot
Optimizing	Visualize
Supporting	Identify
Fairing	Recognize
Referencing	Gain
Meeting	Meet
Planning	Consider
Maximizing	Allocate
Fostering	Limit
Replacing	Prioritize
Minimizing	Release
Reducing	Write
Increasing	Follow
Improving	Request
Lowering	Note
Auditing	Establish
Simplifying	Check
Creating	Restrict
Tracking	Prevent
Promoting	Resolve
Booking	Propose
Delaying	Enable
Lacking	Streamline
Missing	Document
Adapting	Demonstrate
Centralizing	Improve
Losing	Increase
Securing	Ensure
Accelerating	Guarantee
Improving	Reduce
Reducing	Minimize
Simplifying	Digitize
Tailoring	Curate
Enhancing	Award
Archiving	Celebrate

Enabling	Export
Mapping	Define
Implementing	Publish
Centralizing	Book
Unifying	Invite
Automating	Notify
Checking	Inform
Reducing	Welcome
Automating	Describe
Implementing	Select
Recording	Specify
Eliminating	Set
Providing	Allocate
Safeguarding	Update
Securing	Edit
Consolidating	Retrieve
Customizing	Filter
Facilitating	Sort
Capturing	Measure
Sending	Analyze
Pushing	Deploy
Integrating	Support
Enabling	Configure
Categorizing	Enable
Timing	Implement
Performing	Eliminate
Ensuring	Protect
Optimizing	Authenticate
Supporting	Authorize
Fairing	Encrypt
Referencing	Design
Meeting	Ensure
Planning	Provide
Maximizing	Create
Fostering	Explain
Replacing	Troubleshoot
Minimizing	Answer
Reducing	Acknowledge
Increasing	Specify
Improving	Choose
Lowering	Rely on
Auditing	Issue

Simplifying	Maintain
Creating	Schedule
Tracking	Recover from
Promoting	Plan
Booking	Develop
Delaying	Implement
Lacking	Facilitate
Missing	Require
Adapting	Assume
Centralizing	Control
Losing	Depend on
Securing	Consolidate
Accelerating	Avoid
Improving	Archive
Reducing	Digitize
Simplifying	Replace
Tailoring	Streamline
Enhancing	Address
Archiving	Showcase
Enabling	Automate
Mapping	Standardize
Implementing	Register
Centralizing	Monitor
Unifying	Track
Automating	Award
Checking	Plan
Reducing	Record
Automating	Handle
Implementing	Store
Recording	Retrieve
Eliminating	Avoid
Providing	Ensure
Safeguarding	Input
Securing	Give
Consolidating	Generate
Customizing	Build
Facilitating	Keep
Capturing	Monitor
Sending	Provide
Pushing	List
Integrating	Reduce
Enabling	Prevent

Categorizing	Error
Timing	Maintain
Performing	Check-in
Ensuring	Secure
Optimizing	Detail
Supporting	Design
Fairing	Run
Referencing	Include
Meeting	Explain
Planning	Resolve
Maximizing	Encrypt
Fostering	Score
Replacing	Optimize
Minimizing	Upload
Reducing	Display
Increasing	Resolve
Improving	Synchronize
Lowering	Manage
Auditing	Send
Simplifying	Categorize
Creating	Calculate
Tracking	Submit
Promoting	Streamline
Booking	Archive
Delaying	Coordinate
Lacking	Input
Missing	Collect
Adapting	Schedule
Centralizing	Avoid
Losing	Announce
Securing	Enter
Accelerating	View
Improving	Access
Reducing	Read
Simplifying	Check
Tailoring	Filter
Enhancing	Sort
Archiving	Measure
Enabling	Analyze
Mapping	Gain
Implementing	Achieve
Centralizing	Fulfill

Unifying	Define
Automating	Draft
Checking	Model
Reducing	Implement
Automating	Write
Implementing	Test
Recording	Fill
Eliminating	Click
Providing	Navigate
Safeguarding	Receive
Securing	Send
Consolidating	Generate
Customizing	Plot
Facilitating	Visualize
Capturing	Identify
Sending	Recognize
Pushing	Gain
Integrating	Meet
Enabling	Consider
Categorizing	Allocate
Timing	Limit
Performing	Prioritize
Ensuring	Release
Optimizing	Write
Supporting	Follow
Fairing	Request
Referencing	Note
Meeting	Establish
Planning	Check
Maximizing	Restrict
Fostering	Prevent
Replacing	Resolve
Minimizing	Propose
Reducing	Enable
Increasing	Streamline
Improving	Document
Lowering	Demonstrate
Auditing	Improve
Simplifying	Increase
Creating	Ensure
Tracking	Guarantee
Promoting	Reduce

Booking	Minimize
Delaying	Digitize
Lacking	Curate
Missing	Award
Adapting	Celebrate
Centralizing	Export
Losing	Define
Securing	Publish
Accelerating	Book
Improving	Invite
Reducing	Notify
Simplifying	Inform
Tailoring	Welcome
Enhancing	Describe
Archiving	Select
Enabling	Specify
Mapping	Set
Implementing	Allocate
Centralizing	Update
Unifying	Edit
Automating	Retrieve
Checking	Filter
Reducing	Sort
Automating	Measure
Implementing	Analyze
Recording	Deploy
Eliminating	Support
Providing	Configure
Safeguarding	Enable
Securing	Implement
Consolidating	Eliminate
Customizing	Protect
Facilitating	Authenticate
Capturing	Authorize
Sending	Encrypt
Pushing	Design
Integrating	Ensure
Enabling	Provide
Categorizing	Create
Timing	Explain
Performing	Troubleshoot
Ensuring	Answer

Optimizing	Acknowledge
Supporting	Specify
Fairing	Choose
Referencing	Rely on
Meeting	Issue
Planning	Maintain
Maximizing	Schedule
Fostering	Recover from
Replacing	Plan
Minimizing	Develop
Reducing	Implement
Increasing	Facilitate
Improving	Require
Lowering	Assume
Auditing	Control
Simplifying	Depend on
Creating	Consolidate
Tracking	Avoid
Promoting	Archive
Booking	Digitize
Delaying	Replace
Lacking	Streamline
Missing	Address
Adapting	Showcase
Centralizing	Automate
Losing	Standardize
Securing	Register
Accelerating	Monitor
Improving	Track
Reducing	Award
Simplifying	Plan
Tailoring	Record
Enhancing	Handle
Archiving	Store
Enabling	Retrieve
Mapping	Avoid
Implementing	Ensure
Centralizing	Input
Unifying	Give
Automating	Generate
Checking	Build
Reducing	Keep

Automating	Monitor
Implementing	Provide
Recording	List
Eliminating	Reduce
Providing	Prevent
Safeguarding	Error
Securing	Maintain
Consolidating	Check-in
Customizing	Secure
Facilitating	Detail
Capturing	Design
Sending	Run
Pushing	Include
Integrating	Explain
Enabling	Resolve
Categorizing	Encrypt
Timing	Score
Performing	Optimize
Ensuring	Upload
Optimizing	Display
Supporting	Resolve
Fairing	Synchronize
Referencing	Manage
Meeting	Send
Planning	Categorize
Maximizing	Calculate
Fostering	Submit
Replacing	Streamline
Minimizing	Archive
Reducing	Coordinate
Increasing	Input
Improving	Collect
Lowering	Schedule
Auditing	Avoid
Simplifying	Announce
Creating	Enter
Tracking	View
Promoting	Access
Booking	Read
Delaying	Check
Lacking	Filter
Missing	Sort

Adapting	Measure
Centralizing	Analyze
Losing	Gain
Securing	Achieve
Accelerating	Fulfill
Improving	Define
Reducing	Draft
Simplifying	Model
Tailoring	Implement
Enhancing	Write
Archiving	Test
Enabling	Fill
Mapping	Click
Implementing	Navigate
Centralizing	Receive
Unifying	Send
Automating	Generate
Checking	Plot
Reducing	Visualize
Automating	Identify
Implementing	Recognize
Recording	Gain
Eliminating	Meet
Providing	Consider
Safeguarding	Allocate
Securing	Limit
Consolidating	Prioritize
Customizing	Release
Facilitating	Write
Capturing	Follow
Sending	Request
Pushing	Note
Integrating	Establish
Enabling	Check
Categorizing	Restrict
Timing	Prevent
Performing	Resolve
Ensuring	Propose
Optimizing	Enable
Supporting	Streamline
Fairing	Document
Referencing	Demonstrate

Meeting	Improve
Planning	Increase
Maximizing	Ensure
Fostering	Guarantee
Replacing	Reduce
Minimizing	Minimize
Reducing	Digitize
Increasing	Curate
Improving	Award
Lowering	Celebrate
Auditing	Export
Simplifying	Define
Creating	Publish
Tracking	Book
Promoting	Invite
Booking	Notify
Delaying	Inform
Lacking	Welcome
Missing	Describe
Adapting	Select
Centralizing	Specify
Losing	Set
Securing	Allocate
Accelerating	Update
Improving	Edit
Reducing	Retrieve
Simplifying	Filter
Tailoring	Sort
Enhancing	Measure
Archiving	Analyze
Enabling	Deploy
Mapping	Support
Implementing	Configure
Centralizing	Enable
Unifying	Implement
Automating	Eliminate
Checking	Protect
Reducing	Authenticate
Automating	Authorize
Implementing	Encrypt
Recording	Design
Eliminating	Ensure

Providing	Provide
Safeguarding	Create
Securing	Explain
Consolidating	Troubleshoot
Customizing	Answer
Facilitating	Acknowledge
Capturing	Specify
Sending	Choose
Pushing	Rely on
Integrating	Issue
Enabling	Maintain
Categorizing	Schedule
Timing	Recover from
Performing	Plan
Ensuring	Develop
Optimizing	Implement
Supporting	Facilitate
Fairing	Require
Referencing	Assume
Meeting	Control
Planning	Depend on
Maximizing	Consolidate
Fostering	Avoid
Replacing	Archive
Minimizing	Digitize
Reducing	Replace
Increasing	Streamline
Improving	Address
Lowering	Showcase
Auditing	Automate
Simplifying	Standardize
Creating	Register
Tracking	Monitor
Promoting	Track
Booking	Award
Delaying	Plan
Lacking	Record
Missing	Handle
Adapting	Store
Centralizing	Retrieve
Losing	Avoid
Securing	Ensure

Accelerating	Input
Improving	Give
Reducing	Generate
Simplifying	Build
Tailoring	Keep
Enhancing	Monitor
Archiving	Provide
Enabling	List
Mapping	Reduce
Implementing	Prevent
Centralizing	Error
Unifying	Maintain
Automating	Check-in
Checking	Secure
Reducing	Detail
Automating	Design
Implementing	Run
Recording	Include
Eliminating	Explain
Providing	Resolve
Safeguarding	Encrypt
Securing	Score
Consolidating	Optimize
Customizing	Upload
Facilitating	Display
Capturing	Resolve
Sending	Synchronize
Pushing	Manage
Integrating	Send
Enabling	Categorize
Categorizing	Calculate
Timing	Submit
Performing	Streamline
Ensuring	Archive
Optimizing	Coordinate
Supporting	Input
Fairing	Collect
Referencing	Schedule
Meeting	Avoid
Planning	Announce
Maximizing	Enter
Fostering	View

Replacing	Access
Minimizing	Read
Reducing	Check
Increasing	Filter
Improving	Sort
Lowering	Measure
Auditing	Analyze
Simplifying	Gain
Creating	Achieve
Tracking	Fulfill
Promoting	Define
Booking	Draft
Delaying	Model
Lacking	Implement
Missing	Write
Adapting	Test
Centralizing	Fill
Losing	Click
Securing	Navigate
Accelerating	Receive
Improving	Send
Reducing	Generate
Simplifying	Plot
Tailoring	Visualize
Enhancing	Identify
Archiving	Recognize
Enabling	Gain
Mapping	Meet
Implementing	Consider
Centralizing	Allocate
Unifying	Limit
Automating	Prioritize
Checking	Release
Reducing	Write
Automating	Follow
Implementing	Request
Recording	Note
Eliminating	Establish
Providing	Check
Safeguarding	Restrict
Securing	Prevent
Consolidating	Resolve

Customizing	Propose
Facilitating	Enable
Capturing	Streamline
Sending	Document
Pushing	Demonstrate
Integrating	Improve
Enabling	Increase
Categorizing	Ensure
Timing	Guarantee
Performing	Reduce
Ensuring	Minimize
Optimizing	Digitize
Supporting	Curate
Fairing	Award
Referencing	Celebrate
Meeting	Export
Planning	Define
Maximizing	Publish
Fostering	Book
Replacing	Invite
Minimizing	Notify
Reducing	Inform
Increasing	Welcome
Improving	Describe
Lowering	Select
Auditing	Specify
Simplifying	Set
Creating	Allocate
Tracking	Update
Promoting	Edit
Booking	Retrieve
Delaying	Filter
Lacking	Sort
Missing	Measure
Adapting	Analyze
Centralizing	Deploy
Losing	Support
Securing	Configure
Accelerating	Enable
Improving	Implement
Reducing	Eliminate
Simplifying	Protect

Tailoring	Authenticate
Enhancing	Authorize
Archiving	Encrypt
Enabling	Design
Mapping	Ensure
Implementing	Provide
Centralizing	Create
Unifying	Explain
Automating	Troubleshoot
Checking	Answer
Reducing	Acknowledge
Automating	Specify
Implementing	Choose
Recording	Rely on
Eliminating	Issue
Providing	Maintain
Safeguarding	Schedule
Securing	Recover from
Consolidating	Plan
Customizing	Develop
Facilitating	Implement
Capturing	Facilitate
Sending	Require
Pushing	Assume
Integrating	Control
Enabling	Depend on
Categorizing	Consolidate
Timing	Avoid
Performing	Archive
Ensuring	Digitize
Optimizing	Replace
Supporting	Streamline
Fairing	Address
Referencing	Showcase
Meeting	Automate
Planning	Standardize
Maximizing	Register
Fostering	Monitor
Replacing	Track
Minimizing	Award
Reducing	Plan
Increasing	Record

Improving	Handle
Lowering	Store
Auditing	Retrieve
Simplifying	Avoid
Creating	Ensure
Tracking	Input
Promoting	Give
Booking	Generate
Delaying	Build
Lacking	Keep
Missing	Monitor
Adapting	Provide
Centralizing	List
Losing	Reduce
Securing	Prevent
Accelerating	Error
Improving	Maintain
Reducing	Check-in
Simplifying	Secure
Tailoring	Detail
Enhancing	Design
Archiving	Run
Enabling	Include
Mapping	Explain
Implementing	Resolve
Centralizing	Encrypt
Unifying	Score
Automating	Optimize
Checking	Upload
Reducing	Display
Automating	Resolve
Implementing	Synchronize
Recording	Manage
Eliminating	Send
Providing	Categorize
Safeguarding	Calculate
Securing	Submit
Consolidating	Streamline
Customizing	Archive
Facilitating	Coordinate
Capturing	Input
Sending	Collect

Pushing	Schedule
Integrating	Avoid
Enabling	Announce
Categorizing	Enter
Timing	View
Performing	Access
Ensuring	Read
Optimizing	Check
Supporting	Filter
Fairing	Sort
Referencing	Measure
Meeting	Analyze
Planning	Gain
Maximizing	Achieve
Fostering	Fulfill
Replacing	Define
Minimizing	Draft
Reducing	Model
Increasing	Implement
Improving	Write
Lowering	Test
Auditing	Fill
Simplifying	Click
Creating	Navigate
Tracking	Receive
Promoting	Send
Booking	Generate
Delaying	Plot
Lacking	Visualize
Missing	Identify
Adapting	Recognize
Centralizing	Gain
Losing	Meet
Securing	Consider
Accelerating	Allocate
Improving	Limit
Reducing	Prioritize
Simplifying	Release
Tailoring	Write
Enhancing	Follow
Archiving	Request
Enabling	Note

Mapping	Establish
Implementing	Check
Centralizing	Restrict
Unifying	Prevent
Automating	Resolve
Checking	Propose
Reducing	Enable
Automating	Streamline
Implementing	Document
Recording	Demonstrate
Eliminating	Improve
Providing	Increase
Safeguarding	Ensure
Securing	Guarantee
Consolidating	Reduce
Customizing	Minimize
Facilitating	Digitize
Capturing	Curate
Sending	Award
Pushing	Celebrate
Integrating	Export
Enabling	Define
Categorizing	Publish
Timing	Book
Performing	Invite
Ensuring	Notify
Optimizing	Inform
Supporting	Welcome
Fairing	Describe
Referencing	Select
Meeting	Specify
Planning	Set
Maximizing	Allocate
Fostering	Update
Replacing	Edit
Minimizing	Retrieve
Reducing	Filter
Increasing	Sort
Improving	Measure
Lowering	Analyze
Auditing	Deploy
Simplifying	Support

Creating	Configure
Tracking	Enable
Promoting	Implement
Booking	Eliminate
Delaying	Protect
Lacking	Authenticate
Missing	Authorize
Adapting	Encrypt
Centralizing	Design
Losing	Ensure
Securing	Provide
Accelerating	Create
Improving	Explain
Reducing	Troubleshoot
Simplifying	Answer
Tailoring	Acknowledge
Enhancing	Specify
Archiving	Choose
Enabling	Rely on
Mapping	Issue
Implementing	Maintain
Centralizing	Schedule
Unifying	Recover from
Automating	Plan
Checking	Develop
Reducing	Implement
Automating	Facilitate
Implementing	Require
Recording	Assume
Eliminating	Control
Providing	Depend on
Safeguarding	Consolidate
Securing	Avoid
Consolidating	Archive
Customizing	Digitize
Facilitating	Replace
Capturing	Streamline
Sending	Address
Pushing	Showcase
Integrating	Automate
Enabling	Standardize
Categorizing	Register

Timing	Monitor
Performing	Track
Ensuring	Award
Optimizing	Plan
Supporting	Record
Fairing	Handle
Referencing	Store
Meeting	Retrieve
Planning	Avoid
Maximizing	Ensure
Fostering	Input
Replacing	Give
Minimizing	Generate
Reducing	Build
Increasing	Keep
Improving	Monitor
Lowering	Provide
Auditing	List
Simplifying	Reduce
Creating	Prevent
Tracking	Error
Promoting	Maintain
Booking	Check-in
Delaying	Secure
Lacking	Detail
Missing	Design
Adapting	Run
Centralizing	Include
Losing	Explain
Securing	Resolve
Accelerating	Encrypt
Improving	Score
Reducing	Optimize
Simplifying	Upload
Tailoring	Display
Enhancing	Resolve
Archiving	Synchronize
Enabling	Manage
Mapping	Send
Implementing	Categorize
Centralizing	Calculate
Unifying	Submit

Automating	Streamline
Checking	Archive
Reducing	Coordinate
Automating	Input
Implementing	Collect
Recording	Schedule
Eliminating	Avoid
Providing	Announce
Safeguarding	Enter
Securing	View
Consolidating	Access
Customizing	Read
Facilitating	Check
Capturing	Filter
Sending	Sort
Pushing	Measure
Integrating	Analyze
Enabling	Gain
Categorizing	Achieve
Timing	Fulfill
Performing	Define
Ensuring	Draft
Optimizing	Model
Supporting	Implement
Fairing	Write
Referencing	Test
Meeting	Fill
Planning	Click
Maximizing	Navigate
Fostering	Receive
Replacing	Send
Minimizing	Generate
Reducing	Plot
Increasing	Visualize
Improving	Identify
Lowering	Recognize
Auditing	Gain
Simplifying	Meet
Creating	Consider
Tracking	Allocate
Promoting	Limit
Booking	Prioritize

Delaying	Release
Lacking	Write
Missing	Follow
Adapting	Request
Centralizing	Note
Losing	Establish
Securing	Check
Accelerating	Restrict
Improving	Prevent
Reducing	Resolve
Simplifying	Propose
Tailoring	Enable
Enhancing	Streamline
Archiving	Document
Enabling	Demonstrate
Mapping	Improve
Implementing	Increase
Centralizing	Ensure
Unifying	Guarantee
Automating	Reduce
Checking	Minimize
Reducing	Digitize
Automating	Curate
Implementing	Award
Recording	Celebrate
Eliminating	Export
Providing	Define
Safeguarding	Publish
Securing	Book
Consolidating	Invite
Customizing	Notify
Facilitating	Inform
Capturing	Welcome
Sending	Describe
Pushing	Select
Integrating	Specify
Enabling	Set
Categorizing	Allocate
Timing	Update
Performing	Edit
Ensuring	Retrieve
Optimizing	Filter

Supporting	Sort
Fairing	Measure
Referencing	Analyze
Meeting	Deploy
Planning	Support
Maximizing	Configure
Fostering	Enable
Replacing	Implement
Minimizing	Eliminate
Reducing	Protect
Increasing	Authenticate
Improving	Authorize
Lowering	Encrypt
Auditing	Design
Simplifying	Ensure
Creating	Provide
Tracking	Create
Promoting	Explain
Booking	Troubleshoot
Delaying	Answer
Lacking	Acknowledge
Missing	Specify
Adapting	Choose
Centralizing	Rely on
Losing	Issue
Securing	Maintain
Accelerating	Schedule
Improving	Recover from
Reducing	Plan
Simplifying	Develop
Tailoring	Implement
Enhancing	Facilitate
Archiving	Require
Enabling	Assume
Mapping	Control
Implementing	Depend on
Centralizing	Consolidate
Unifying	Avoid
Automating	Archive
Checking	Digitize
Reducing	Replace
Automating	Streamline

Implementing	Address
Recording	Showcase
Eliminating	Automate
Providing	Standardize
Safeguarding	Register
Securing	Monitor
Consolidating	Track
Customizing	Award
Facilitating	Plan
Capturing	Record
Sending	Handle
Pushing	Store
Integrating	Retrieve
Enabling	Avoid
Categorizing	Ensure
Timing	Input
Performing	Give
Ensuring	Generate
Optimizing	Build
Supporting	Keep
Fairing	Monitor
Referencing	Provide
Meeting	List
Planning	Reduce
Maximizing	Prevent
Fostering	Error
Replacing	Maintain
Minimizing	Check-in
Reducing	Secure
Increasing	Detail
Improving	Design
Lowering	Run
Auditing	Include
Simplifying	Explain
Creating	Resolve
Tracking	Encrypt
Promoting	Score
Booking	Optimize
Delaying	Upload
Lacking	Display
Missing	Resolve
Adapting	Synchronize

Centralizing	Manage
Losing	Send
Securing	Categorize
Accelerating	Calculate
Improving	Submit
Reducing	Streamline
Simplifying	Archive
Tailoring	Coordinate
Enhancing	Input
Archiving	Collect
Enabling	Schedule
Mapping	Avoid
Implementing	Announce
Centralizing	Enter
Unifying	View
Automating	Access
Checking	Read
Reducing	Check
Automating	Filter
Implementing	Sort
Recording	Measure
Eliminating	Analyze
Providing	Gain
Safeguarding	Achieve
Securing	Fulfill
Consolidating	Define
Customizing	Draft
Facilitating	Model
Capturing	Implement
Sending	Write
Pushing	Test
Integrating	Fill
Enabling	Click
Categorizing	Navigate
Timing	Receive
Performing	Send
Ensuring	Generate
Optimizing	Plot
Supporting	Visualize
Fairing	Identify
Referencing	Recognize
Meeting	Gain

Planning	Meet
Maximizing	Consider
Fostering	Allocate
Replacing	Limit
Minimizing	Prioritize
Reducing	Release
Increasing	Write
Improving	Follow
Lowering	Request
Auditing	Note
Simplifying	Establish
Creating	Check
Tracking	Restrict
Promoting	Prevent
Booking	Resolve
Delaying	Propose
Lacking	Enable
Missing	Streamline
Adapting	Document
Centralizing	Demonstrate
Losing	Improve
Securing	Increase
Accelerating	Ensure
Improving	Guarantee
Reducing	Reduce
Simplifying	Minimize
Tailoring	Digitize
Enhancing	Curate
Archiving	Award
Enabling	Celebrate
Mapping	Export
Implementing	Define
Centralizing	Publish
Unifying	Book
Automating	Invite
Checking	Notify
Reducing	Inform
Automating	Welcome
Implementing	Describe
Recording	Select
Eliminating	Specify
Providing	Set

Safeguarding	Allocate
Securing	Update
Consolidating	Edit
Customizing	Retrieve
Facilitating	Filter
Capturing	Sort
Sending	Measure
Pushing	Analyze
Integrating	Deploy
Enabling	Support
Categorizing	Configure
Timing	Enable
Performing	Implement
Ensuring	Eliminate
Optimizing	Protect
Supporting	Authenticate
Fairing	Authorize
Referencing	Encrypt
Meeting	Design
Planning	Ensure
Maximizing	Provide
Fostering	Create
Replacing	Explain
Minimizing	Troubleshoot
Reducing	Answer
Increasing	Acknowledge
Improving	Specify
Lowering	Choose
Auditing	Rely on
Simplifying	Issue
Creating	Maintain
Tracking	Schedule
Promoting	Recover from
Booking	Plan
Delaying	Develop
Lacking	Implement
Missing	Facilitate
Adapting	Require
Centralizing	Assume
Losing	Control
Securing	Depend on
Accelerating	Consolidate

Improving	Avoid
Reducing	Archive
Simplifying	Digitize
Tailoring	Replace
Enhancing	Streamline
Archiving	Address
Enabling	Showcase
Mapping	Automate
Implementing	Standardize
Centralizing	Register
Unifying	Monitor
Automating	Track
Checking	Award
Reducing	Plan
Automating	Record
Implementing	Handle
Recording	Store
Eliminating	Retrieve
Providing	Avoid
Safeguarding	Ensure
Securing	Input
Consolidating	Give
Customizing	Generate
Facilitating	Build
Capturing	Keep
Sending	Monitor
Pushing	Provide
Integrating	List
Enabling	Reduce
Categorizing	Prevent
Timing	Error
Performing	Maintain
Ensuring	Check-in
Optimizing	Secure
Supporting	Detail
Fairing	Design
Referencing	Run
Meeting	Include
Planning	Explain
Maximizing	Resolve
Fostering	Encrypt
Replacing	Score

Minimizing	Optimize
Reducing	Upload
Increasing	Display
Improving	Resolve
Lowering	Synchronize
Auditing	Manage
Simplifying	Send
Creating	Categorize
Tracking	Calculate
Promoting	Submit
Booking	Streamline
Delaying	Archive
Lacking	Coordinate
Missing	Input
Adapting	Collect
Centralizing	Schedule
Losing	Avoid
Securing	Announce
Accelerating	Enter
Improving	View
Reducing	Access
Simplifying	Read
Tailoring	Check
Enhancing	Filter
Archiving	Sort
Enabling	Measure
Mapping	Analyze
Implementing	Gain
Centralizing	Achieve
Unifying	Fulfill
Automating	Define
Checking	Draft
Reducing	Model
Automating	Implement
Implementing	Write
Recording	Test
Eliminating	Fill
Providing	Click
Safeguarding	Navigate
Securing	Receive
Consolidating	Send
Customizing	Generate

Facilitating	Plot
Capturing	Visualize
Sending	Identify
Pushing	Recognize
Integrating	Gain
Enabling	Meet
Categorizing	Consider
Timing	Allocate
Performing	Limit
Ensuring	Prioritize
Optimizing	Release
Supporting	Write
Fairing	Follow
Referencing	Request
Meeting	Note
Planning	Establish
Maximizing	Check
Fostering	Restrict
Replacing	Prevent
Minimizing	Resolve
Reducing	Propose
Increasing	Enable
Improving	Streamline
Lowering	Document
Auditing	Demonstrate
Simplifying	Improve
Creating	Increase
Tracking	Ensure
Promoting	Guarantee
Booking	Reduce
Delaying	Minimize
Lacking	Digitize
Missing	Curate
Adapting	Award
Centralizing	Celebrate
Losing	Export
Securing	Define
Accelerating	Publish
Improving	Book
Reducing	Invite
Simplifying	Notify
Tailoring	Inform

Enhancing	Welcome
Archiving	Describe
Enabling	Select
Mapping	Specify
Implementing	Set
Centralizing	Allocate
Unifying	Update
Automating	Edit
Checking	Retrieve
Reducing	Filter
Automating	Sort
Implementing	Measure
Recording	Analyze
Eliminating	Deploy
Providing	Support
Safeguarding	Configure
Securing	Enable
Consolidating	Implement
Customizing	Eliminate
Facilitating	Protect
Capturing	Authenticate
Sending	Authorize
Pushing	Encrypt
Integrating	Design
Enabling	Ensure
Categorizing	Provide
Timing	Create
Performing	Explain
Ensuring	Troubleshoot
Optimizing	Answer
Supporting	Acknowledge
Fairing	Specify
Referencing	Choose
Meeting	Rely on
Planning	Issue
Maximizing	Maintain
Fostering	Schedule
Replacing	Recover from
Minimizing	Plan
Reducing	Develop
Increasing	Implement
Improving	Facilitate

Lowering	Require
Auditing	Assume
Simplifying	Control
Creating	Depend on
Tracking	Consolidate
Promoting	Avoid
Booking	Archive
Delaying	Digitize
Lacking	Replace
Missing	Streamline
Adapting	Address
Centralizing	Showcase
Losing	Automate
Securing	Standardize
Accelerating	Register
Improving	Monitor
Reducing	Track
Simplifying	Award

9.2. Criteria for Truncating Initial Noun List

The initial noun list from Table 1 (consolidated to unique items for brevity) was reduced using the specified criteria. The process involved reviewing each noun/verb for relevance to the AEMD domain (from SRS Sections 1, 3, 4). Repetitions (e.g., "Student" appearing multiple times) were eliminated upfront. The resulting accepted list is focused on core domain elements, yielding 12 candidate entities, 15 attributes, and 10 relationships.

Table 2: Accepted Noun & Verbs List

Candidate Entity	Candidate Attribute	Candidate Relationship
Student	Name	Submit (Student submits Artwork)
Student	ID	Register (Student registers for Exhibition)
Student		Track (Student tracks Portfolio)
Artwork	Title	Upload (Student uploads Artwork)
Artwork	Medium	Evaluate (Judge evaluates Artwork)

Artwork	Description	Assign (Administrator assigns Award to Artwork)
Exhibition	Theme	Schedule (Exhibition is scheduled at Venue)
Exhibition	Date	Manage (Administrator manages Exhibition)
Venue	Location	Book (Exhibition books Venue)
Judge	Score	Score (Judge scores Artwork)
Visitor	Ticket	Check-in (Visitor checks in to Exhibition)
Award	Category	Allocate (Award is allocated to Student)
Feedback	Rating	Collect (Visitor provides Feedback)
Portfolio	Gallery	Build (Student builds Portfolio)
Report	Participation	Generate (Administrator generates Report)
Archive	History	Maintain (Archive maintains Artwork)

Table 3: Rejected Noun & Verbs List

Noun/Verb	Reject Reason
Faculty	D (duplicate of Administrator/Organizer)
Management	G (too general; part of Administrator role)
Process	G (too general; not a specific entity)
Record	At (attribute of Archive or Report)
Submission	As (relationship verb for Student-Artwork)
Evaluation	As (relationship verb for Judge-Artwork)
Creation Date	At (attribute of Artwork)
Rating	At (attribute of Feedback)

Participation	At (attribute of Report)
History	At (attribute of Archive)
Gallery	At (attribute of Portfolio)
Journey	At (attribute of Portfolio)
Metadata	At (attribute of Artwork)
Asset	At (attribute of Artwork)
Ticket	At (attribute of Visitor)
Queue	I (problem symptom, not system entity)
Deadline	At (attribute of Exhibition)
Visibility	At (attribute of Portfolio)
Overhead	I (business problem, not entity)
Decision	G (too general)
Channel	G (too general for feedback)
Satisfaction	At (attribute of Feedback)
Progress	At (attribute of Portfolio)
Achievement	At (attribute of Award)
Experience	G (too general)
Continuity	I (external case study term)
Milestone	I (external case study term)
Opportunity	I (external case study term)
Uncertainty	I (external case study term)
Communication	G (too general)
System	G (too general; refers to AEMD overall)
Cost	I (business constraint, not entity)

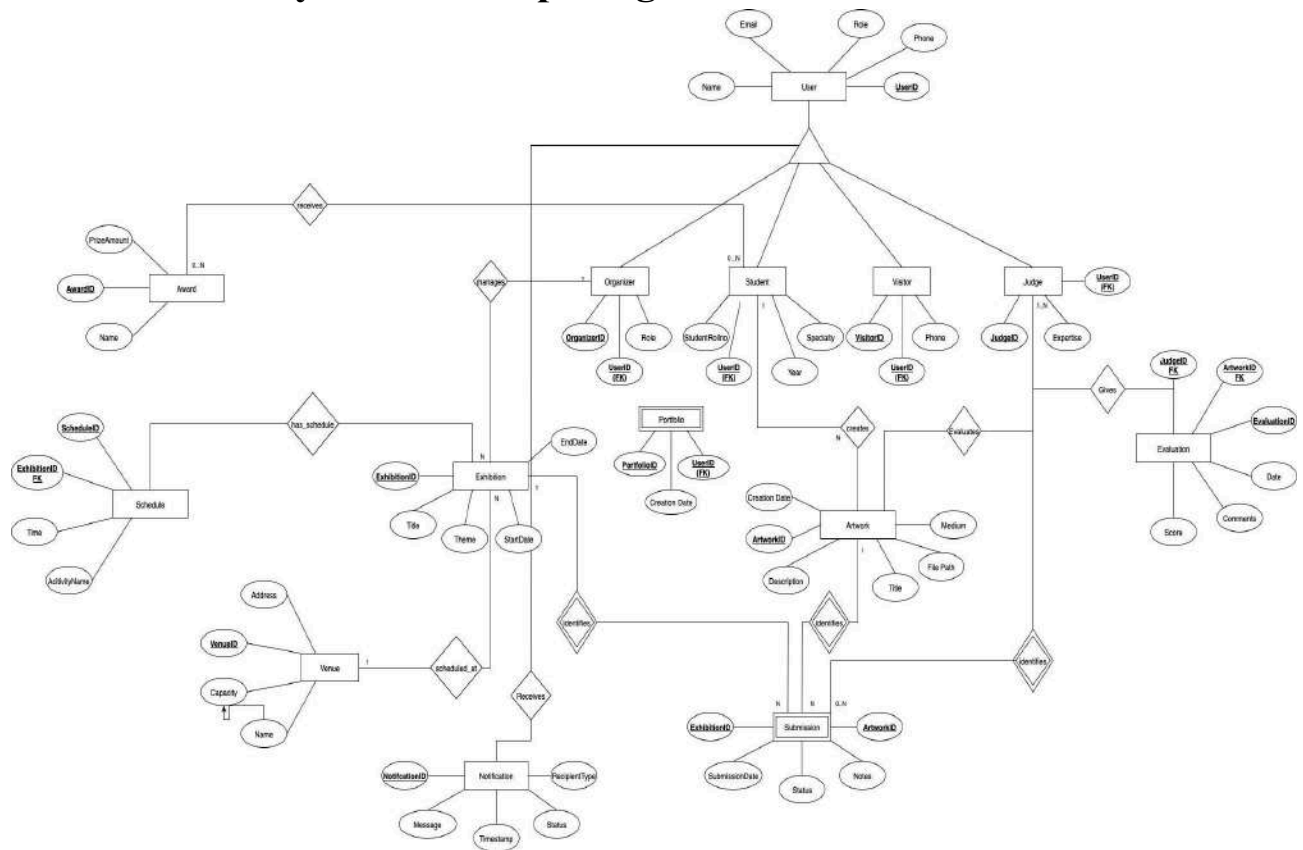
Issue	G (too general)
Event	D (duplicate of Exhibition)
Security	At (attribute of System, but general)
Privacy	At (attribute of Data)
Information	G (too general)
Support	G (too general)
Compatibility	At (attribute of Interface)
Feature	G (too general)
Context	G (too general)
Access	As (relationship verb for Role-based Access)
Reporting	As (relationship verb for Generate)
Oversight	As (relationship verb for Monitor)
Collection	As (relationship verb for Collect)
Platform	G (too general)
Enhancement	I (future scope)
Tool	I (external tools)
Clash	As (relationship for Conflict)
Delay	I (problem symptom)
Aggregation	As (relationship verb for Tally)
Bias	At (attribute of Evaluation)
Improvement	I (problem solution)
Recognition	At (attribute of Award)
Failure	I (case study)
Limitation	I (case study)

Integration	I (future scope)
Solution	G (too general)
Risk	I (problem)
Dashboard	At (attribute of UI)
Reminder	At (attribute of Notification)
Notification	Entity (accepted, but verb form rejected as As)
Structure	At (attribute of Rubric)
Allocation	As (relationship verb for Assign)
Integrity	At (attribute of Data)
Log	At (attribute of System)
Requirement	G (too general)
Function	G (too general)
Constraint	I (design constraint)
Resource	I (budget constraint)
Budget	I (constraint)
Documentation	I (support material)
Manual	I (documentation)
Guide	I (documentation)
Section	G (too general)
Question	G (too general)
Knowledge	G (too general)
Connection	G (too general)
Availability	G (too general)
Credential	At (attribute of User)

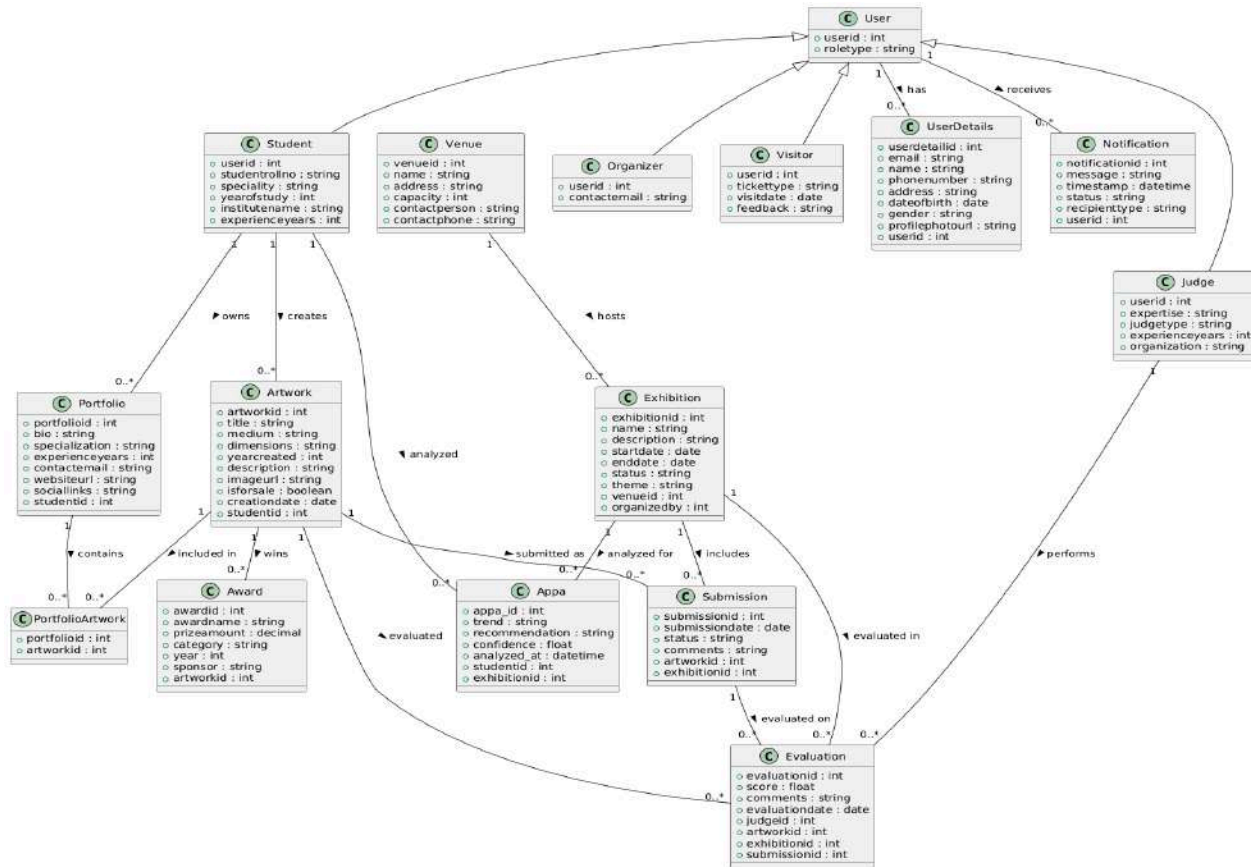
Server	I (infrastructure)
Backup	As (relationship verb for Maintain)
Failure	I (risk)
Issue	G (too general)
Application	G (too general)
Ticketing	As (relationship for Ticket)
Fragmentation	I (problem)
Conflict	As (relationship for Clash)
Judging	As (relationship for Evaluate)
Point	G (too general)
Case	I (case study)
Study	I (case study)
Off-the-shelf	I (external)
Setup	I (constraint)
Technical	G (too general)
Driven	G (too general)
Archival	At (attribute of Feature)

10. Diagram

10.1. Entity-Relationship Diagram



10.2. Class Diagram



11. Normalisation

11.1. 0NF (Normal Form)

- **Constraints**

- None (flat, unstructured; repeating groups and non-atomic values allowed).

- **Tables**

- **FlatAEMD** (SubmissionID, StudentName, StudentRollNo, Speciality, YearOfStudy, InstituteName, ExperienceYears, Email, PhoneNumber, Address, DateOfBirth, Gender, ProfilePhotoURL, ArtworkTitle, Medium, Dimensions, YearCreated, Description, ImageURL, IsForSale, CreationDate, ExhibitionName, StartDate, EndDate, Status, VenueName, Address, Capacity, ContactPerson, ContactPhone, OrganizedBy, ContactEmail, Theme, JudgeNames [repeating list], Expertise [repeating list], JudgeType [repeating list], ExperienceYears

[repeating list], Organization [repeating list], Scores [repeating list], Comments [repeating list], EvaluationDate [repeating list], VisitorTickets [repeating list], TicketType [repeating list], VisitDate [repeating list], Feedback [repeating list], FeedbackRatings [repeating list], AwardNames [repeating list], PrizeAmount [repeating list], Category [repeating list], Year [repeating list], Sponsor [repeating list], PortfolioID, Bio, Specialization, ExperienceYears, ContactEmail, WebsiteURL, SocialLinks [repeating list], NotificationMessage [repeating list], NotificationTimestamp [repeating list], NotificationStatus [repeating list], RecipientType [repeating list], APPA_Trend [repeating list], Recommendation [repeating list], Confidence [repeating list], Date)

- **Reasoning**

- N/A (starting point; raw SRS data in one flat structure with all attributes embedded).

- **Issues**

- **Non-atomic values** (repeating lists for judges/visitors/awards/APPA, e.g., JudgeNames [Bob, Carol] – hard to query individual items).
- **Redundancy** (StudentName/Email repeated for each judge/feedback entry).
- **Anomalies**
 - Insertion (can't add a judge evaluation without re-entering submission details)
 - Deletion (deleting a submission loses all embedded judges/awards/notifications)
 - Update (changing student email requires editing multiple cells/rows).

- **Leads to:** 1NF by splitting repeating groups into atomic rows/tables and adding PKs for uniqueness.

- **Total tables: 1**

11.2. 1NF (First Normal Form)

- **Constraints**

- Atomic values (single value per attribute)

- No repeating groups (one row per multi-entry)
- PK for each table's uniqueness.

- **Tables**

- **Submission** (submissionid [PK SERIAL], studentname, studentrollno, speciality, yearofstudy, institutename, experienceyears, email, phonenumber, address, dateofbirth, gender, profilephotourl, artworktitle, medium, dimensions, yearcreated, description, imageurl, isforsale, creationdate, exhibitionname, startdate, enddate, status, venue, address, capacity, contactperson, contactphone, organizedby, contactemail, theme, date [CHECK date > '2020-01-01'])
- **Evaluation** (evaluationid [PK SERIAL], submissionid [FK → Submission.submissionid], judgename, expertise, judgetype, experienceyears, organization, score [CHECK score BETWEEN 0 AND 10], comments, evaluationdate [NOT NULL])
- **Feedback** (feedbackid [PK SERIAL], submissionid [FK → Submission.submissionid], visitorticket, tickettype, visitdate, feedback, feedbackratings [CHECK feedbackratings BETWEEN 1 AND 5])
- **Award** (awardid [PK SERIAL], submissionid [FK → Submission.submissionid], awardname, prizeamount [CHECK prizeamount >= 0], category, year [CHECK year > 2000], sponsor)
- **Notification** (notificationid [PK SERIAL], submissionid [FK → Submission.submissionid], notificationmessage [NOT NULL], notificationtimestamp [DEFAULT CURRENT_TIMESTAMP NOT NULL], notificationstatus [DEFAULT 'Unread' CHECK notificationstatus IN ('Unread', 'Read', 'Deleted')], recipienttype)
- **Portfolio** (portfolioid [PK SERIAL], submissionid [FK → Submission.submissionid], bio, specialization, experienceyears [CHECK experienceyears >= 0 DEFAULT 0], contactemail, websiteurl, sociallinks [type: JSONB])
- **APPA** (appa_id [PK SERIAL], submissionid [FK → Submission.submissionid], appa_trend, recommendation, confidence [CHECK confidence BETWEEN 0 AND 1])

- **Reasoning**

- From 0NF FlatAEMD, repeating groups split into atomic rows

- JudgeNames/Expertise/Scores → Evaluation rows, one per judge
 - VisitorTickets/TicketType/FeedbackRatings → Feedback rows
 - AwardNames/PrizeAmount/Category → Award rows
 - NotificationMessage/Timestamp/Status → Notification rows
 - APPA_Trend/Recommendation/Confidence → APPA rows
 - Portfolio/Bio/Specialization → Portfolio rows
- SERIAL PKs ensure uniqueness
- CHECK for domain (e.g., score 0-10).
- FK notes for links.
- **Issues**
 - **Partial dependencies** (e.g., in Evaluation, expertise depends only on judgements part of implied composite PK submissionid + judgement – update expertise requires multiple rows).
 - **Redundancy** (e.g., studentname repeated in Submission for multiple evaluations).
 - **Anomalies**
 - Insertion (add judge eval OK, but can't add judge details independently)
 - Deletion (delete submission loses judge comments)
 - Update (changing student email requires editing all related submissions).
- **Leads to:** 2NF by splitting partial dep attributes (e.g., expertise to Judge table) with FKs for full PK deps.
- **Total tables: 7**

11.3. 2NF (Second Normal Form)

- **Constraints**
 - Full dependency on composite PKs (non-key attributes depend on entire PK)
 - FKs to enforce relationships and isolate partial deps
 - UNIQUE for composites where needed.
- **Tables**

- **User** (userid [PK SERIAL], roletype [CHECK roletype IN ('Student', 'Judge', 'Visitor', 'Organizer') NOT NULL])
- **UserDetails** (userdetailid [PK SERIAL], userid [FK → User.userid UNIQUE NOT NULL], email [UNIQUE NOT NULL], name [NOT NULL], phonenumber, address, dateofbirth, gender, profilephotourl)
- **Student** (userid [PK/FK → User.userid], studentrollno [UNIQUE NOT NULL], speciality, yearofstudy [CHECK yearofstudy BETWEEN 1 AND 4], institutename, experienceyears [CHECK experienceyears >= 0 DEFAULT 0])
- **Organizer** (userid [PK/FK → User.userid], contactemail [UNIQUE NOT NULL])
- **Judge** (userid [PK/FK → User.userid], expertise, judgetype, experienceyears [CHECK experienceyears >= 0 DEFAULT 0], organization)
- **Visitor** (userid [PK/FK → User.userid], tickettype [DEFAULT 'General'], visitdate, feedback)
- **Artwork** (studentid [partial PK/FK → Student.userid NOT NULL], artworkid [partial PK SERIAL], title [NOT NULL], medium, dimensions, yearcreated [CHECK yearcreated > 1900], description, imageurl, isforsale [DEFAULT FALSE], creationdate [DEFAULT CURRENT_TIMESTAMP])
- **Exhibition** (exhibitionid [PK SERIAL], name [NOT NULL], description, startdate [NOT NULL], enddate [NOT NULL CHECK enddate > startdate], status [DEFAULT 'Planned' CHECK status IN ('Planned', 'Active', 'Archived')], venueid [FK → Venue.venueid], organizedby [FK → Organizer.userid], theme)
- **Venue** (venueid [PK SERIAL], name [NOT NULL], address, capacity [CHECK capacity > 0], contactperson, contactphone)
- **Submission** (submissionid [PK SERIAL], submissiondate [DEFAULT CURRENT_TIMESTAMP NOT NULL], status [DEFAULT 'Pending' CHECK status IN ('Pending', 'Approved', 'Rejected')], artworkid [FK → Artwork.artworkid NOT NULL], exhibitionid [FK → Exhibition.exhibitionid NOT NULL], comments, UNIQUE (artworkid, exhibitionid))

- **Evaluation** (evaluationid [PK SERIAL], score [CHECK score BETWEEN 0 AND 10], comments, evaluationdate [DEFAULT CURRENT_TIMESTAMP], judgeid [FK → Judge.userid NOT NULL], artworkid [FK → Artwork.artworkid], exhibitionid [FK → Exhibition.exhibitionid], submissionid [FK → Submission.submissionid], UNIQUE (judgeid, artworkid, exhibitionid))
- **Award** (awardid [PK SERIAL], awardname [NOT NULL], prizeamount [CHECK prizeamount >= 0], artworkid [FK → Artwork.artworkid], category, year [CHECK year > 2000], sponsor)
- **Notification** (notificationid [PK SERIAL], message [NOT NULL], timestamp [DEFAULT CURRENT_TIMESTAMP NOT NULL], status [DEFAULT 'Unread' CHECK status IN ('Unread', 'Read', 'Deleted')], recipienttype, userid [FK → User.userid])
- **Portfolio** (portfolioid [PK SERIAL], studentid [FK → Student.userid UNIQUE NOT NULL], bio, specialization, experienceyears [CHECK experienceyears >= 0 DEFAULT 0], contactemail, websiteurl, sociallinks [type: JSONB])
- **PortfolioArtwork** (portfolioid [PK/FK → Portfolio.portfolioid], artworkid [PK/FK → Artwork.artworkid])
- **APPA** (appa_id [PK SERIAL], studentid [FK → Student.userid], exhibitionid [FK → Exhibition.exhibitionid], trend, recommendation, confidence [CHECK confidence BETWEEN 0 AND 1], UNIQUE (studentid, exhibitionid))
- **Reasoning**
 - From 1NF's Submission/Evaluation/etc., partial dependencies (e.g., in Evaluation, expertise depended on judge part of implied composite PK submissionid + judgename – update expertise required multiple rows) are removed by splitting role attributes to specialized tables (e.g., expertise to Judge; FD: JudgeID → Expertise now full in Judge PK).
 - FKs (e.g., evaluation.judgeid → judge.userid) enforce full dep on composites like evaluation(evaluationid).
 - User base added for roles (partial dependency on roletype → specific attributes like studentrollno moved to Student).

- CHECK/UNIQUE for domain (e.g., unique artworkid per studentid).
- Solves deletion anomaly (delete judge doesn't lose evals; FK cascades).
- **Issues**
 - **Transitive dependencies** (e.g., in Exhibition, organizedby → organizer.contactemail is transitive via FK to Organizer – update contactemail requires propagating if not FK-isolated)
 - **Redundancy** minimal but present (e.g., studentrollno depends on userid, but if user roletype changes, affects student table indirectly)
 - **Anomalies**
 - Insertion (add organizer without contactemail OK, but transitive update for contactemail across exhibitions)
 - Deletion (delete organizer orphans organizedby FKs)
 - Update (change expertise once in Judge, but transitive if organization → sponsor in Award)
- **Leads to:** 3NF/BCNF by isolating transitive deps with FKs/UNIQUE (e.g., unique on appa.studentid + exhibitionid) and verifying superkeys.
- **Total tables: 16**

11.4. 3NF (Third Normal Form)

- **Constraints**
 - No transitive dependencies (non-key attributes depend only on PK)
 - BCNF (every determinant is a superkey)
 - FKs/UNIQUE/CHECK for integrity/domain
- **Tables**
 - **User** (userid [PK SERIAL], roletype [CHECK roletype IN ('Student', 'Judge', 'Visitor', 'Organizer') NOT NULL])
 - **artwork** (artworkid [PK SERIAL], title [NOT NULL], medium, dimensions, yearcreated [CHECK yearcreated > 1900], description, imageurl, studentid [FK → student.userid NOT NULL], isforsale [DEFAULT FALSE], creationdate [DEFAULT CURRENT_TIMESTAMP])

- **award** (awardid [PK SERIAL], awardname [NOT NULL], prizeamount [CHECK prizeamount >= 0], artworkid [FK → artwork.artworkid], category, year [CHECK year > 2000], sponsor)
- **evaluation** (evaluationid [PK SERIAL], score [CHECK score BETWEEN 0 AND 10], comments, evaluationdate [DEFAULT CURRENT_TIMESTAMP], judgeid [FK → judge.userid NOT NULL], artworkid [FK → artwork.artworkid], exhibitionid [FK → exhibition.exhibitionid], submissionid [FK → submission.submissionid], UNIQUE (judgeid, artworkid, exhibitionid))
- **exhibition** (exhibitionid [PK SERIAL], name [NOT NULL], description, startdate [NOT NULL], enddate [NOT NULL CHECK enddate > startdate], status [DEFAULT 'Planned' CHECK status IN ('Planned', 'Active', 'Archived')], venueid [FK → venue.venueid], organizedby [FK → organizer.userid], theme)
- **judge** (userid [PK/FK → User.userid], expertise, judgetype, experienceyears [CHECK experienceyears >= 0 DEFAULT 0], organization)
- **notification** (notificationid [PK SERIAL], message [NOT NULL], timestamp [DEFAULT CURRENT_TIMESTAMP NOT NULL], status [DEFAULT 'Unread' CHECK status IN ('Unread', 'Read', 'Deleted')], recipienttype, userid [FK → User.userid])
- **organizer** (userid [PK/FK → User.userid], contactemail [UNIQUE NOT NULL])
- **portfolio** (portfolioid [PK SERIAL], studentid [FK → student.userid UNIQUE NOT NULL], bio, specialization, experienceyears [CHECK experienceyears >= 0 DEFAULT 0], contactemail, websiteurl, sociallinks [type: JSONB])
- **portfolioartwork** (portfolioid [FK → portfolio.portfolioid], artworkid [FK → artwork.artworkid], PRIMARY KEY (portfolioid, artworkid))
- **student** (userid [PK/FK → User.userid], studentrollno [UNIQUE NOT NULL], speciality, yearofstudy [CHECK yearofstudy BETWEEN 1 AND 4], institutename, experienceyears [CHECK experienceyears >= 0 DEFAULT 0])

- **submission** (submissionid [PK SERIAL], submissiondate [DEFAULT CURRENT_TIMESTAMP NOT NULL], status [DEFAULT 'Pending' CHECK status IN ('Pending', 'Approved', 'Rejected')], artworkid [FK → artwork.artworkid NOT NULL], exhibitionid [FK → exhibition.exhibitionid NOT NULL], comments, UNIQUE (artworkid, exhibitionid))
- **userdetails** (userdetailid [PK SERIAL], email [UNIQUE NOT NULL], name [NOT NULL], phonenumber, address, dateofbirth, gender, profilephotourl, userid [FK → User.userid UNIQUE NOT NULL])
- **venue** (venueid [PK SERIAL], name [NOT NULL], address, capacity [CHECK capacity > 0], contactperson, contactphone)
- **visitor** (userid [PK/FK → User.userid], tickettype [DEFAULT 'General'], visitdate, feedback)
- **appa** (appa_id [PK SERIAL], studentid [FK → student.userid], exhibitionid [FK → exhibition.exhibitionid], trend, recommendation, confidence [CHECK confidence BETWEEN 0 AND 1], analyzed_at [DEFAULT CURRENT_TIMESTAMP], UNIQUE (studentid, exhibitionid))
- **Reasoning**
 - From 2NF, no transitive deps (e.g., submission.artworkid → artwork.studentid is via FK, not in submission – studentid isolated in student table). BCNF holds (e.g., no FD like judge.userid → organization where userid not superkey – organization depends on userid PK).
 - Added UNIQUE on composites (e.g., appa.studentid + exhibitionid prevents dupes without partial/transitive).
 - CHECK for domain (e.g., score 0-10).
 - Solves all anomalies (insert/update/delete safe; e.g., add APPA trend without full student details via FK).
- **Issues**
 - None (fully normalized; no transitive/partial deps; FKs/UNIQUE/CHECK ensure integrity).
- **Total tables: 16**

11.5. Final Normalized Schemas

The final relations in 3NF/BCNF are listed below. Each is denoted as $R_name_3NF(attributes)$, with PK underlined (e.g., **userid** [PK]). FKs noted in parentheses.

- **R_User_3NF** (**userid** [PK], roletype)
- **R_artwork_3NF** (**artworkid** [PK], title, medium, dimensions, yearcreated, description, imageurl, studentid [FK → R_student.userid], isforsale, creationdate)
- **R_award_3NF** (**awardid** [PK], awardname, prizeamount, artworkid [FK → R_artwork.artworkid], category, year, sponsor)
- **R_evaluation_3NF** (**evaluationid** [PK], score, comments, evaluationdate, judgeid [FK → R_judge.userid], artworkid [FK → R_artwork.artworkid], exhibitionid [FK → R_exhibition.exhibitionid], submissionid [FK → R_submission.submissionid])
- **R_exhibition_3NF** (**exhibitionid** [PK], name, description, startdate, enddate, status, venueid [FK → R_venue.venueid], organizedby [FK → R_organizer.userid], theme)
- **R_judge_3NF** (**userid** [PK/FK → R_User.userid], expertise, judgetype, experienceyears, organization)
- **R_notification_3NF** (**notificationid** [PK], message, timestamp, status, recipienttype, userid [FK → R_User.userid])
- **R_organizer_3NF** (**userid** [PK/FK → R_User.userid], contactemail)
- **R_portfolio_3NF** (**portfolioid** [PK], studentid [FK → R_student.userid], bio, specialization, experienceyears, contactemail, websiteurl, sociallinks)
- **R_portfolioartwork_3NF** (**portfolioid** [PK/FK → R_portfolio.portfolioid], **artworkid** [PK/FK → R_artwork.artworkid])
- **R_student_3NF** (**userid** [PK/FK → R_User.userid], studentrollno, speciality, yearofstudy, institutenname, experienceyears)
- **R_submission_3NF** (**submissionid** [PK], submissiondate, status, artworkid [FK → R_artwork.artworkid], exhibitionid [FK → R_exhibition.exhibitionid], comments)
- **R_userdetails_3NF** (**userdetailid** [PK], email, name, phonenumber, address, dateofbirth, gender, profilephotourl, userid [FK → R_User.userid])

- **R_venue_3NF** (venueid [PK], name, address, capacity, contactperson, contactphone)
- **R_visitor_3NF** (userid [PK/FK → R_User.userid], tickettype, visitdate, feedback)
- **R_appa_3NF** (appa_id [PK], studentid [FK → R_student.userid], exhibitionid [FK → R_exhibition.exhibitionid], trend, recommendation, confidence, analyzed_at)

12. DDL Scripts

I. User Table

```
CREATE TABLE "User" (
  userid SERIAL PRIMARY KEY,
  roletype CHARACTER VARYING(50) NOT NULL CHECK (roletype IN ('Student', 'Judge', 'Visitor', 'Organizer'))
);
```

II. Artwork Table

```
CREATE TABLE artwork (
  artworkid SERIAL PRIMARY KEY,
  title CHARACTER VARYING(200) NOT NULL,
  medium CHARACTER VARYING(50),
  dimensions CHARACTER VARYING(50),
  yearcreated INTEGER CHECK (yearcreated > 1900),
  description TEXT,
  imageurl TEXT,
  studentid INTEGER NOT NULL REFERENCES student(userid) ON DELETE CASCADE,
  isforsale BOOLEAN DEFAULT FALSE,
  creationdate DATE DEFAULT CURRENT_DATE
);
```

III. Award Table

```
CREATE TABLE award (
  awardid SERIAL PRIMARY KEY,
  awardname CHARACTER VARYING(200) NOT NULL,
  prizeamount NUMERIC(10,2) CHECK (prizeamount >= 0),
  artworkid INTEGER REFERENCES artwork(artworkid) ON DELETE CASCADE,
  category CHARACTER VARYING(50),
  year INTEGER CHECK (year > 2000),
```

```

sponsor CHARACTER VARYING(100)
);

```

IV. Evaluation Table

```

CREATE TABLE evaluation (
  evaluationid SERIAL PRIMARY KEY,
  score NUMERIC CHECK (score BETWEEN 0 AND 10),
  comments TEXT,
  evaluationdate DATE DEFAULT CURRENT_DATE,
  judgeid INTEGER NOT NULL REFERENCES judge(userid) ON DELETE CASCADE,
  artworkid INTEGER REFERENCES artwork(artworkid) ON DELETE CASCADE,
  exhibitionid INTEGER REFERENCES exhibition(exhibitionid) ON DELETE CASCADE,
  submissionid INTEGER REFERENCES submission(submissionid) ON DELETE CASCADE,
  UNIQUE (judgeid, artworkid, exhibitionid)
);

```

V. Exhibition Table

```

CREATE TABLE exhibition (
  exhibitionid SERIAL PRIMARY KEY,
  name CHARACTER VARYING(200) NOT NULL,
  description TEXT,
  startdate DATE NOT NULL,
  enddate DATE NOT NULL CHECK (enddate > startdate),
  status CHARACTER VARYING(20) DEFAULT 'Planned' CHECK (status IN ('Planned', 'Active', 'Archived')),
  venueid INTEGER REFERENCES venue(venueid) ON DELETE SET NULL,
  organizedby INTEGER REFERENCES organizer(userid) ON DELETE SET NULL,
  theme CHARACTER VARYING(100)
);

```

VI. Judge Table

```

CREATE TABLE judge (
  userid INTEGER PRIMARY KEY REFERENCES "User"(userid) ON DELETE CASCADE,
  expertise CHARACTER VARYING(100),
  judgetype CHARACTER VARYING(50),
  experienceyears INTEGER DEFAULT 0 CHECK (experienceyears >= 0),
  organization CHARACTER VARYING(100)
);

```

VII. Notification Table

```

CREATE TABLE notification (

```

```
notificationid SERIAL PRIMARY KEY,
message TEXT NOT NULL,
timestamp TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP NOT NULL,
status CHARACTER VARYING(20) DEFAULT 'Unread' CHECK (status IN ('Unread', 'Read', 'Deleted')),
recipienttype CHARACTER VARYING(50),
userid INTEGER REFERENCES "User"(userid) ON DELETE CASCADE
);
```

VIII. Organizer Table

```
CREATE TABLE organizer (
userid INTEGER PRIMARY KEY REFERENCES "User"(userid) ON DELETE CASCADE,
contactemail CHARACTER VARYING(100) UNIQUE NOT NULL
);
```

IX. Portfolio Table

```
CREATE TABLE portfolio (
portfolioid SERIAL PRIMARY KEY,
studentid INTEGER UNIQUE NOT NULL REFERENCES student(userid) ON DELETE CASCADE,
bio TEXT,
specialization CHARACTER VARYING(100),
experienceyears INTEGER DEFAULT 0 CHECK (experienceyears >= 0),
contactemail CHARACTER VARYING(100),
websiteurl TEXT,
sociallinks TEXT
);
```

X. Portfolioartwork Table

```
CREATE TABLE portfolioartwork (
portfolioid INTEGER REFERENCES portfolio(portfolioid) ON DELETE CASCADE,
artworkid INTEGER REFERENCES artwork(artworkid) ON DELETE CASCADE,
PRIMARY KEY (portfolioid, artworkid)
);
```

XI. Student Table

```
CREATE TABLE student (
userid INTEGER PRIMARY KEY REFERENCES "User"(userid) ON DELETE CASCADE,
studentrollno CHARACTER VARYING(20) UNIQUE NOT NULL,
speciality CHARACTER VARYING(50),
yearofstudy INTEGER CHECK (yearofstudy BETWEEN 1 AND 4),
institutename CHARACTER VARYING(100),
```

```
experienceyears INTEGER DEFAULT 0 CHECK (experienceyears >= 0)
);
```

XII. Submission Table

```
CREATE TABLE submission (
  submissionid SERIAL PRIMARY KEY,
  submissiondate DATE DEFAULT CURRENT_DATE NOT NULL,
  status CHARACTER VARYING(20) DEFAULT 'Pending' CHECK (status IN ('Pending', 'Approved', 'Rejected')),
  artworkid INTEGER NOT NULL REFERENCES artwork(artworkid) ON DELETE CASCADE,
  exhibitionid INTEGER NOT NULL REFERENCES exhibition(exhibitionid) ON DELETE CASCADE,
  comments TEXT,
  UNIQUE (artworkid, exhibitionid)
);
```

XIII. Userdetails Table

```
CREATE TABLE userdetails (
  userdetailid SERIAL PRIMARY KEY,
  email CHARACTER VARYING(100) UNIQUE NOT NULL,
  name CHARACTER VARYING(100) NOT NULL,
  phonenumber CHARACTER VARYING(20),
  address TEXT,
  dateofbirth DATE,
  gender CHARACTER VARYING(10),
  profilephotourl TEXT,
  userid INTEGER UNIQUE NOT NULL REFERENCES "User"(userid) ON DELETE CASCADE
);
```

XIV. Venue Table

```
CREATE TABLE venue (
  venueid SERIAL PRIMARY KEY,
  name CHARACTER VARYING(100) NOT NULL,
  address TEXT,
  capacity INTEGER CHECK (capacity > 0),
  contactperson CHARACTER VARYING(100),
  contactphone CHARACTER VARYING(20)
);
```

XV. Visitor Table

```
CREATE TABLE visitor (
  userid INTEGER PRIMARY KEY REFERENCES "User"(userid) ON DELETE CASCADE,
```

```
tickettype CHARACTER VARYING(50) DEFAULT 'General',
visitdate DATE,
feedback TEXT
);
```

XVI. Appa Table

```
CREATE TABLE appa (
  appa_id SERIAL PRIMARY KEY,
  studentid INTEGER REFERENCES student(userid) ON DELETE CASCADE,
  exhibitionid INTEGER REFERENCES exhibition(exhibitionid) ON DELETE CASCADE,
  trend CHARACTER VARYING(200),
  recommendation TEXT,
  confidence FLOAT CHECK (confidence BETWEEN 0 AND 1),
  analyzed_at TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (studentid, exhibitionid)
);
```

13. Snapshots

13.1. User Table

```
[temp=# SELECT * FROM "User" LIMIT 5;
  userid | roletype
-----+-----
      1 | Student
      2 | Student
      3 | Student
      4 | Student
      5 | Student
(5 rows)
```

-
- Number Of Records: 400

13.2. Artwork Table

artworkid	title	medium	dimensions	yearcreated	description	imageurl	studentid
isforsale							
1	Work 1	Oil on Canvas	71x132 cm	2021	Generated artwork 1 by student 1	https://example.com/art/1.jpg	1
2	Work 2	Oil on Canvas	37x148 cm	2022	Generated artwork 2 by student 1	https://example.com/art/2.jpg	1
3	Work 3	Mixed Media	60x102 cm	2021	Generated artwork 3 by student 2	https://example.com/art/3.jpg	2
4	Work 4	Digital Print	86x76 cm	2024	Generated artwork 4 by student 2	https://example.com/art/4.jpg	2
5	Work 5	Photograph	27x64 cm	2024	Generated artwork 5 by student 3	https://example.com/art/5.jpg	3

(5 rows)

- Number Of Records: 200

13.3. Award Table

awardid	awardname	prizeamount	artworkid	category	year	sponsor
1	Curator's Pick	2000.00	102	Popular	2025	Gallery One
2	Best in Medium	3000.00	163	Overall	2025	City Arts Council
3	People's Choice	2000.00	119	Digital	2025	City Arts Council
4	Best in Show	2000.00	42	Sculpture	2025	Art Foundation
5	Curator's Pick	2000.00	177	Student	2025	Canvas Collective

(5 rows)

- Number Of Records: 180

13.4. Evaluation Table

evaluationid	score	comments	evaluationdate	judgeid	artworkid	exhibitionid	submissionid
1	81.31	Compelling idea	2025-01-06	109	119	1	1
2	88.73	Needs refinement	2025-01-28	111	130	1	2
3	90.92	Good technique	2025-01-13	112	102	1	3
4	84.94	Good technique	2025-01-26	104	163	1	4
5	50.19	Work on contrast	2024-12-29	109	32	1	5

(5 rows)

- Number Of Records: 300

13.5. Exhibition Table

exhibitionid	name	description	startdate	enddate	status	venueid	organizedby	theme
1	Exhibition 1	Curated show number 1	2025-01-10	2025-01-19	Completed	1	121	Photography
2	Exhibition 2	Curated show number 2	2025-01-20	2025-01-29	Completed	2	122	Photography
3	Exhibition 3	Curated show number 3	2025-01-30	2025-02-08	Completed	3	123	Photography
4	Exhibition 4	Curated show number 4	2025-02-09	2025-02-18	Completed	4	124	Materiality
5	Exhibition 5	Curated show number 5	2025-02-19	2025-02-28	Completed	5	125	Digital Media

(5 rows)

- Number Of Records: 60

13.6. Judge Table

```
temp=# SELECT * FROM judge LIMIT 5;
```

userid	expertise	judgetype	experienceyears	organization
101	Painting	External	25	Clayworks Society
102	Curation	Guest	10	University of Fine Arts
103	Textile Art	Senior	18	Design Guild
104	Textile Art	External	11	Cultural Arts Org
105	Painting	Guest	27	Visual Arts Institute

(5 rows)

- Number Of Records: 20

13.7. Notification Table

```
temp=# SELECT * FROM notification LIMIT 5;
```

notificationid	message	timestamp	status	recipienttype	userid
6	Evaluation due for exhibition 27	2025-06-18 10:44:25.9008	Read	Organizer	284
7	Award allocated: Best Painting	2025-11-18 22:43:45.8112	Read	Organizer	289
8	Submission approved for artwork ID 71	2025-02-15 14:40:52.176	Unread	Student	26
9	Award allocated: Best Painting	2025-10-20 05:44:12.7104	Unread	Organizer	146
10	Submission approved for artwork ID 194	2025-03-25 11:13:38.8704	Read	Student	210

(5 rows)

- Number Of Records: 200

13.8. Organizer Table

```
temp=# SELECT * FROM organizer LIMIT 5;
```

userid	contactemail
121	contact121@orgexample.org
122	contact122@orgexample.org
123	contact123@orgexample.org
124	contact124@orgexample.org
125	contact125@orgexample.org

(5 rows)

- Number Of Records: 20

13.9. Portfolio Table

```
(temp=# SELECT * FROM portfolio LIMIT 5;
```

portfolioid	studentid	bio	specialization	experienceyears	contactemail	websiteurl	sociallinks
1	1	Portfolio of student 1 focusing on Photography	Textile Art	2	portfolio1@example.com	https://portfolio.example.com/1	{ "instagram": "@artist1" }
2	2	Portfolio of student 2 focusing on Digital Art	Digital Art	1	portfolio2@example.com	https://portfolio.example.com/2	{ "instagram": "@artist2" }
3	3	Portfolio of student 3 focusing on Graphic Design	Graphic Design	0	portfolio3@example.com	https://portfolio.example.com/3	{ "instagram": "@artist3" }
4	4	Portfolio of student 4 focusing on Graphic Design	Digital Art	1	portfolio4@example.com	https://portfolio.example.com/4	{ "instagram": "@artist4" }
5	5	Portfolio of student 5 focusing on Ceramics	Printmaking	2	portfolio5@example.com	https://portfolio.example.com/5	{ "instagram": "@artist5" }

(5 rows)

- Number Of Records: 100

13.10. Portfolioartwork Table

```
[temp=# SELECT * FROM portfolioartwork LIMIT 5;
```

portfolioid	artworkid
1	1
1	2
2	3
2	4
3	5

(5 rows)

- Number Of Records: 200

13.11. Student Table

```
(temp=# SELECT * FROM student LIMIT 5;
```

userid	studentrollno	speciality	yearofstudy	institutename	experienceyears
1	STU2025-001	Graphic Design	5	Indian School of Art	3
2	STU2025-002	Digital Art	2	Indian School of Art	1
3	STU2025-003	Ceramics	1	Art & Design Institute	2
4	STU2025-004	Photography	4	School of Creative Studies	3
5	STU2025-005	Photography	5	Visual Arts Academy	2

(5 rows)

- Number Of Records: 100

13.12. Submission Table

```
temp=# SELECT * FROM submission LIMIT 5;
```

submissionid	submissiondate	status	artworkid	exhibitionid	comments	userid
1	2024-12-19	Pending	119	1	Auto-submission 1	390
2	2025-01-02	Rejected	130	1	Auto-submission 2	260
3	2024-12-25	Under Review	102	1	Auto-submission 3	11
4	2025-01-04	Under Review	163	1	Auto-submission 4	116
5	2024-12-22	Rejected	32	1	Auto-submission 5	184

(5 rows)

-
- Number Of Records: 300

13.13. Userdetails Table

```
temp=# SELECT * FROM userdetails LIMIT 5;
```

userid	email	name	phonenumber	address	dateofbirth	gender	profilephotourl	userid
1	kajal.mehta1@example.com	Kajal Mehta	+919466027313	150 Park St, Indore, India	1999-03-17	Female		1
2	meera.gopal2@example.com	Meera Gopal	+919060590821	29 Lake St, Raipur, India	1999-11-14	Other	https://randomuser.me/api/portraits/men/2.jpg	2
3	aditya.khurana3@example.com	Aditya Khurana	+919405760504	153 Park St, Visakhapatnam, India	2000-09-28	Male		3
4	komal.patil4@example.com	Komal Patil	+919122998994	175 Hill St, Madurai, India	2000-11-11	Male		4
5	ruhi.pandey5@example.com	Ruhi Pandey	+919169820594	117 Green St, Hubli, India	2003-05-17	Male	https://randomuser.me/api/portraits/women/5.jpg	5

(5 rows)

-
- Number Of Records: 200

13.14. Venue Table

```
temp=# SELECT * FROM venue LIMIT 5;
```

venueid	name	address	capacity	contactperson	contactphone
1	Venue 1 Arts Center	1 Cultural Rd, Vadodara	250	Ms. Priya Menon	+919753703117
2	Venue 2 Arts Center	2 Cultural Rd, Agra	350	Mr. Anil Kapoor	+919861135562
3	Venue 3 Arts Center	3 Cultural Rd, Ranchi	200	Ms. Nisha Rao	+919730511568
4	Venue 4 Arts Center	4 Cultural Rd, Madurai	500	Mr. Rohit Verma	+919564258474
5	Venue 5 Arts Center	5 Cultural Rd, Patna	400	Mr. Sanjay Roy	+919083862154

(5 rows)

-
- Number Of Records: 20

13.15. Visitor Table

```
temp=# SELECT * FROM visitor LIMIT 5;
```

userid	tickettype	visitdate	feedback	feedbackratings
141	Multi-day	2025-01-19	Loved the student works.	5
142	VIP	2025-09-21	Enjoyed the workshops.	4
143	VIP	2025-07-05	Loved the student works.	4
144	VIP	2025-01-19	Great curation.	2
145	Multi-day	2025-04-01	Well organized.	2

(5 rows)

-
- Number Of Records: 60

13.16. Appa Table

```
temp=# SELECT * FROM appa LIMIT 5;
```

appa_id	studentid	exhibitionid	trend	recommendation	confidence	analyzed_at
1	1	13	Technique low	Practice skills	0	2025-11-16 14:41:36.534819
2	1	4	Technique low	Practice skills	0	2025-11-16 14:41:36.534819
3	2	39	Technique low	Practice skills	0	2025-11-16 14:41:36.534819
4	2	7	Technique low	Practice skills	0	2025-11-16 14:41:36.534819
5	3	3	Technique low	Practice skills	0	2025-11-16 14:41:36.534819

(5 rows)

-
- Number Of Records: 200

14. SQL Queries

1) List all students.

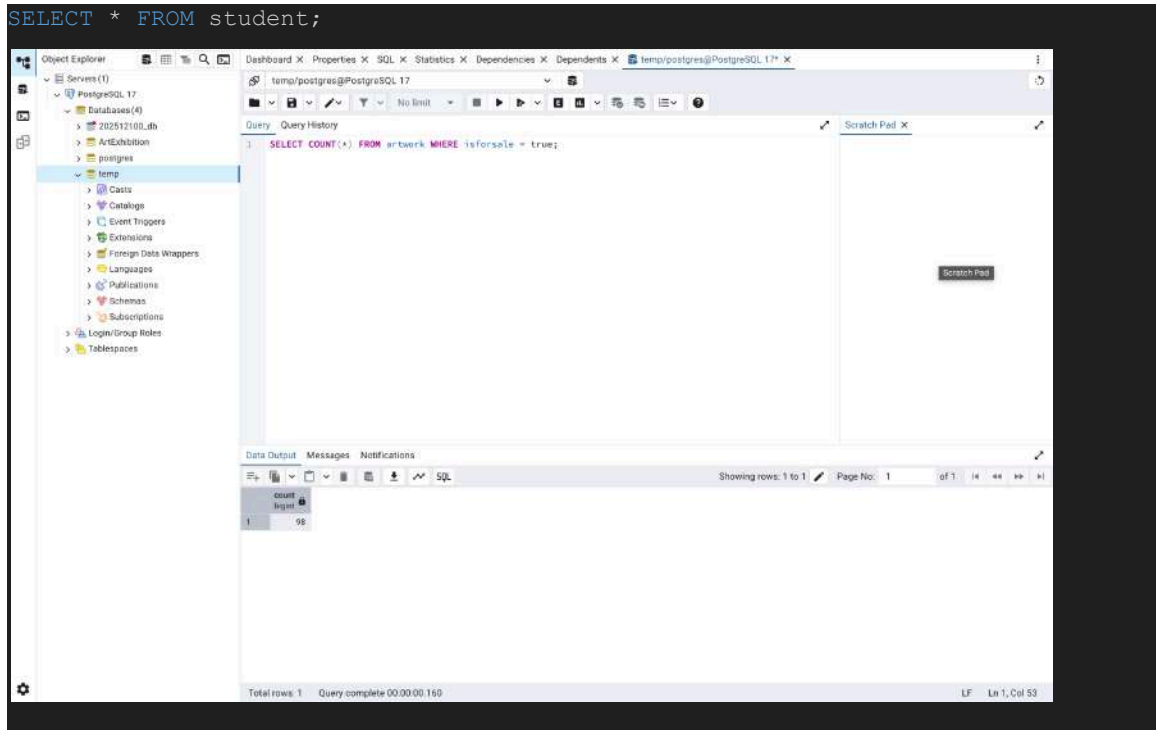
The screenshot shows a PostgreSQL query editor with the query `SELECT * FROM student;` executed. The results are displayed in a table with 10 rows and 6 columns: `userid`, `studentid`, `specialty`, `yearofstudy`, `institute`, and `experienceyears`.

userid	studentid	specialty	yearofstudy	institute	experienceyears
1	1	Graphic Design	5	Indian School of Art	3
2	2	Digital Art	2	Indian School of Art	1
3	3	Ceramics	1	Art & Design Institute	2
4	4	Photography	4	School of Creative Studies	3
5	5	Photography	5	Visual Arts Academy	2
6	6	Ceramics	2	ID College	3
7	7	Ceramics	3	Art & Design Institute	3
8	8	Digital Art	2	Indian School of Art	2
9	9	Painting	5	Clayworks Institute	2
10	10	Printmaking	1	College of Arts Mumbai	2

Total rows: 100 Query complete 00:00:00.234

2) Count artworks for sale.

`SELECT * FROM student;`

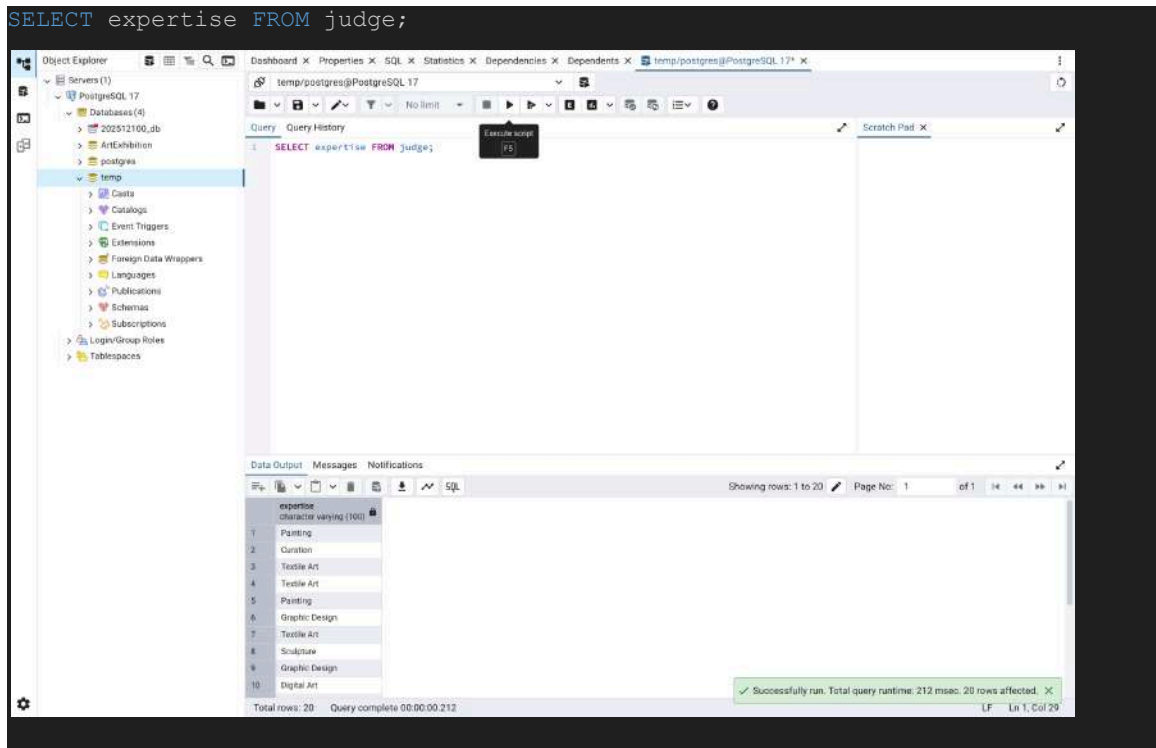


The screenshot shows a PostgreSQL IDE interface. The left pane displays the 'Object Explorer' with a tree view of the database structure, including 'Servers(1)', 'PostgreSQL 17', 'Databases(4)', '202512100.db', 'ArtExhibition', 'postgres', 'temp', 'Casts', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'Login/Group Roles', and 'Tablespaces'. The main query editor shows the query: `SELECT COUNT(*) FROM artwork WHERE isforsale = true;`. The 'Data Output' pane at the bottom displays the result: a single row with the value 98. The status bar at the bottom indicates 'Total rows: 1', 'Query complete 00:00:00.160', and 'LF Ln 1, Col 53'.

count(*)
98

3) Get judge expertise.

`SELECT expertise FROM judge;`

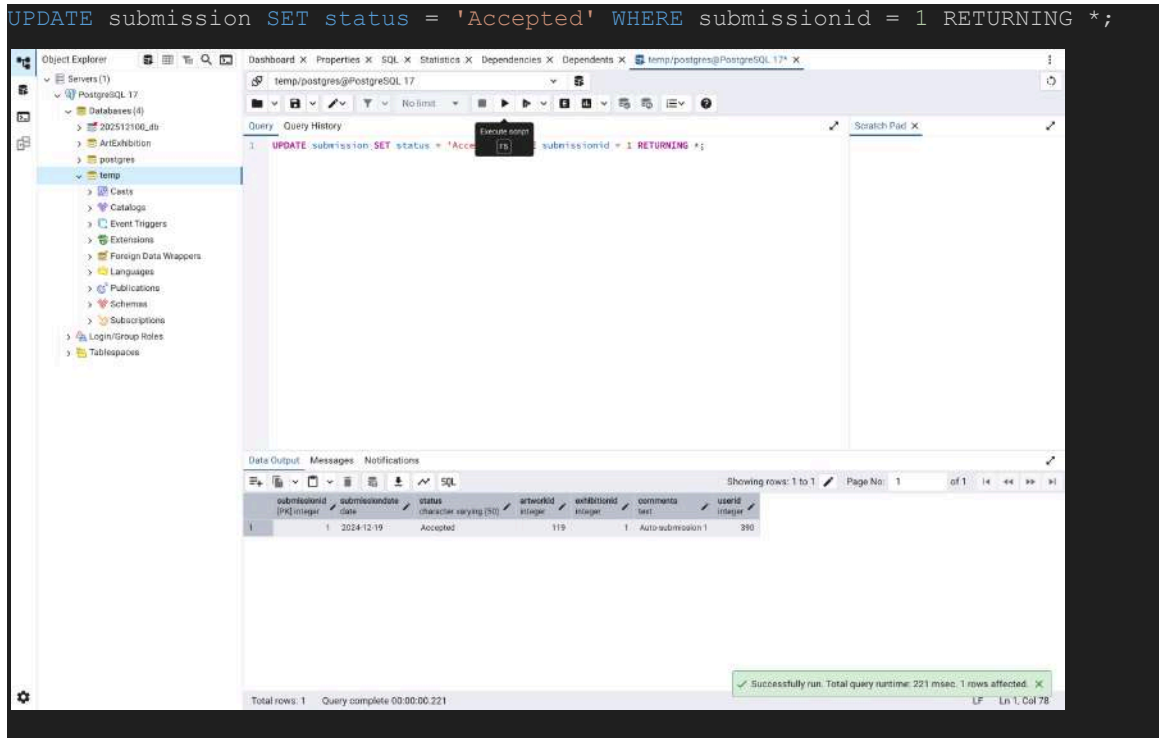


The screenshot shows the same PostgreSQL IDE interface. The main query editor shows the query: `SELECT expertise FROM judge;`. The 'Data Output' pane at the bottom displays the result: a list of 10 expertise values. The status bar at the bottom indicates 'Total rows: 20', 'Query complete 00:00:00.212', and 'LF Ln 1, Col 29'. A green message bar at the bottom right states: 'Successfully run. Total query runtime: 212 msec. 20 rows affected.'

expertise
Painting
Curator
Textile Art
Textile Art
Painting
Graphic Design
Textile Art
Sculpture
Graphic Design
Digital Art

4) Update submission status to 'Approved'.

UPDATE submission SET status = 'Accepted' WHERE submissionid = 1 RETURNING *;



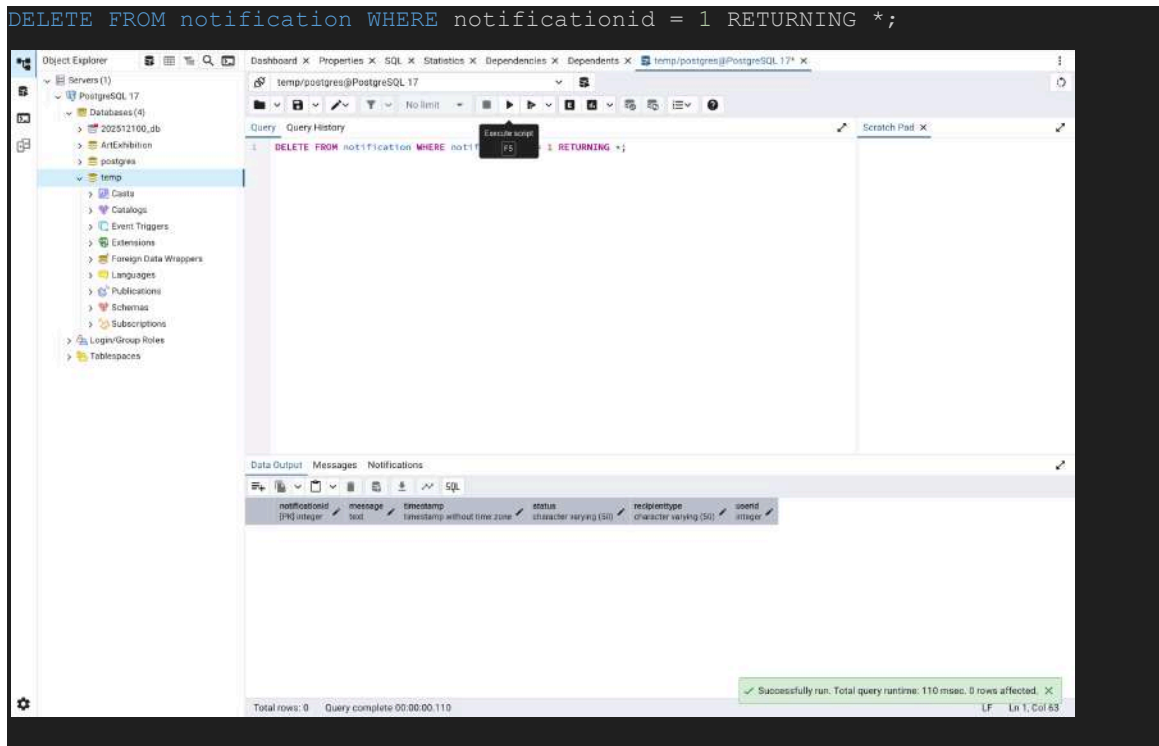
The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left displays the database structure, including the 'temp' database. The Query editor in the center contains the SQL statement: `UPDATE submission SET status = 'Accepted' WHERE submissionid = 1 RETURNING *;`. The Data Output pane at the bottom displays the result of the query, showing a single row with the following columns and values:

submissionid	submissiondate	status	articleid	articleid	comments	userid
1	2024-12-19	Accepted	119	1	Auto-submission 1	390

The status bar at the bottom indicates: Total rows: 1, Query complete 00:00:00.221, and a success message: Successfully run. Total query runtime: 221 msec. 1 rows affected.

5) Delete a notification.

DELETE FROM notification WHERE notificationid = 1 RETURNING *;



The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left displays the database structure, including the 'temp' database. The Query editor in the center contains the SQL statement: `DELETE FROM notification WHERE notificationid = 1 RETURNING *;`. The Data Output pane at the bottom displays the result of the query, showing a single row with the following columns and values:

notificationid	message	timestamp	status	recipienttype	userid
1					

The status bar at the bottom indicates: Total rows: 0, Query complete 00:00:00.110, and a success message: Successfully run. Total query runtime: 110 msec. 0 rows affected.

6) Insert new artwork.

```
INSERT INTO artwork (artworkid, title, studentid) VALUES (201, 'New Art', 1)
RETURNING *;
```

The screenshot shows the PostgreSQL IDE interface. The 'Query' tab is active, displaying the SQL query: `INSERT INTO artwork (artworkid, title, studentid) VALUES (201, 'New Art', 1) RETURNING *;`. The 'Data Output' tab shows the result of the query, which is a single row with the following columns and values:

artworkid	title	medium	dimension	yearcreated	description	imageurl	studentid	isfavorite
201	New Art	[null]	[null]	[null]	[null]	[null]	1	false

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.176'.

7) Check venue capacity.

```
SELECT capacity FROM venue WHERE venueid = 1;
```

The screenshot shows the PostgreSQL IDE interface. The 'Query' tab is active, displaying the SQL query: `SELECT capacity FROM venue WHERE venueid = 1;`. The 'Data Output' tab shows the result of the query, which is a single row with the following columns and values:

capacity
250

The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.235'. A green message box at the bottom right states: 'Successfully run. Total query runtime: 235 msec. 1 rows affected.'

8) Get a portfolio bio.

`SELECT bio FROM portfolio WHERE portfolioid = 1;`

The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left displays the database structure, including the 'temp' schema. The Query Editor in the center contains the SQL query: `SELECT bio FROM portfolio WHERE portfolioid = 1;`. The Data Output pane at the bottom shows the result of the query, which is a single row with the value 'Portfolio of student 1 focusing on Photography'. The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.168'.

bio
Portfolio of student 1 focusing on Photography

Showing rows: 1 to 1 Page No: 1 of 1

Successfully run. Total query runtime: 168 msec. 1 rows affected.

9) List award categories

`SELECT DISTINCT category FROM award;`

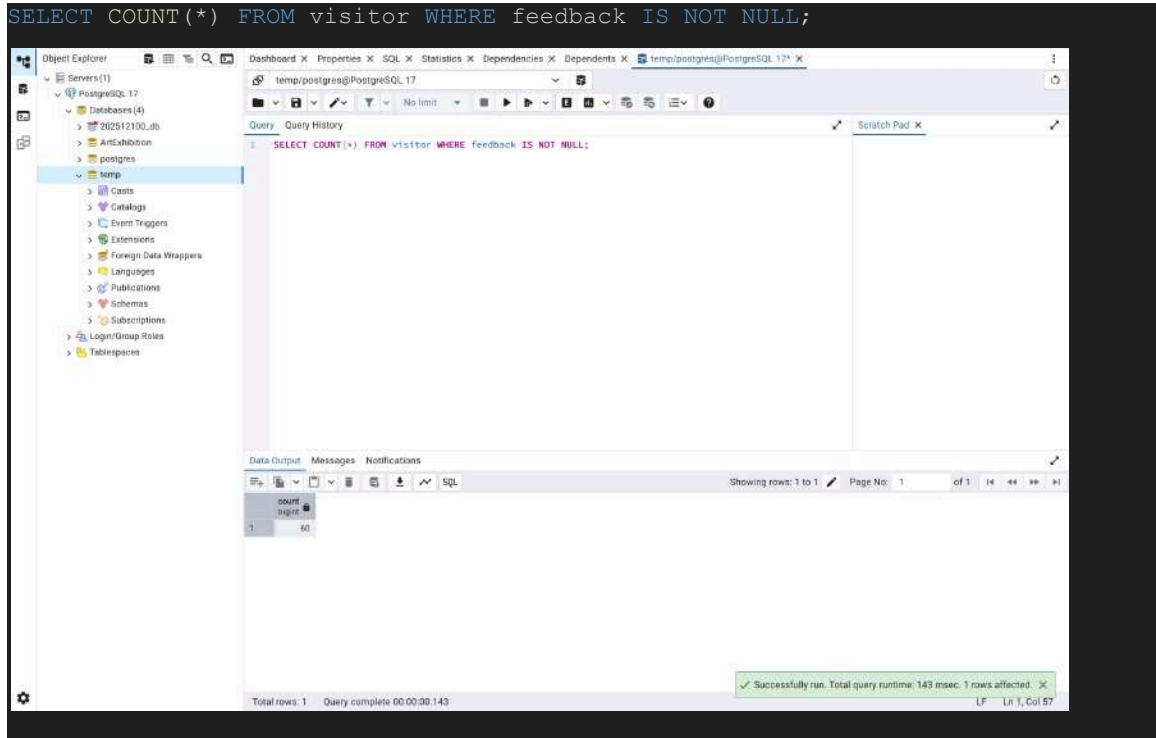
The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left displays the database structure, including the 'temp' schema. The Query Editor in the center contains the SQL query: `SELECT DISTINCT category FROM award;`. The Data Output pane at the bottom shows the result of the query, which is a list of six distinct award categories: Sculpture, Overall, Student, Photography, Digital, and Popular. The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.219'.

category
Sculpture
Overall
Student
Photography
Digital
Popular

Showing rows: 1 to 6 Page No: 1 of 1

Successfully run. Total query runtime: 219 msec. 6 rows affected.

10) Count feedback.



The screenshot shows a PostgreSQL query editor interface. The query being executed is:

```
SELECT COUNT(*) FROM visitor WHERE feedback IS NOT NULL;
```

The query history shows the same query. The data output table is as follows:

count	visitor
1	60

The status bar at the bottom indicates: "Total rows: 1 Query complete 00:00:00.143". A green message box at the bottom right says: "Successfully run. Total query runtime: 143 msec. 1 rows affected."

11) Top-scoring artworks with student and exhibition

```
SELECT a.title, s.studentrollno, e.name, AVG(ev.score) AS avg_score
FROM artwork a
JOIN student s ON a.studentid = s.userid
JOIN submission sub ON a.artworkid = sub.artworkid
JOIN exhibition e ON sub.exhibitionid = e.exhibitionid
JOIN evaluation ev ON sub.submissionid = ev.submissionid
GROUP BY a.title, s.studentrollno, e.name
ORDER BY avg_score DESC LIMIT 3;
```

The screenshot shows the PostgreSQL Client Explorer interface. The left pane displays the database structure, including the 'temp' database. The main pane shows a SQL query in the 'Query' tab:

```

1 SELECT a.title, s.studentrollno, w.name, AVG(ev.score) AS avg_score
2 FROM artwork a
3 JOIN student s ON a.studentid = s.userid
4 JOIN submission sub ON a.artworkid = sub.artworkid
5 JOIN exhibition e ON sub.exhibitionid = e.exhibitionid
6 JOIN evaluation ev ON sub.submissionid = ev.submissionid
7 GROUP BY a.title, s.studentrollno, w.name
8 ORDER BY avg_score DESC LIMIT 3;

```

The 'Data Output' tab shows the results of the query:

	title	studentrollno	name	avg_score
1	Work 7a	STU0225-026	Exhibition 8a	97.830010000000000000
2	Work 143	STU0225-072	Exhibition 9	97.840010000000000000
3	Work 135	STU0225-079	Exhibition 14	97.820010000000000000

The status bar at the bottom indicates: 'Total rows: 3 Query complete 00:00:00.112' and 'Successfully run. Total query runtime: 112 msec. 3 rows affected.'

12) Students with APPA trends in exhibitions with high venue capacity

```

SELECT DISTINCT s.studentrollno, a.trend
FROM student s
JOIN appa a ON s.userid = a.studentid
JOIN exhibition e ON a.exhibitionid = e.exhibitionid
JOIN venue v ON e.venueid = v.venueid
WHERE v.capacity > (SELECT AVG(capacity) FROM venue);

```

The screenshot shows the PostgreSQL Client Explorer interface. The left pane displays the database structure, including the 'public' database. The main pane shows a SQL query in the 'Query' tab:

```

1 SELECT DISTINCT s.studentrollno, a.trend
2 FROM student s
3 JOIN appa a ON s.userid = a.studentid
4 JOIN exhibition e ON a.exhibitionid = e.exhibitionid
5 JOIN venue v ON e.venueid = v.venueid
6 WHERE v.capacity > (SELECT AVG(capacity) FROM venue);

```

The 'Data Output' tab shows the results of the query:

	studentrollno	trend
1	STU0225-056	Creativity up
2	STU0225-073	Technique low
3	STU0225-028	Technique low
4	STU0225-075	Technique low
5	STU0225-061	Balanced growth
6	STU0225-091	Balanced growth
7	STU0225-068	Creativity up
8	STU0225-006	Technique low
9	STU0225-091	Creativity up
10	STU0225-040	Technique low
11	STU0225-064	Creativity up
12	STU0225-098	Balanced growth
13	STU0225-057	Creativity up
14	STU0225-086	Creativity up

The status bar at the bottom indicates: 'Total rows: 124 Query complete 00:00:00.338' and 'Successfully run. Total query runtime: 338 msec. 124 rows affected.'

13) Award-winning artworks with judge count > average per category

```
SELECT a.title, aw.category, COUNT(ev.evaluationid) AS eval_count
FROM artwork a
JOIN award aw ON a.artworkid = aw.artworkid
LEFT JOIN submission sub ON a.artworkid = sub.artworkid
LEFT JOIN evaluation ev ON sub.submissionid = ev.submissionid
GROUP BY a.title, aw.category
HAVING COUNT(ev.evaluationid) > (SELECT AVG(eval_count)
FROM (SELECT COUNT(ev2.evaluationid) AS eval_count
FROM evaluation ev2
GROUP BY ev2.submissionid) AS avg_eval)
ORDER BY eval_count DESC;
```

The screenshot shows a PostgreSQL IDE interface. The query editor contains the SQL query for finding award-winning artworks. The query is executed, and the results are displayed in a table with 14 rows. The table has three columns: title, category, and eval_count. The status bar at the bottom indicates that the query was successfully run, affecting 127 rows in 140 milliseconds.

id	title	category	eval_count
1	Work 165	Student	14
2	Work 189	Illustration	10
3	Work 124	Photography	8
4	Work 14	Sculpture	8
5	Work 165	Digital	7
6	Work 163	Overall	7
7	Work 163	Sculpture	7
8	Work 107	Sculpture	6
9	Work 1	Sculpture	6
10	Work 160	Photography	6
11	Work 156	Digital	5
12	Work 43	Sculpture	5
13	Work 43	Overall	5
14	Work 193	Overall	5

14) Judges who scored >8 in multiple exhibitions

```
SELECT j.expertise, COUNT(DISTINCT ev.exhibitionid) AS exhib_count,
AVG(ev.score) AS avg_score
FROM judge j
JOIN evaluation ev ON j.userid = ev.judgeid
WHERE ev.score > 8
GROUP BY j.expertise
HAVING COUNT(DISTINCT ev.exhibitionid) > 1
ORDER BY avg_score DESC;
```

The screenshot shows the PostgreSQL Client Explorer interface. The left pane displays the database structure, including the 'temp' database. The main pane shows a SQL query and its results.

Query:

```

1 SELECT j.expertise, COUNT(DISTINCT ev.exhibitionid) AS exhib_count, AVG(ev.score) AS avg_score
2 FROM judge j
3 JOIN evaluation ev ON j.userid = ev.judgeid
4 WHERE ev.score > 8
5 GROUP BY j.expertise
6 HAVING COUNT(DISTINCT ev.exhibitionid) > 1
7 ORDER BY avg_score DESC;

```

Data Output:

expertise	exhib_count	avg_score
Sketching	14	72.61739393939393
Duration	32	70.60476190476192
Graphic Design	51	69.68133333333333
Textile Art	38	69.69014038742377
Painting	29	66.77920000000000
Digital Art	29	66.53023074923074
Ceramics	12	62.27415284112846

Successfully run. Total query runtime: 104 msec. 7 rows affected.

15) APPA recommendations for students with low confidence (<0.7) and low average score

```

SELECT s.studentrollno, a.recommendation
FROM student s
JOIN appa a ON s.userid = a.studentid
WHERE a.confidence < 0.7 AND s.userid IN (SELECT s2.userid
FROM student s2
JOIN artwork ar ON s2.userid = ar.studentid
JOIN submission sub ON ar.artworkid = sub.artworkid
JOIN evaluation ev ON sub.submissionid = ev.submissionid
GROUP BY s2.userid
HAVING AVG(ev.score) < (SELECT AVG(score) FROM evaluation))
ORDER BY s.studentrollno;

```

The screenshot shows the PostgreSQL IDE interface. The left pane displays the 'Object Explorer' with a tree view of the database structure, including 'Servers (1)', 'PostgreSQL 17', 'Databases (4)', 'ArtExhibition', 'postgres', 'temp', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'Login/Group Roles', and 'Tablespaces'. The main pane shows a SQL query in the 'Query' tab:

```

1 SELECT s.studentrollno, a.recommendation
2 FROM student s
3 JOIN saps a ON s.userid = a.userid
4 WHERE a.confidence < 0.7 AND s.userid IN (SELECT s2.userid
5 FROM student s2
6 JOIN artwork ar ON s2.userid = ar.studentid
7 JOIN submission sub ON ar.artworkid = sub.artworkid
8 JOIN evaluation ev ON sub.submissionid = ev.submissionid
9 GROUP BY s2.userid
10 HAVING AVG(ev.score) < (SELECT AVG(score) FROM evaluation))
11 ORDER BY s.studentrollno;

```

The 'Data Output' pane at the bottom shows the results of the query, displaying columns 'studentrollno' and 'recommendation'. The results are as follows:

studentrollno	recommendation
STU0205-091	Practice skills
STU0205-091	Practice skills
STU0205-092	Practice skills
STU0205-093	Practice skills
STU0205-093	Practice skills
STU0205-093	Practice skills
STU0205-095	Practice skills
STU0205-096	Practice skills
STU0205-096	Practice skills
STU0205-096	Practice skills
STU0205-096	Practice skills
STU0205-096	Practice skills

The status bar at the bottom indicates 'Total rows: 92' and 'Query complete 00:00:00.103'.

16) Portfolio artworks with submission status 'Accepted' and award

```

SELECT p.bio, a.title, sub.status, aw.awardname
FROM portfolio p
JOIN portfolioartwork pa ON p.portfolioid = pa.portfolioid
JOIN artwork a ON pa.artworkid = a.artworkid
JOIN submission sub ON a.artworkid = sub.artworkid
LEFT JOIN award aw ON a.artworkid = aw.artworkid
WHERE sub.status = 'Accepted';

```

The screenshot shows the PostgreSQL IDE interface. The left pane displays the 'Object Explorer' with a tree view of the database structure, including 'Servers (1)', 'PostgreSQL 17', 'Databases (4)', 'ArtExhibition', 'postgres', 'temp', 'Catalogs', 'Event Triggers', 'Extensions', 'Foreign Data Wrappers', 'Languages', 'Publications', 'Schemas', 'Subscriptions', 'Login/Group Roles', and 'Tablespaces'. The main pane shows a SQL query in the 'Query' tab:

```

1 SELECT p.bio, a.title, sub.status, aw.awardname
2 FROM portfolio p
3 JOIN portfolioartwork pa ON p.portfolioid = pa.portfolioid
4 JOIN artwork a ON pa.artworkid = a.artworkid
5 JOIN submission sub ON a.artworkid = sub.artworkid
6 LEFT JOIN award aw ON a.artworkid = aw.artworkid
7 WHERE sub.status = 'Accepted';

```

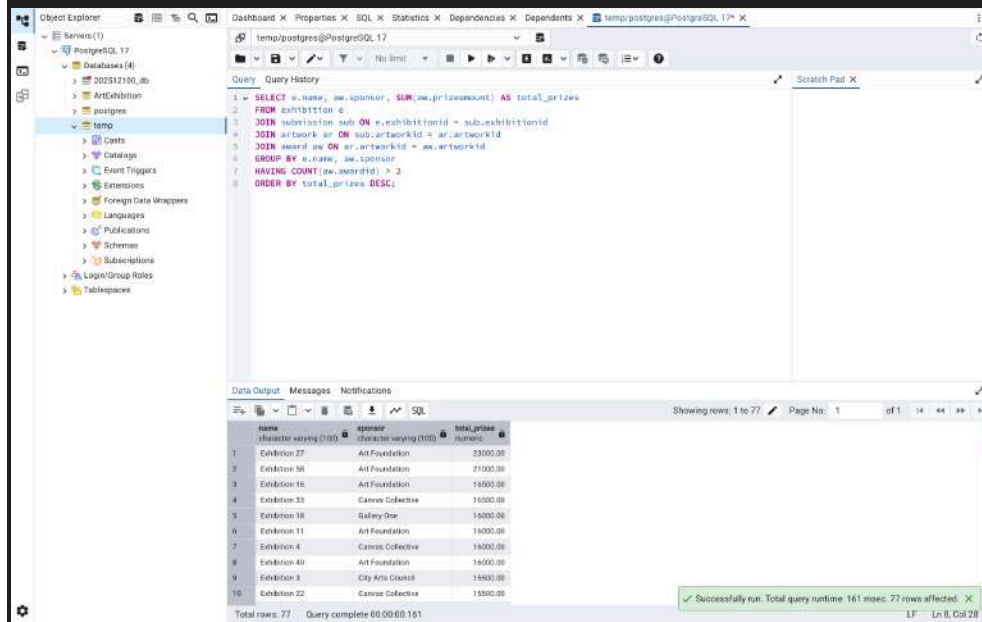
The 'Data Output' pane at the bottom shows the results of the query, displaying columns 'bio', 'title', 'status', and 'awardname'. The results are as follows:

bio	title	status	awardname
Portfolio of student 82 focusing on Digital Art	Work 163	Accepted	Best in Medium
Portfolio of student 82 focusing on Digital Art	Work 163	Accepted	Best in Medium
Portfolio of student 82 focusing on Digital Art	Work 163	Accepted	Best in Medium
Portfolio of student 82 focusing on Digital Art	Work 163	Accepted	Best in Medium
Portfolio of student 82 focusing on Digital Art	Work 163	Accepted	Best in Medium
Portfolio of student 89 focusing on Digital Art	Work 177	Accepted	Curator's Pick
Portfolio of student 89 focusing on Digital Art	Work 177	Accepted	Curator's Pick
Portfolio of student 34 focusing on Ceramics	Work 67	Accepted	People's Choice
Portfolio of student 36 focusing on Ceramics	Work 69	Accepted	Curator's Pick
Portfolio of student 62 focusing on Printmaking	Work 124	Accepted	People's Choice

The status bar at the bottom indicates 'Total rows: 142' and 'Query complete 00:00:00.110'. A green message box at the bottom right states 'Successfully run. Total query runtime: 110 msec. 142 rows affected.'

17) Total prize money per exhibition with sponsor

```
SELECT e.name, aw.sponsor, SUM(aw.prizeamount) AS total_prizes
FROM exhibition e
JOIN submission sub ON e.exhibitionid = sub.exhibitionid
JOIN artwork ar ON sub.artworkid = ar.artworkid
JOIN award aw ON ar.artworkid = aw.artworkid
GROUP BY e.name, aw.sponsor
HAVING COUNT(aw.awardid) > 2
ORDER BY total_prizes DESC;
```



	name	sponsor	total_prizes
1	Exhibition 17	Art Foundation	23000.00
2	Exhibition 58	Art Foundation	21000.00
3	Exhibition 16	Art Foundation	16500.00
4	Exhibition 33	Canine Collective	16500.00
5	Exhibition 18	Galaxy One	16000.00
6	Exhibition 11	Art Foundation	16000.00
7	Exhibition 4	Canine Collective	16000.00
8	Exhibition 49	Art Foundation	16000.00
9	Exhibition 3	City Arts Council	16000.00
10	Exhibition 22	Canine Collective	15500.00

Total rows: 77 Query complete 00:00:00.161

Successfully run. Total query runtime: 161 msec. 77 rows affected.

18) Artworks with award and APPA confidence percentile rank

```
WITH ranked AS (
    SELECT
        a.title,
        aw.awardname,
        appa.confidence,
        PERCENT_RANK() OVER (ORDER BY appa.confidence DESC) AS confidence_rank
    FROM artwork a
    JOIN award aw ON a.artworkid = aw.artworkid
    JOIN appa ON a.studentid = appa.studentid
)
SELECT *
FROM ranked
WHERE confidence_rank <= 0.2
ORDER BY confidence_rank;
```

The screenshot shows a PostgreSQL IDE interface. The top pane displays a SQL query using a Common Table Expression (CTE) to rank artworks by confidence. The bottom pane shows the results of the query, which includes columns for title, awardname, confidence, and confidence_rank.

Query:

```

1 WITH ranked AS (
2     SELECT
3         a.title,
4         aw.awardname,
5         appa.confidence,
6         PERCENT_RANK() OVER (ORDER BY appa.confidence DESC) AS confidence_rank
7     FROM artwork a
8     JOIN award aw ON a.artworkid = aw.artworkid
9     JOIN appa ON a.studentid = appa.studentid
10 )
11 SELECT *
12 FROM ranked
13 WHERE confidence_rank <= 0.2
14 ORDER BY confidence_rank;

```

Data Output:

	title	awardname	confidence	confidence_rank
1	Work 144	Curator's Pick	1	0
2	Work 190	Best in Show	1	0
3	Work 144	Best in Medium	1	0
4	Work 190	Best in Medium	1	0
5	Work 119	People's Choice	0	0.010069565217391304
6	Work 119	People's Choice	0	0.010069565217391304
7	Work 42	Best in Show	0	0.010069565217391304
8	Work 42	Best in Show	0	0.010069565217391304
9	Work 177	Curator's Pick	0	0.010069565217391304
10	Work 177	Curator's Pick	0	0.010069565217391304
11	Work 67	People's Pick	0	0.010069565217391304
12	Work 166	Curator's Pick	0	0.010069565217391304
13	Work 166	Curator's Pick	0	0.010069565217391304
14	Work 58	Curator's Pick	0	0.010069565217391304

Successfully run. Total query runtime: 73 msec. 369 rows affected.

19) Judges who evaluated artworks in multiple categories with average score per category

```

SELECT j.expertise, aw.category, AVG(ev.score) AS avg_score
FROM judge j
JOIN evaluation ev ON j.userid = ev.judgeid
JOIN artwork a ON ev.artworkid = a.artworkid
JOIN award aw ON a.artworkid = aw.artworkid
GROUP BY j.expertise, aw.category
HAVING COUNT(ev.evaluationid) > 1
ORDER BY avg_score DESC;

```

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' showing a database structure. The main window displays a SQL query in the 'Query' tab. Below the query is the 'Data Output' tab, which shows the results of the query in a table format. The table has three columns: 'expertise', 'category', and 'avg_score'. The results are sorted by 'avg_score' in descending order. A status bar at the bottom indicates 'Total rows: 40' and 'Query complete 00:00:00.143'.

```

SELECT j.expertise, aw.category, AVG(ev.score) AS avg_score
FROM judge j
JOIN evaluation ev ON j.userid = ev.judgeid
JOIN artwork a ON ev.artworkid = a.artworkid
JOIN award aw ON a.artworkid = aw.artworkid
GROUP BY j.expertise, aw.category
HAVING COUNT(ev.evaluationid) > 1
ORDER BY avg_score DESC

```

expertise	category	avg_score
Sculpture	Popular	85.29333333333333
Graphic Design	Popular	79.31222222222222
Graphic Design	Overall	77.95318181818182
Curator	Photography	73.10263636363636
Textile Art	Photography	74.21142857142857
Painting	Photography	74.00200000000000
Curator	Overall	73.39000000000000
Textile Art	Overall	73.39000000000000
Textile Art	Student	73.31333333333333
Sculpture	Sculpture	71.95166666666667

Successfully run. Total query runtime: 143 msec, 40 rows affected.

20) Judges' evaluations with subquery for above-speciality average score

```

SELECT
    ev.evaluationid,
    ev.score,
    j.expertise
FROM evaluation ev
JOIN judge j
    ON ev.judgeid = j.userid
JOIN (
    SELECT j2.expertise, AVG(ev2.score) AS avg_score
    FROM evaluation ev2
    JOIN judge j2 ON ev2.judgeid = j2.userid
    GROUP BY j2.expertise
) AS avg_by_exp
    ON avg_by_exp.expertise = j.expertise
WHERE ev.score > avg_by_exp.avg_score;

```

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' showing a database schema with tables like 'User', 'apps', 'award', 'evaluation', 'exhibition', 'judge', 'notification', 'organizer', 'portfolio', 'portfolioartwork', 'student', 'submissions', 'userdetails', 'venue', 'visitor', 'Trigger Functions', 'Types', and 'Views'. The main window displays a SQL query in the 'Query' tab, which is a complex join query involving 'evaluation', 'judge', and 'exhibition' tables, calculating average scores and expertise. Below the query, the 'Data Output' tab shows the results of the query, displaying columns 'evaluationid', 'score', and 'expertise'. The results are shown as a table with 151 rows. A status bar at the bottom indicates 'Total rows: 151' and 'Query complete 00:00:00.121'.

```

1 SELECT
2   ev.evaluationid,
3   ev.score,
4   j.expertise
5 FROM evaluation ev
6 JOIN judge j
7   ON ev.judgeid = j.userid
8 JOIN (
9   SELECT j2.expertise, AVG(ev2.score) AS avg_score
10  FROM evaluation ev2
11  JOIN judge j2 ON ev2.judgeid = j2.userid
12  GROUP BY j2.expertise
13 ) AS avg_by_exp
14   ON avg_by_exp.expertise = j.expertise
15 WHERE ev.score > avg_by_exp.avg_score;

```

evaluationid	score	expertise
1	81.31	Graphic Design
2	88.73	Painting
3	90.92	Graphic Design
4	84.94	Textile Art
5	88.81	Textile Art
6	91.74	Painting
7	87.13	Curation
8	71.52	Painting
9	82.53	Textile Art
10	66.13	Ceramics
11	79.68	Textile Art
12	82.68	Curation
13	70.04	Graphic Design
14	93.24	Painting

Showing rows: 1 to 151 | Page No: 1 | of 1 | 14 | 44 | 99 | 11

✓ Successfully run. Total query runtime: 121 msec. 151 rows affected. X

LF Ln 16, Col 1

21) Cumulative submissions by theme with year-over-year growth lag

```

WITH monthly AS (
    SELECT
        e.theme,
        DATE_TRUNC('month', sub.submissiondate) AS month,
        COUNT(*) AS monthly_sub
    FROM exhibition e
    JOIN submission sub
        ON e.exhibitionid = sub.exhibitionid
    GROUP BY e.theme, DATE_TRUNC('month', sub.submissiondate)
)
SELECT
    theme,
    month,
    monthly_sub,
    SUM(monthly_sub) OVER (
        PARTITION BY theme
        ORDER BY month
    ) AS cumulative,
    LAG(monthly_sub) OVER (
        PARTITION BY theme
        ORDER BY month
    ) AS prev_month_growth
FROM monthly

```

ORDER BY theme, month;

```

6 FROM exhibition e
7 JOIN submission sub
8 ON e.exhibitionid = sub.exhibitionid
9 GROUP BY e.theme, DATE_TRUNC('month', sub.submissiondate)
10 )
11 SELECT
12 theme,
13 month,
14 monthly_sub,
15 SUM(monthly_sub) OVER (
16 PARTITION BY theme
17 ORDER BY month
18 ) AS cumulative,
19 LAG(monthly_sub) OVER (
20 PARTITION BY theme
21 ORDER BY month
22 ) AS prev_month_growth
23 FROM monthly
24 ORDER BY theme, month;

```

theme	month	monthly_sub	cumulative	prev_month_growth
Contemporary	2025-03-01 00:00:00+05:30	5	5	[null]
Contemporary	2025-06-01 00:00:00+05:30	1	6	5
Contemporary	2025-06-01 00:00:00+05:30	4	10	1
Contemporary	2025-08-01 00:00:00+05:30	7	17	4
Contemporary	2025-09-01 00:00:00+05:30	3	20	7
Contemporary	2026-02-01 00:00:00+05:30	8	28	3
Contemporary	2026-03-01 00:00:00+05:30	3	31	8
Contemporary	2026-04-01 00:00:00+05:30	7	38	3
Contemporary	2026-06-01 00:00:00+05:30	6	44	7
Contemporary	2026-06-01 00:00:00+05:30	6	50	6
Contemporary	2026-07-01 00:00:00+05:30	2	52	6
Contemporary	2026-08-01 00:00:00+05:30	3	55	3
Digital Media	2025-01-01 00:00:00+05:30	2	2	[null]
Digital Media	2025-02-01 00:00:00+05:30	3	5	2

Successfully run. Total query runtime: 119 msec. 68 rows affected.

22) Total submissions by student speciality with average score

```

SELECT s.speciality, COUNT(sub.submissionid) AS total_submissions,
AVG(ev.score) AS avg_score
FROM student s
JOIN artwork a ON s.userid = a.studentid
JOIN submission sub ON a.artworkid = sub.artworkid
LEFT JOIN evaluation ev ON sub.submissionid = ev.submissionid
GROUP BY s.speciality
ORDER BY total_submissions DESC;

```

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' pane showing a database schema with tables like 'student', 'submission', 'award', 'exhibition', 'judge', 'notification', 'organizer', 'portfolio', 'portfolioartwork', 'student', 'submission', 'userdetails', 'venue', 'visitor', 'Trigger Functions', 'Types', and 'Views'. The main window displays a SQL query in the 'Query' pane:

```
1 SELECT s.speciality, COUNT(sub.submissionid) AS total_submissions, AVG(ev.score) AS avg_score
2 FROM student s
3 JOIN artwork a ON s.userid = a.studentid
4 JOIN submission sub ON a.artworkid = sub.artworkid
5 LEFT JOIN evaluation ev ON sub.submissionid = ev.submissionid
6 GROUP BY s.speciality
7 ORDER BY total_submissions DESC;
```

The 'Data Output' pane shows the results of the query, displaying a table with 9 rows and 3 columns: 'speciality', 'totalSubmissions', and 'avgScore'. The data is as follows:

speciality	totalSubmissions	avgScore
Permaculture	49	87.0349938773310204
Ceramics	45	89.94864444444444
Photography	43	88.5741860465116279
Graphic Design	41	85.66658365836585
Illustration	36	86.36655555555556
Digital Art	27	88.93333333333333
Painting	25	81.64800000000000
Sculpture	20	76.55555555555556
Textile Art	14	74.71759714285714

The status bar at the bottom indicates 'Total rows: 9' and 'Query complete 00:00:00.127'.

23) Visitors who attended multiple awarded exhibitions

```
SELECT
    v.userid,
    COUNT(*) AS awarded_exhib_count
FROM visitor v
JOIN (
    SELECT DISTINCT
        sub.userid,
        sub.exhibitionid
    FROM submission sub
    JOIN award aw
        ON sub.artworkid = aw.artworkid
) AS awarded ON awarded.userid = v.userid
GROUP BY v.userid
HAVING COUNT(*) > 1
ORDER BY awarded_exhib_count DESC;
```

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' showing a database schema with tables like 'user', 'artwork', 'award', 'exhibition', etc. The main window displays a SQL query in the 'Query' tab. The query is a complex join and aggregation. Below the query editor, the 'Data Output' tab shows the results of the query, which is a table with 10 rows and 2 columns: 'userid' and 'awarded_exhib_count'. The status bar at the bottom indicates 'Total rows: 10' and 'Query complete 00:00:00.130'.

```

1 SELECT
2   v.userid,
3   COUNT(*) AS awarded_exhib_count
4 FROM visitor v
5 JOIN (
6   SELECT DISTINCT
7     sub.userid,
8     sub.exhibitionid
9   FROM submission sub
10  JOIN award aw
11    ON sub.artworkid = aw.artworkid
12 ) AS awarded ON awarded.userid = v.userid
13 GROUP BY v.userid
14 HAVING COUNT(*) > 1
15 ORDER BY awarded_exhib_count DESC;

```

userid	awarded_exhib_count
190	2
197	2
189	2
157	2
191	2
169	2
177	2
182	2
171	2
145	2

Successfully run. Total query runtime: 130 msec. 10 rows affected.

24) Exhibitions with sponsor contributions > average prize

```

SELECT e.name, SUM(aw.prizeamount) AS total_prize
FROM exhibition e
JOIN submission sub ON e.exhibitionid = sub.exhibitionid
JOIN artwork a ON sub.artworkid = a.artworkid
JOIN award aw ON a.artworkid = aw.artworkid
GROUP BY e.name
HAVING SUM(aw.prizeamount) > (SELECT AVG(prizeamount)
FROM award WHERE prizeamount IS NOT NULL);

```

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' pane showing a database schema with tables like 'exhibition', 'submission', 'artwork', and 'award'. The main window displays a SQL query in the 'Query' pane:

```

1 SELECT e.name, SUM(aw.prizeamount) AS total_prize
2 FROM exhibition e
3 JOIN submission sub ON e.exhibitionid = sub.exhibitionid
4 JOIN artwork a ON sub.artworkid = a.artworkid
5 JOIN award aw ON a.artworkid = aw.artworkid
6 GROUP BY e.name
7 HAVING SUM(aw.prizeamount) > (SELECT AVG(prizeamount)
8 FROM award WHERE prizeamount IS NOT NULL);

```

Below the query, the 'Data Output' pane shows the results of the query. The results are displayed in a table with two columns: 'name' and 'total_prize'.

name	total_prize
Exhibition 9	14000.00
Exhibition 22	18500.00
Exhibition 55	36000.00
Exhibition 44	21500.00
Exhibition 55	24500.00
Exhibition 16	42000.00
Exhibition 30	43000.00
Exhibition 19	24000.00
Exhibition 34	38000.00
Exhibition 14	33500.00

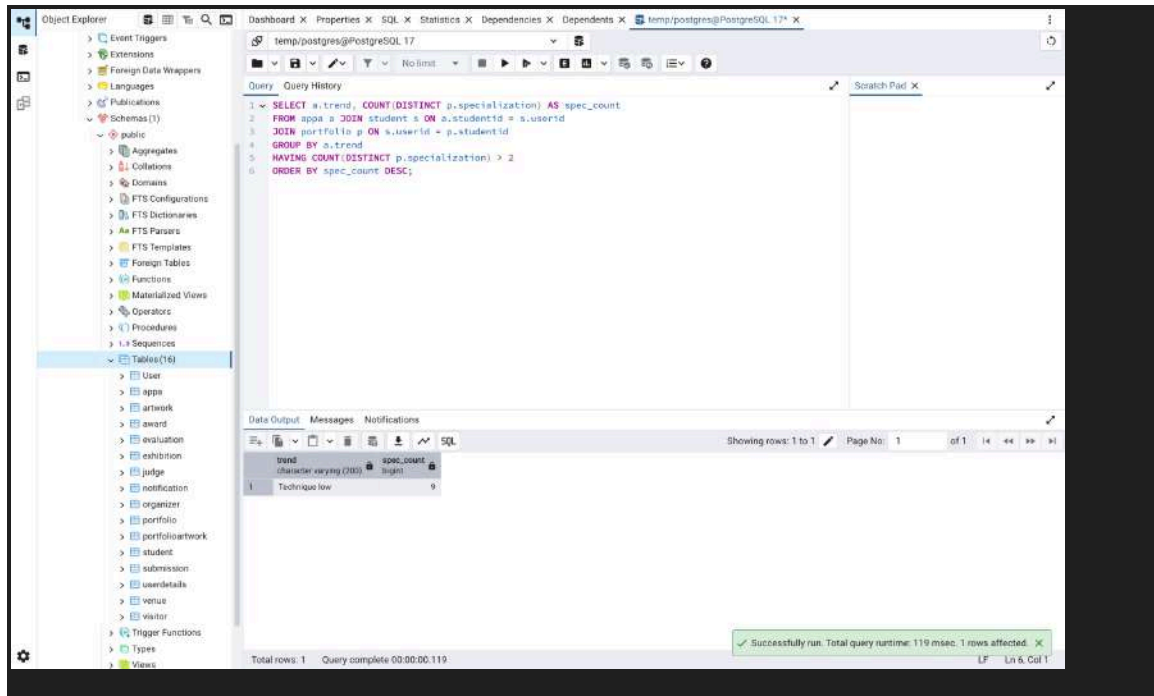
At the bottom of the IDE, a status bar indicates 'Total rows: 60' and 'Query complete 00:00:00.214'. A green message box at the bottom right states: 'Successfully run. Total query runtime: 214 msec. 60 rows affected.'

25) APPA trends for students with portfolio in >2 specializations

```

SELECT e.name, SUM(aw.prizeamount) AS total_prize
FROM exhibition e
JOIN submission sub ON e.exhibitionid = sub.exhibitionid
JOIN artwork a ON sub.artworkid = a.artworkid
JOIN award aw ON a.artworkid = aw.artworkid
GROUP BY e.name
HAVING SUM(aw.prizeamount) > (SELECT AVG(prizeamount)
FROM award WHERE prizeamount IS NOT NULL);

```



26) Judges' average score by artwork medium with window rank

```
WITH ranked AS (
    SELECT
        j.expertise,
        a.medium,
        AVG(ev.score) AS avg_score,
        RANK() OVER (
            PARTITION BY a.medium
            ORDER BY AVG(ev.score) DESC
        ) AS rank
    FROM judge j
    JOIN evaluation ev ON j.userid = ev.judgeid
    JOIN artwork a ON ev.artworkid = a.artworkid
    GROUP BY j.expertise, a.medium
)
SELECT *
FROM ranked
WHERE rank <= 3
ORDER BY medium, rank;
```

The screenshot shows a PostgreSQL IDE interface. The left sidebar displays the 'Object Explorer' with a tree view of the database schema, including tables like 'user', 'exhibition', 'portfolio', 'submission', and 'artwork'. The main window shows a SQL query in the 'Query' tab, which uses the RANK() function to rank artworks by average score. The 'Data Output' window at the bottom displays the results of the query, showing columns for 'expertise', 'medium', 'avg_score', and 'rank'. The results are sorted by 'rank' in descending order, showing the top 10 artworks.

rank	expertise	medium	avg_score
1	Graphic Design	Acrylic	76.54000000000000
2	Curator	Acrylic	73.82000000000000
3	Digital Art	Acrylic	67.15200000000000
4	Graphic Design	Ceramic	76.07900000000000
5	Textile Art	Ceramic	71.39666666666667
6	Ceramics	Ceramic	70.17500000000000
7	Sculpture	Digital Print	73.79000000000000
8	Graphic Design	Digital Print	73.46545454545455
9	Curator	Digital Print	66.75600000000000
10	Textile Art	Ink	81.00000000000000

27) Students who took part in more than 2 exhibitions

```

SELECT
    ud.name AS student_name,
    ud.gender,
    COUNT(DISTINCT sub.exhibitionid) AS exhibition_count
FROM student s
JOIN userdetails ud
    ON ud.userid = s.userid
JOIN artwork a
    ON s.userid = a.studentid
JOIN submission sub
    ON a.artworkid = sub.artworkid
GROUP BY
    ud.name,
    ud.gender
HAVING
    COUNT(DISTINCT sub.exhibitionid) > 2
ORDER BY
    exhibition_count DESC;

```

The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left lists various database objects, including Tables (16). The main query window displays the following SQL query:

```

1 SELECT
2   ud.name AS student_name,
3   ud.gender,
4   COUNT(DISTINCT sub.exhibitionid) AS exhibition_count
5 FROM student s
6 JOIN userdetails ud
7   ON ud.userid = s.userid
8 JOIN artwork a
9   ON s.userid = a.studentid
10 JOIN submission sub
11   ON a.artworkid = sub.artworkid
12 GROUP BY
13   ud.name,
14   ud.gender
15 HAVING
16   COUNT(DISTINCT sub.exhibitionid) > 2
17 ORDER BY
18   exhibition_count DESC;

```

The Data Output pane shows the results of the query, displaying 52 rows. The columns are student_name, gender, and exhibition_count. The status bar at the bottom indicates "Total rows: 52 Query complete 00:00:00.106".

student_name	gender	exhibition_count
Ritu Sharma	Female	9
Rahul Jodhi	Other	9
Swati Raj	Other	7
Vikas Kumar	Other	6
Vikas Saxena	Other	6
Raj Verma	Male	6
Ishaan Agar	Female	6
Ishaan Mehta	Male	6
Megha Agar	Female	5
Isha Verma	Female	5
Nika Gandhi	Other	5
Rahul Kumar	Male	5
Megha Gandhi	Female	5
Megha Singh	Female	5

28) Cumulative prize money by year with running total

```

SELECT year, SUM(prizeamount) AS yearly_total,
SUM(SUM(prizeamount))
OVER (ORDER BY year) AS running_total
FROM award GROUP BY year ORDER BY year;

```

The screenshot shows the PostgreSQL IDE interface. The Object Explorer on the left lists various database objects, including Tables (16). The main query window displays the following SQL query:

```

1 SELECT year, SUM(prizeamount) AS yearly_total,
2 SUM(SUM(prizeamount))
3 OVER (ORDER BY year) AS running_total
4 FROM award GROUP BY year ORDER BY year;

```

The Data Output pane shows the results of the query, displaying 1 row. The columns are year, yearly_total, and running_total. The status bar at the bottom indicates "Total rows: 1 Query complete 00:00:00.200".

year	yearly_total	running_total
2023	628000.00	628000.00

29) Students data who has more than 3 artworks.

```

SELECT
    ud.name AS student_name,
    a.title AS artwork_name,
    e.name AS exhibition_name,
    v.name AS venue_name,
    AVG(ev.score) AS avg_score
FROM userdetails ud
JOIN student s
    ON ud.userid = s.userid
JOIN artwork a
    ON s.userid = a.studentid
JOIN submission sub
    ON a.artworkid = sub.artworkid
JOIN exhibition e
    ON sub.exhibitionid = e.exhibitionid
JOIN venue v
    ON e.venueid = v.venueid
LEFT JOIN evaluation ev
    ON sub.submissionid = ev.submissionid
WHERE
    sub.status = 'Accepted'
GROUP BY
    ud.name, a.title, e.name, v.name
ORDER BY
    avg_score DESC NULLS LAST;

```

Query History

```

8 JOIN student s
9   ON ud.userid = s.userid
10 JOIN artwork a
11   ON s.userid = a.studentid
12 JOIN submission sub
13   ON a.artworkid = sub.artworkid
14 JOIN exhibition e
15   ON sub.exhibitionid = e.exhibitionid
16 JOIN venue v
17   ON e.venueid = v.venueid
18 LEFT JOIN evaluation ev
19   ON sub.submissionid = ev.submissionid
20 WHERE
21   sub.status = 'Accepted'
22 GROUP BY
23   ud.name, a.title, e.name, v.name
24 ORDER BY
25   avg_score DESC NULLS LAST;

```

Data Output

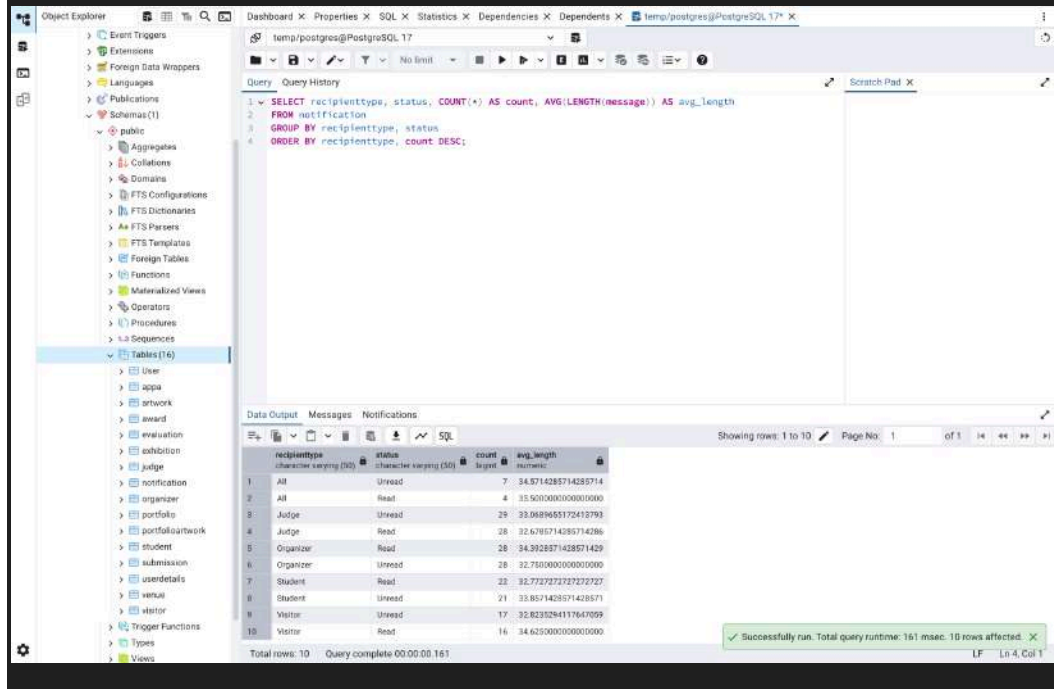
student_name	artwork_name	exhibition_name	venue_name	avg_score
character varying (100)	character varying (100)	character varying (100)	character varying (100)	numeric
Diya Gupta	Work 76	Exhibition 55	Venue 15 Arts Center	97.5800000000000000
Vikas Kumar	Work 143	Exhibition 9	Venue 9 Arts Center	97.5800000000000000
Indrani Agar	Work 43	Exhibition 16	Venue 15 Arts Center	93.9800000000000000
Tania Pawar	Work 67	Exhibition 2	Venue 2 Arts Center	91.7400000000000000
Ansh Agarwal	Work 71	Exhibition 6	Venue 5 Arts Center	91.4600000000000000
Megha Gopal	Work 165	Exhibition 23	Venue 5 Arts Center	90.3900000000000000
Riya Agarwal	Work 168	Exhibition 28	Venue 9 Arts Center	90.3100000000000000
Swati Raj	Work 194	Exhibition 45	Venue 5 Arts Center	89.7000000000000000
Swati Raj	Work 193	Exhibition 17	Venue 17 Arts Center	89.3300000000000000
Vikram Sasana	Work 19	Exhibition 53	Venue 12 Arts Center	87.5300000000000000
Misra Gopal	Work 108	Exhibition 26	Venue 6 Arts Center	87.2200000000000000
Neeta Verma	Work 192	Exhibition 2	Venue 2 Arts Center	87.1200000000000000
Ravya Kohli	Work 27	Exhibition 43	Venue 3 Arts Center	86.7200000000000000
Isha Khurana	Work 136	Exhibition 15	Venue 15 Arts Center	86.4500000000000000

Total rows: 77 Query complete 00:00:00.978

Successfully run. Total query runtime: 78 msec. 77 rows affected.

30) Notification trends by recipienttype with average message length

```
SELECT recipienttype, status, COUNT(*) AS count, AVG(LENGTH(message)) AS
avg_length
FROM notification
GROUP BY recipienttype, status
ORDER BY recipienttype, count DESC;
```



The screenshot shows a PostgreSQL IDE interface. The left pane displays the 'Object Explorer' with a tree view of the database schema, including 'Tables (16)' and 'Sequences'. The main pane shows a query editor with the following SQL query:

```
1 SELECT recipienttype, status, COUNT(*) AS count, AVG(LENGTH(message)) AS avg_length
2 FROM notification
3 GROUP BY recipienttype, status
4 ORDER BY recipienttype, count DESC;
```

Below the query editor, the 'Data Output' pane displays the results of the query. The results are shown in a table with columns: recipienttype, status, count, and avg_length. The table contains 10 rows of data, sorted by recipienttype and then by count in descending order.

recipienttype	status	count	avg_length
All	Unread	7	34.9714285714285714
All	Read	4	35.5000000000000000
Judge	Unread	29	33.0889655172412793
Judge	Read	28	32.6785714285714286
Organizer	Read	28	34.3928571428571429
Organizer	Unread	28	32.7500000000000000
Student	Read	22	32.7727272727272727
Student	Unread	21	33.8571428571428571
Visitor	Unread	17	32.8333284117647059
Visitor	Read	16	34.6250000000000000

The status of each row is indicated by a small icon: a green circle for 'Unread' and a red circle for 'Read'. The table also shows the total number of rows (10) and the query completion time (00:00:00.161).

31) Artworks with award and APPA confidence > 0.8

```
SELECT a.title, aw.awardname, appa.confidence, s.speciality
FROM artwork a
JOIN award aw ON a.artworkid = aw.artworkid
JOIN appa ON a.studentid = appa.studentid
JOIN student s ON a.studentid = s.userid
WHERE appa.confidence > 0.8;
```

The screenshot shows a PostgreSQL SQL client interface. The left sidebar displays the database structure, including Schemas, Tables, and Views. The main window shows a query being executed:

```
SELECT a.title, aw.awardname, appa.confidence, s.speciality
FROM artwork a
JOIN award aw ON a.artworkid = aw.artworkid
JOIN appa ON a.studentid = appa.studentid
JOIN student s ON a.studentid = s.userid
WHERE appa.confidence > 8.8;
```

The query results are displayed in a table with the following columns: title, awardname, confidence, and speciality. The results show 7 rows of data, including titles like 'Work 01', 'Work 151', 'Work 01', 'Work 14', 'Work 14', 'Work 14', and 'Work 172'.

title	awardname	confidence	speciality
Work 01	Best in Show	1	Graphic Design
Work 151	Best in Medium	1	Printmaking
Work 01	People's Choice	1	Graphic Design
Work 14	Curator's Pick	1	Ceramics
Work 14	Curator's Pick	1	Ceramics
Work 14	Curator's Pick	1	Ceramics
Work 172	Curator's Pick	1	Ceramics

The status bar at the bottom indicates: Total rows: 7, Query complete 00:00:00.125, Successfully run. Total query runtime: 125 msec. 7 rows affected.

32) Exhibitions with visitor count > average, ranked by total awards

```
SELECT e.name, COUNT(DISTINCT v.userid) AS visitor_count, COUNT(aw.awardid) AS
award_count,
RANK() OVER (ORDER BY COUNT(aw.awardid) DESC) AS award_rank
FROM exhibition e
JOIN submission sub ON e.exhibitionid = sub.exhibitionid
LEFT JOIN visitor v ON sub.userid = v.userid
LEFT JOIN artwork a ON sub.artworkid = a.artworkid
LEFT JOIN award aw ON a.artworkid = aw.artworkid
GROUP BY e.name
HAVING COUNT(DISTINCT v.userid) > (SELECT AVG(visitor_count)
FROM (SELECT COUNT(DISTINCT v2.userid) AS visitor_count
FROM visitor v2 GROUP BY v2.userid) AS avg_vis)
ORDER BY award_count DESC;
```

The screenshot shows the PostgreSQL IDE interface. The left pane displays the Object Explorer with the 'public' schema selected. The main pane shows a query window with the following SQL code:

```

1 SELECT e.name, COUNT(DISTINCT v.userid) AS visitor_count, COUNT(a.awardid) AS award_count,
2 RANK() OVER (ORDER BY COUNT(a.awardid) DESC) AS award_rank
3 FROM exhibition e
4 JOIN submission sub ON e.exhibitionid = sub.exhibitionid
5 LEFT JOIN visitor v ON sub.userid = v.userid
6 LEFT JOIN artwork a ON sub.artworkid = a.artworkid
7 LEFT JOIN award aw ON a.artworkid = aw.artworkid
8 GROUP BY e.name
9 HAVING COUNT(DISTINCT v.userid) > (SELECT AVG(visitor_count)
10 FROM (SELECT COUNT(DISTINCT v.userid) AS visitor_count
11 FROM visitor v2 GROUP BY v2.userid) AS avg_vis)
12 ORDER BY award_count DESC;

```

The Data Output pane shows the results of the query, displaying 11 rows. The columns are: name, visitor_count, award_count, and award_rank. The rows are sorted by award_count in descending order.

name	visitor_count	award_count	award_rank
Exhibition 58	2	13	1
Exhibition 48	2	12	2
Exhibition 58	3	11	3
Exhibition 42	2	9	4
Exhibition 45	2	8	5
Exhibition 25	2	8	5
Exhibition 53	2	8	5
Exhibition 48	2	7	8
Exhibition 21	3	7	8
Exhibition 42	2	6	10

At the bottom, a status bar indicates: Total rows: 11, Query complete 00:00:00.108. A green message box says: Successfully run. Total query runtime: 108 msec. 11 rows affected.

33) Judges' evaluations with subquery for above-average score in their speciality

```

SELECT ev.evaluationid, ev.score, j.expertise
FROM evaluation ev
JOIN judge j ON ev.judgeid = j.userid
WHERE ev.score > (SELECT AVG(ev2.score)
FROM evaluation ev2
JOIN judge j2 ON ev2.judgeid = j2.userid
WHERE j2.expertise = j.expertise);

```

The screenshot shows the PostgreSQL IDE interface. The left pane displays the Object Explorer with the 'public' schema selected. The main pane shows a query window with the following SQL code:

```

1 SELECT ev.evaluationid, ev.score, j.expertise
2 FROM evaluation ev
3 JOIN judge j ON ev.judgeid = j.userid
4 WHERE ev.score > (SELECT AVG(ev2.score)
5 FROM evaluation ev2
6 JOIN judge j2 ON ev2.judgeid = j2.userid
7 WHERE j2.expertise = j.expertise);

```

The Data Output pane shows the results of the query, displaying 10 rows. The columns are: evaluationid, score, and expertise. The rows are sorted by score in descending order.

evaluationid	score	expertise
1	81.31	Graphic Design
2	80.73	Painting
3	90.92	Graphic Design
4	84.94	Textile Art
5	80.81	Textile Art
6	91.74	Painting
7	80.13	Curating
8	71.52	Painting
9	82.53	Textile Art
10	69.13	Ceramics

At the bottom, a status bar indicates: Total rows: 10, Query complete 00:00:00.189. The bottom right corner shows: LF Ln 7, Col 35.

34) Cumulative submissions by month with running total

```
SELECT DATE_TRUNC('month', submissiondate) AS month, COUNT(*) AS monthly_count,
SUM(COUNT(*)) OVER (ORDER BY DATE_TRUNC('month', submissiondate)) AS
running_total
FROM submission GROUP BY DATE_TRUNC('month', submissiondate) ORDER BY month;
```

The screenshot shows a PostgreSQL IDE interface. On the left is the 'Object Explorer' showing a database schema with various objects like Event Triggers, Extensions, Languages, Publications, Schemas, Aggregates, Collations, Domains, FTS Configurations, FTS Dictionaries, FTS Parameters, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, Tables (16), User, appa, artwork, award, evaluation, exhibition, judge, notification, organizer, portfolio, portfolioartwork, student, submission, userdetails, venue, visitor, Trigger Functions, Types, and Views. The 'Tables (16)' folder is expanded, showing 'submission'. The main window displays a SQL query in the 'Query' tab, which is the same query as shown in the code block above. The 'Data Output' tab shows the results of the query, which are 21 rows. The results are displayed in a table with three columns: 'month', 'monthly_count', and 'running_total'. The 'month' column is of type 'timestamp with time zone', 'monthly_count' is 'integer', and 'running_total' is 'numeric'. The data shows a cumulative count of submissions over time, starting from 2024-12-01 and ending on 2025-09-01. The running total increases from 5 to 143 over the 21 rows.

month	monthly_count	running_total
2024-12-01 00:00:00+05:30	5	5
2025-01-01 00:00:00+05:30	18	23
2025-02-01 00:00:00+05:30	13	36
2025-03-01 00:00:00+05:30	15	51
2025-04-01 00:00:00+05:30	13	64
2025-05-01 00:00:00+05:30	17	81
2025-06-01 00:00:00+05:30	16	97
2025-07-01 00:00:00+05:30	13	110
2025-08-01 00:00:00+05:30	18	128
2025-09-01 00:00:00+05:30	15	143

At the bottom of the results table, it says 'Total rows: 21' and 'Query complete 00:00:00.116'. A green status bar at the bottom right indicates 'Successfully run. Total query runtime: 110 msec. 21 rows affected.'

35) Organizer and the exhibition they organised

```
SELECT
    ud.name AS organizer_name,
    e.name AS exhibition_name,
    v.name AS venue_name
FROM userdetails ud
JOIN organizer o
    ON ud.userid = o.userid
JOIN exhibition e
    ON o.userid = e.organizedby
JOIN venue v
    ON e.venueid = v.venueid
ORDER BY
    ud.name;
```

The screenshot shows a PostgreSQL IDE interface. On the left is the Object Explorer with a tree view of database objects. The main window displays a SQL query in the Query Editor. Below the query is the Data Output pane showing the results of the query. The query is a SELECT statement that joins user details, exhibition, and venue tables. The results table has three columns: organizer_name, exhibition_name, and venue_name. The status bar at the bottom indicates the query was successfully run with a total runtime of 363 msec and 60 rows affected.

```

1 SELECT
2   ud.name AS organizer_name,
3   e.name AS exhibition_name,
4   v.name AS venue_name
5 FROM userdetails ud
6 JOIN organizer o
7   ON ud.userid = o.userid
8 JOIN exhibition e
9   ON o.userid = e.organizedby
10 JOIN venue v
11   ON e.venueid = v.venueid
12 ORDER BY
13   ud.name;
14

```

organizer_name	exhibition_name	venue_name
Anaya Patti	Exhibition 4	Venue 4 Arts Center
Anaya Patti	Exhibition 44	Venue 4 Arts Center
Anaya Patti	Exhibition 24	Venue 4 Arts Center
Anaya Sethi	Exhibition 32	Venue 12 Arts Center
Anaya Sethi	Exhibition 12	Venue 12 Arts Center
Anaya Sethi	Exhibition 52	Venue 12 Arts Center
Arun Nair	Exhibition 28	Venue 8 Arts Center
Arun Nair	Exhibition 8	Venue 8 Arts Center
Arun Nair	Exhibition 48	Venue 8 Arts Center
Dev Bose	Exhibition 16	Venue 16 Arts Center
Dev Bose	Exhibition 36	Venue 16 Arts Center
Dev Bose	Exhibition 56	Venue 16 Arts Center
Kajal Sawant	Exhibition 2	Venue 2 Arts Center
Kajal Sawant	Exhibition 22	Venue 2 Arts Center

Successfully run. Total query runtime: 363 msec. 60 rows affected.

36) Students with notifications unread > average per role

```

SELECT s.studentrollno, COUNT(n.notificationid) AS unread_count
FROM student s
JOIN notification n ON s.userid = n.userid
WHERE n.status = 'Unread'
GROUP BY s.studentrollno
HAVING COUNT(n.notificationid) > (SELECT AVG(unread_count)
FROM (SELECT COUNT(*) AS unread_count
FROM notification WHERE status = 'Unread'
GROUP BY userid) AS avg_unread)
ORDER BY unread_count DESC;

```

The screenshot shows the PostgreSQL IDE interface. The left pane displays the database schema with tables like 'notification' and 'studentinfo'. The main query window contains the following SQL:

```

1 SELECT s.studentidno, COUNT(n.notificationid) AS unread_count
2 FROM student s
3 JOIN notification n ON s.userid = n.userid
4 WHERE n.status = 'unread'
5 GROUP BY s.studentidno
6 HAVING COUNT(n.notificationid) > (SELECT AVG(unread_count)
7 FROM (SELECT COUNT(*) AS unread_count
8 FROM notification WHERE status = 'unread'
9 GROUP BY userid) AS avg_unread)
10 ORDER BY unread_count DESC;

```

The Data Output pane shows the results of the query:

studentidno	unread_count
STU0205-066	2
STU0205-065	2
STU0205-051	2

A status message at the bottom indicates: "Successfully run. Total query runtime: 211 msec. 3 rows affected."

37) Awards with sponsor and average score from evaluations

```

SELECT aw.sponsor, AVG(ev.score) AS avg_score
FROM award aw
LEFT JOIN artwork a ON aw.artworkid = a.artworkid
LEFT JOIN submission sub ON a.artworkid = sub.artworkid
LEFT JOIN evaluation ev ON sub.submissionid = ev.submissionid
GROUP BY aw.sponsor
HAVING COUNT(aw.awardid) > 1
ORDER BY avg_score DESC;

```

The screenshot shows the PostgreSQL IDE interface. The left pane displays the database schema. The main query window contains the following SQL:

```

1 SELECT aw.sponsor, AVG(ev.score) AS avg_score
2 FROM award aw
3 LEFT JOIN artwork a ON aw.artworkid = a.artworkid
4 LEFT JOIN submission sub ON a.artworkid = sub.artworkid
5 LEFT JOIN evaluation ev ON sub.submissionid = ev.submissionid
6 GROUP BY aw.sponsor
7 HAVING COUNT(aw.awardid) > 1
8 ORDER BY avg_score DESC;

```

The Data Output pane shows the results of the query:

sponsor	avg_score
Salafy One	70.76518185181818
Art Foundation	68.50737410419609
City Arts Council	66.17141866666667
Canvas Collective	64.66761579475084

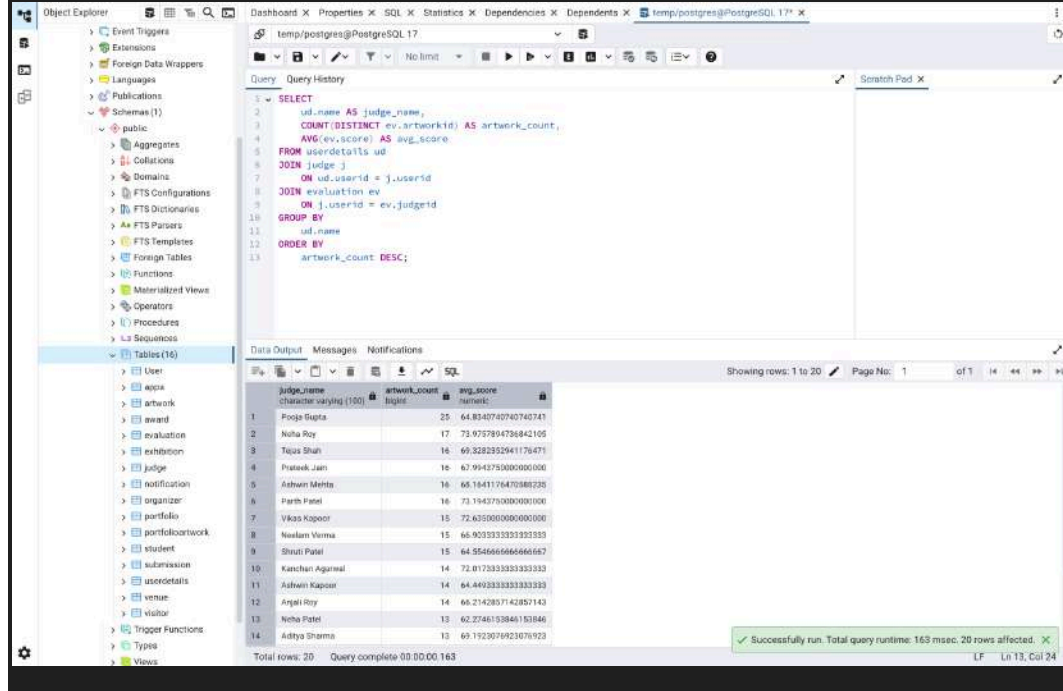
A status message at the bottom indicates: "Successfully run. Total query runtime: 105 msec. 4 rows affected."

38) Judge and no. of artworks they evaluated

```

SELECT
    ud.name AS judge_name,
    COUNT(DISTINCT ev.artworkid) AS artwork_count,
    AVG(ev.score) AS avg_score
FROM userdetails ud
JOIN judge j
    ON ud.userid = j.userid
JOIN evaluation ev
    ON j.userid = ev.judgeid
GROUP BY
    ud.name
ORDER BY
    artwork_count DESC;

```



The screenshot shows the PostgreSQL 17 IDE interface. The SQL query is entered in the main editor, and the results are displayed in the Data Output pane. The results table has three columns: judge_name, artwork_count, and avg_score. The data is sorted by artwork_count in descending order.

judge_name	artwork_count	avg_score
Pooja Gupta	25	64.83407407407407
Neha Roy	17	73.87578947368421
Tajus Shah	16	69.32252941176471
Prateek Jain	16	67.99437500000000
Ashwin Mehta	16	68.18411764705882
Parth Patel	16	73.19437500000000
Vikas Kapoor	15	72.63500000000000
Navleen Verma	15	66.90333333333333
Shruti Patel	15	64.55466666666667
Kanchan Agarwal	14	72.81733333333333
Ashwin Kapoor	14	64.44933333333333
Anjali Roy	14	66.21428571428571
Neha Patel	13	62.27461538461538
Aditya Sharma	13	69.19230769230769

Successfully run. Total query runtime: 163 msec. 20 rows affected.

39) Trigger – Auto-generate notification on award allocation

```

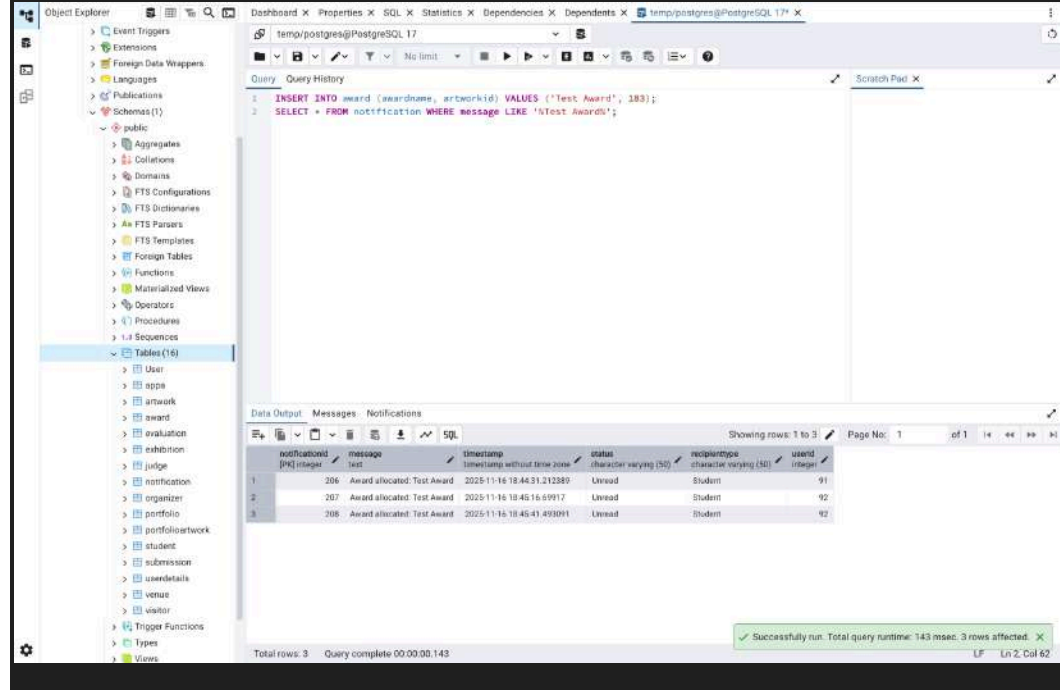
CREATE OR REPLACE FUNCTION notify_award()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO notification (message, userid, status, recipienttype)
    VALUES ('Award allocated: ' || NEW.awardname,
            (SELECT studentid FROM artwork WHERE artworkid = NEW.artworkid),
            'Unread', 'Student');
RETURN NEW;

```

```
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trig_notify_award
AFTER INSERT ON award FOR EACH ROW
EXECUTE FUNCTION notify_award();

INSERT INTO award (awardname, artworkid) VALUES ('Test Award', 81);
SELECT * FROM notification WHERE message LIKE '%Test Award%';
```



The screenshot shows a PostgreSQL IDE interface. The left sidebar displays the database schema, including tables like 'award', 'notification', 'exhibition', and 'submission'. The main window shows a query history with two queries: an INSERT into 'award' and a SELECT from 'notification'. The 'Data Output' tab is active, displaying the results of the SELECT query. The results table has columns: notificationid, message, timestamp, status, recipienttype, and awardid. It contains three rows of data.

notificationid	message	timestamp	status	recipienttype	awardid
206	Award allocated: Test Award	2025-11-16 18:44:31.212380	Unread	Student	91
207	Award allocated: Test Award	2025-11-16 18:45:16.69917	Unread	Student	92
208	Award allocated: Test Award	2025-11-16 18:45:41.493091	Unread	Student	92

At the bottom, a status bar indicates: 'Successfully run. Total query runtime: 143 msec. 3 rows affected.' and 'Total rows: 3 Query complete 00:00:00.143'.

40)Function – Compute APPA recommendation based on score delta

```
CREATE OR REPLACE FUNCTION get_appa_recommendation(student_i INTEGER, exhib_i
INTEGER)
RETURNS TEXT AS $$
DECLARE
    avg_score FLOAT;
    delta FLOAT;
BEGIN
    SELECT AVG(ev.score) INTO avg_score
    FROM evaluation ev
    JOIN submission sub ON ev.submissionid = sub.submissionid
    JOIN artwork a ON a.artworkid = sub.artworkid
    WHERE sub.exhibitionid = exhib_i
        AND a.studentid = student_i;

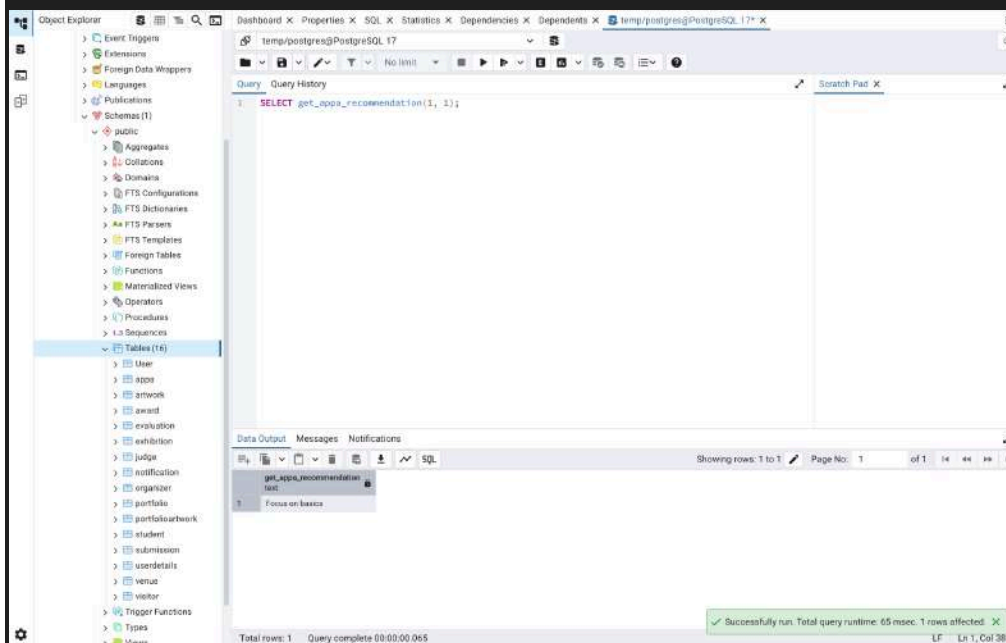
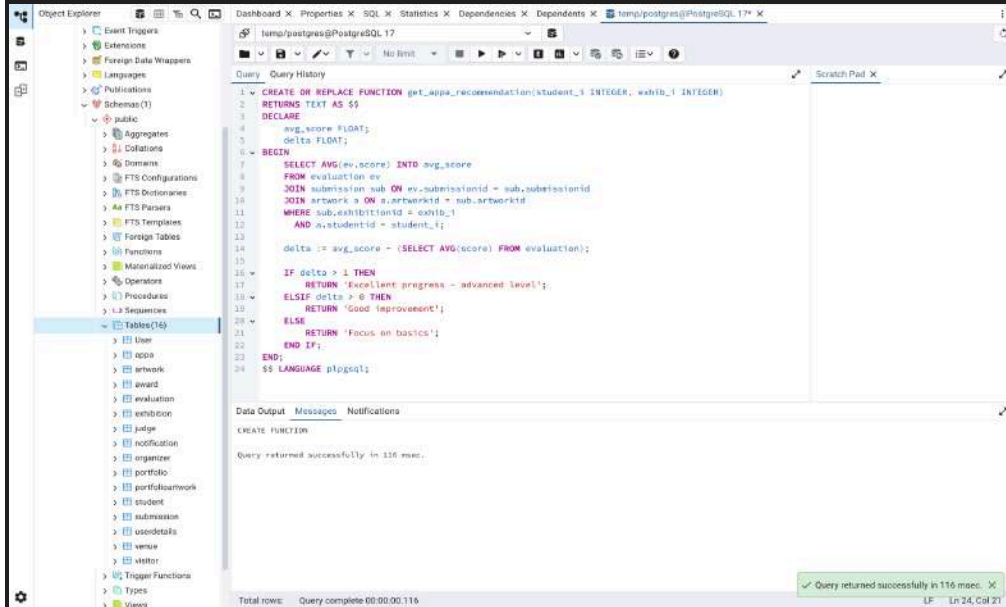
    delta := avg_score - (SELECT AVG(score) FROM evaluation);

    IF delta > 1 THEN
```

```

        RETURN 'Excellent progress - advanced level';
    ELSIF delta > 0 THEN
        RETURN 'Good improvement';
    ELSE
        RETURN 'Focus on basics';
    END IF;
END;
$$ LANGUAGE plpgsql;
SELECT get_appa_recommendation(1, 1);

```



15. Database Connectivity

15.1. Database Connectivity Code

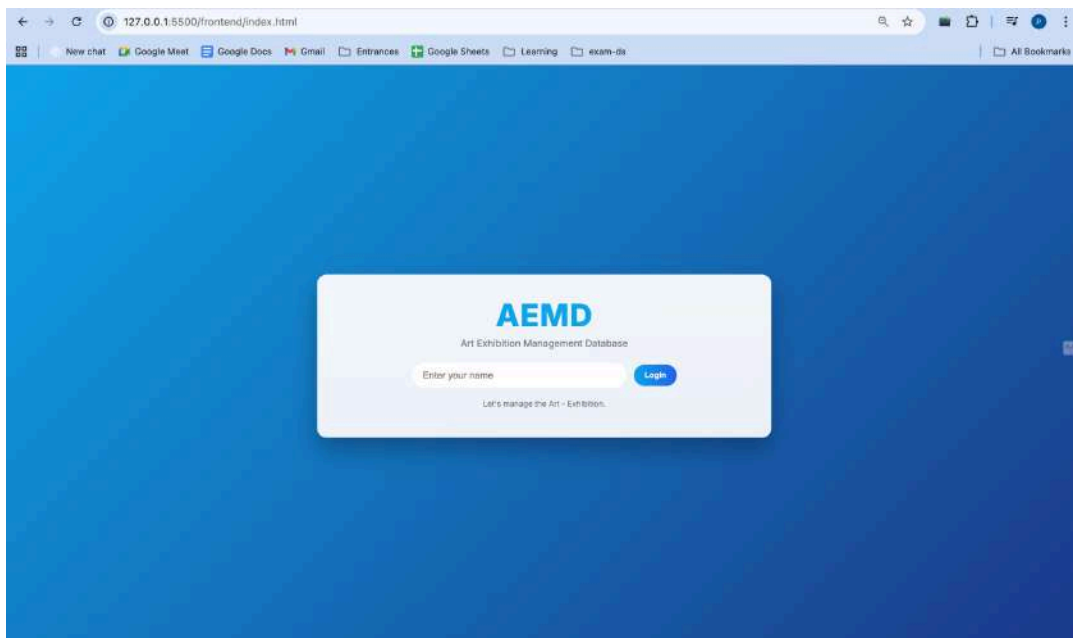
```
package com.aemd.backend;

import java.sql.Connection;
import java.sql.DriverManager;

public class DatabaseConnection {

    private static final String URL = "jdbc:postgresql://localhost:5432/temp";
    private static final String USER = "postgres";
    private static final String PASS = "1212";

    public static Connection getConnection() {
        try {
            Class.forName("org.postgresql.Driver");
            return DriverManager.getConnection(URL, USER, PASS);
        } catch (Exception e) {
            e.printStackTrace();
            return null; // ApiServer checks for null
        }
    }
}
```



15.2. CRUD Operation Code

15.2.1. CREATE

- **Description/Reasoning**
 - Implements INSERT for new visitor/artwork records via form submissions in script.js.
 - Uses RETURNING clause to return generated IDs for confirmation.
- **SQL Executed**

```
INSERT INTO visitor (tickettype, visitdate, feedback, feedbackratings)
VALUES (?, ?, ?, ?) RETURNING userid
```

- **Java Function (CRUDOperations.java)**

```
public static int insertVisitor(String tickettype, String visitdate,
String feedback, Integer feedbackratings) {
    int newId = -1;
    String sql = "INSERT INTO visitor (tickettype, visitdate,
feedback, feedbackratings) VALUES (?, ?, ?, ?) RETURNING userid";
    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, tickettype);
        if (visitdate != null && !visitdate.trim().isEmpty())
ps.setDate(2, Date.valueOf(visitdate)); else ps.setNull(2, Types.DATE);
        ps.setString(3, feedback);
        if (feedbackratings != null) ps.setInt(4, feedbackratings);
    else ps.setNull(4, Types.INTEGER);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) newId = rs.getInt("userid");
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return newId;
}
```

15.2.2. READ

- **Description/Reasoning**
 - Fetches top/bottom 10 rows from tables via buttons in index.html, using SELECT with ORDER BY and LIMIT.

- Sanitizes table names for security; results rendered in DataTables via script.js.
- **SQL Executed**

```
SELECT * FROM artwork ORDER BY 1 ASC LIMIT 10
```

- **Java Function (CRUDOperations.java)**

```
public static TableResult getTopAndBottom10(String tableName) {
    String safe = tableName.replaceAll("[^a-zA-Z0-9_]", "");
    String topJson = executeQuery("SELECT * FROM " + safe + " ORDER BY
1 ASC LIMIT 10");
    String bottomJson = executeQuery("SELECT * FROM " + safe + " ORDER
BY 1 DESC LIMIT 10");
    return new TableResult(topJson, bottomJson);
}

private static String executeQuery(String sql) {
    StringBuilder sb = new StringBuilder();
    sb.append("[");
    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql);
        ResultSet rs = ps.executeQuery()) {

        StringBuilder arr = new StringBuilder();
        arr.append("[");
        ResultSetMetaData meta = rs.getMetaData();
        int cols = meta.getColumnCount();
        boolean firstRow = true;

        while (rs.next()) {
            if (!firstRow) arr.append(",");
            firstRow = false;
            arr.append("{");
            for (int i = 1; i <= cols; i++) {
                String col = meta.getColumnName(i);
                String val = rs.getString(i);
                if (val == null) val = "";
                arr.append("\").append(col).append("\":");
                arr.append("\").append(val.replace("\",",
"\\\""));
                arr.append("\");
                if (i < cols) arr.append(",");
            }
        }
    }
}
```

```

        arr.append("}");
    }
    arr.append("]");
    sb = arr;
} catch (SQLException e) {
    e.printStackTrace();
}
return sb.toString();
}

```

15.2.3. UPDATE

- **Description/Reasoning**
 - Updates notification status (e.g., Unread to Read) via form in script.js, targeting by ID.
 - Returns boolean success; frontend alerts and reloads table if needed.
- **SQL Executed**

```
UPDATE notification SET status = ? WHERE notificationid = ?
```

- **Java Function (CRUDOperations.java)**

```

public static boolean updateNotificationStatus(int notificationid, String
status) {
    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement ps = conn.prepareStatement("UPDATE
notification SET status = ? WHERE notificationid = ?")) {
        ps.setString(1, status);
        ps.setInt(2, notificationid);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

```

15.2.4. DELETE

- **Description/Reasoning**
 - Deletes notification by ID via form submission in script.js, using WHERE for precision.

- Confirms via boolean; frontend closes form and refreshes table on success.
- **SQL Executed**

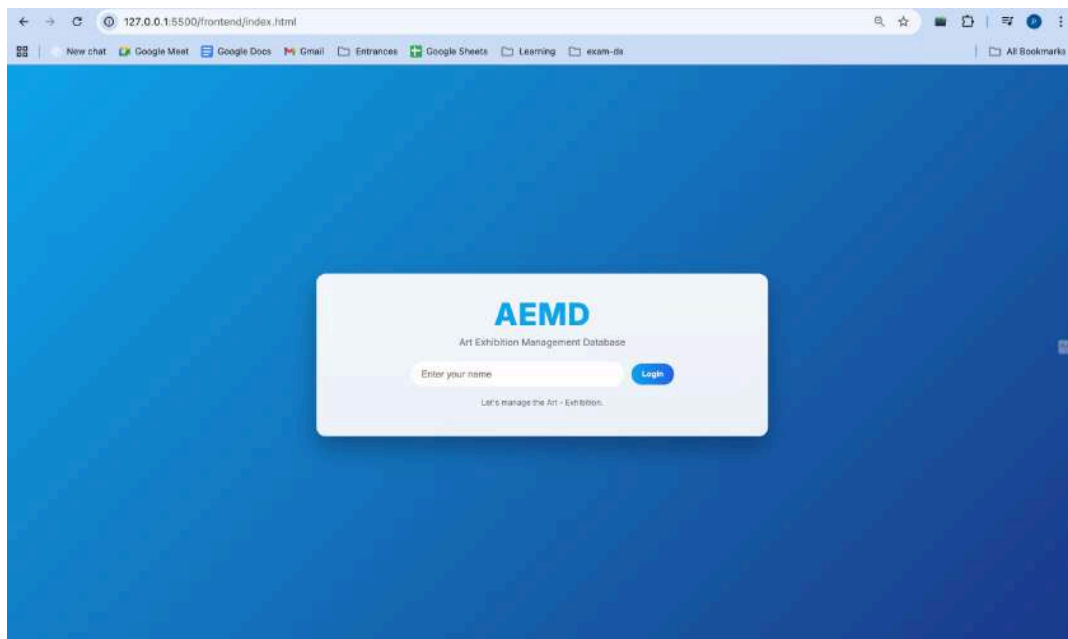
```
DELETE FROM notification WHERE notificationid = ?
```

- **Java Function (CRUDOperations.java)**

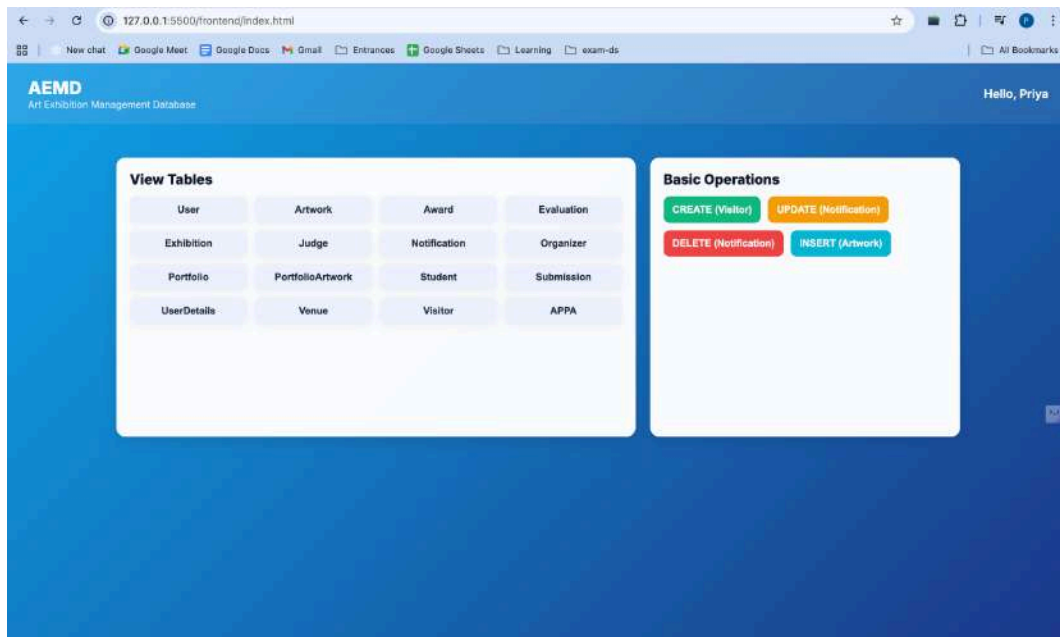
```
public static boolean updateNotificationStatus(int notificationid, String status) {  
    try (Connection conn = DatabaseConnection.getConnection();  
        PreparedStatement ps = conn.prepareStatement("UPDATE notification SET status = ? WHERE notificationid = ?")) {  
        ps.setString(1, status);  
        ps.setInt(2, notificationid);  
        return ps.executeUpdate() > 0;  
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return false;  
}
```

15.3. Frontend Screenshots

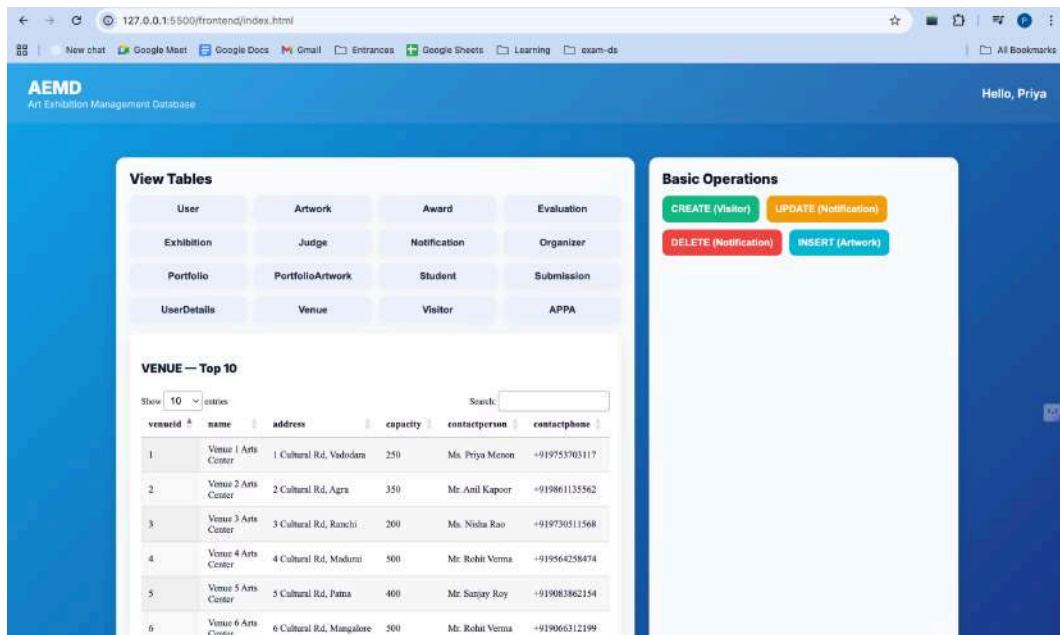
- **Login Page**



● Home Page



● Record of Venue Table



● Record of Submission Table

127.0.0.1:5500/frontend/index.html

New chatGoogle MeetGoogle DocsGmailEntrancesGoogle SheetsLearningexam-ds

100%Reset

All Bookmarks

AEMD

Art Exhibition Management Database

Hello, Priya

View Tables

User

Artwork

Award

Evaluation

Exhibition

Judge

Notification

Organizer

Portfolio

PortfolioArtwork

Student

Submission

UserDetails

Venue

Visitor

APPA

Basic Operations

CREATE (Visitor)

UPDATE (Notification)

DELETE (Notification)

INSERT (Artwork)

SUBMISSION — Top 10

Show10entries

Search:

submissionid	submissiondate	status	artworkid	exhibitionid	comments	userid
1	2024-12-19	Accepted	119	1	Auto-submission 1	390
2	2025-01-02	Rejected	130	1	Auto-submission 2	260
3	2024-12-25	Under Review	102	1	Auto-submission 3	11
4	2025-01-04	Under Review	163	1	Auto-submission 4	116
5	2024-12-22	Rejected	32	1	Auto-submission 5	184
6	2025-01-17	Rejected	89	2	Auto-submission 6	32

● Record of Evaluation Table

127.0.0.1:5500/frontend/index.html

New chatGoogle MeetGoogle DocsGmailEntrancesGoogle SheetsLearningexam-ds

All Bookmarks

AEMD

Art Exhibition Management Database

Hello, Priya

View Tables

User

Artwork

Award

Evaluation

Exhibition

Judge

Notification

Organizer

Portfolio

PortfolioArtwork

Student

Submission

UserDetails

Venue

Visitor

APPA

Basic Operations

CREATE (Visitor)

UPDATE (Notification)

DELETE (Notification)

INSERT (Artwork)

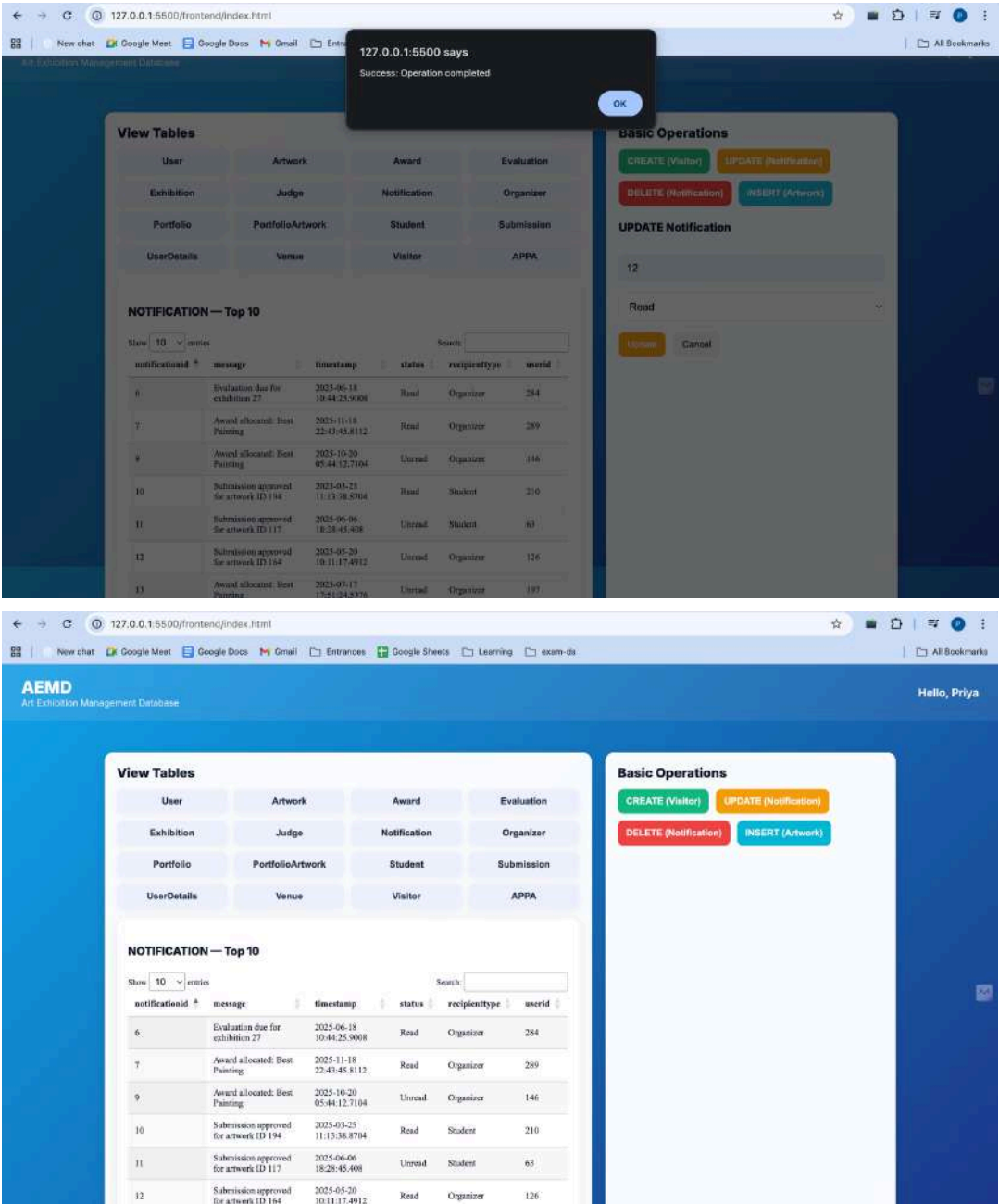
EVALUATION — Top 10

Show10entries

Search:

evaluationid	score	comments	evaluationdate	judgeid	artworkid	exhibitionid	submissionid
1	81.31	Compelling idea	2025-01-06	109	119	1	1
2	68.73	Needs refinement	2025-01-28	111	130	1	2
3	90.92	Good technique	2025-01-13	112	102	1	3
4	84.94	Good technique	2025-01-26	104	163	1	4
5	56.19	Work on contrast	2024-12-29	109	32	1	5
6	89.81	Needs refinement	2025-02-08	107	89	2	6

● Notification Update



● Delete Notification

The screenshot shows the AEMD (Art Exhibition Management Database) web application. A success notification from the browser console states: "127.0.0.1:5500 says Success: Operation completed". The main interface includes a "View Tables" section with a grid of database tables, a "Basic Operations" section with buttons for CREATE, UPDATE, DELETE, and INSERT, and a "DELETE Notification" modal. The modal displays a table of notifications, with the first row highlighted.

View Tables

User	Artwork	Award	Evaluation
Exhibition	Judge	Notification	Organizer
Portfolio	PortfolioArtwork	Student	Submission
UserDetails	Venue	Visitor	APPA

NOTIFICATION — Top 10

Show 10 entries

notificationid	message	timestamp	status	recipienttype	userid
6	Evaluation due for exhibition 27	2025-06-18 10:44:25.9008	Read	Organizer	284
7	Award allocated: Best Painting	2025-11-18 22:43:45.8112	Read	Organizer	289
9	Award allocated: Best Painting	2025-10-20 05:44:12.7104	Unread	Organizer	146
10	Submission approved for artwork ID 194	2025-03-25 11:13:38.8704	Read	Student	210
11	Submission approved for artwork ID 117	2025-06-06 18:26:45.4096	Unread	Student	63
12	Submission approved for artwork ID 164	2025-05-20 10:11:17.4912	Read	Organizer	126

DELETE Notification

11

Delete Cancel

● Insert Visitor

The screenshot shows the AEMD web application interface. The top navigation bar includes links for New chat, Google Meet, Google Docs, Gmail, Entrances, Google Sheets, Learning, exam-ds, and All Bookmarks. The main header displays 'AEMD Art Exhibition Management Database' and a user greeting 'Hello, Priya'.

View Tables

User	Artwork	Award	Evaluation
Exhibition	Judge	Notification	Organizer
Portfolio	PortfolioArtwork	Student	Submission
UserDetails	Venue	Visitor	APPA

Basic Operations

CREATE (Visitor) UPDATE (Notification)
DELETE (Notification) INSERT (Artwork)

Create Visitor

One-Day
11/11/2025
Good work.
4
Submit Cancel

VISITOR — Top 10

userid	tickettype	visitdate	feedback	feedbackratings
141	Multi-day	2025-01-19	Loved the student works.	5
142	VIP	2025-09-21	Enjoyed the workshops.	4
143	VIP	2025-07-05	Loved the student works.	4
144	VIP	2025-01-19	Great curation.	2
145	Multi-day	2025-04-01	Well organized.	2
146	Multi-day	2025-07-05	Helpful staff.	2
147	VIP	2025-04-14	Enjoyed the workshops.	4
148	VIP	2025-08-28	Helpful staff.	3

● Insert Artwork

The screenshot shows the AEMD web application interface. The top navigation bar includes links for New chat, Google Meet, Google Docs, Gmail, Entrances, Google Sheets, Learning, exam-ds, and All Bookmarks. The main header displays 'AEMD Art Exhibition Management Database' and a user greeting 'Hello, Priya'.

View Tables

User	Artwork	Award	Evaluation	Exhibition	Judge
Notification	Organizer	Portfolio	PortfolioArtwork	Student	Submission
UserDetails	Venue	Visitor	APPA		

Basic Operations

CREATE (Visitor) UPDATE (Notification)
DELETE (Notification) INSERT (Artwork)

Insert Artwork

Sunset
Digital Print
24x36
2025
https://example.com/art1.jpg
Generated artwork by student 10
10
For sale
Submit Cancel

ARTWORK — Top 10

artworkid	title	medium	dimensions	yearcreated	description	assigned	assignedid	interests
1	Work 1	Oil on Canvas	71x152 cm	2021	Generated artwork 1 by student 1	https://example.com/art1.jpg	1	1
2	Work 2	Oil on Canvas	37x148 cm	2022	Generated artwork 2 by student 1	https://example.com/art2.jpg	1	1
4	Work 4	Digital Print	8x24 cm	2024	Generated artwork 4 by student 2	https://example.com/art4.jpg	2	1
5	Work 5	Photograph	27x44 cm	2024	Generated artwork 5 by student 2	https://example.com/art5.jpg	2	1
6	Work 6	Oil on Canvas	44x117 cm	2023	Generated artwork 6 by student 3	https://example.com/art6.jpg	3	1
7	Work 7	Canvas	39x96 cm	2024	Generated artwork 7 by student 4	https://example.com/art7.jpg	4	1
8	Work 8	Watercolor	81x81 cm	2021	Generated artwork 8 by student 4	https://example.com/art8.jpg	4	1
9	Work 9	Digital Print	59x34 cm	2024	Generated artwork 9 by student 5	https://example.com/art9.jpg	5	1

15.4. Reflection / Conclusion

- During this project, I faced several backend integration challenges such as database connection failures, SQL exceptions, incorrect table mappings, and issues caused by outdated or cached servlet classes inside Tomcat.

- Troubleshooting these errors helped me understand how even small misconfigurations—like incorrect WAR deployment, stale class files, or unsafe SQL queries—can break the full UI ↔ backend ↔ database flow.
- I learned the importance of validating table names, handling exceptions properly, using prepared statements, and ensuring that the UI sends clean, structured requests to the server.
- Overall, this project strengthened my understanding of how front-end interactions translate into backend logic and finally into database transactions, and how to maintain this pipeline securely and reliably.